



Red Hat OpenShift AI Self-Managed 3.5

Govern LLM access with Models-as-a-Service

Govern LLM access with Models-as-a-Service in Red Hat OpenShift AI Self-Managed

Red Hat OpenShift AI Self-Managed 3.5 Govern LLM access with Models-as-a-Service

Govern LLM access with Models-as-a-Service in Red Hat OpenShift AI Self-Managed

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

As a Red Hat OpenShift AI user, you can deploy Models-as-a-Service (MaaS) in Red Hat OpenShift AI Self-Managed to provide governed access to large language models across your organization.

Table of Contents

PREFACE	5
CHAPTER 1. DEPLOY AND MANAGE MODELS-AS-A-SERVICE	6
1.1. CONFIGURE MODELS-AS-A-SERVICE	6
1.2. PLATFORM AND OPERATOR PREREQUISITES FOR MODELS-AS-A-SERVICE	10
1.3. GATEWAY REQUIREMENTS FOR MODELS-AS-A-SERVICE	11
1.4. CONFIGURE TLS FOR MODELS-AS-A-SERVICE	11
1.5. CONFIGURE THE DATABASE SECRET FOR MODELS-AS-A-SERVICE	14
1.6. MAAS CONFIGURATION	15
1.7. DASHBOARD CONFIGURATION FOR MODELS-AS-A-SERVICE	15
1.8. OBSERVABILITY CONFIGURATION FOR MODELS-AS-A-SERVICE	16
1.9. VERIFY MODELS-AS-A-SERVICE DEPLOYMENT	16
1.10. DEPLOY MODELS WITH MODELS-AS-A-SERVICE	19
1.11. MODELS-AS-A-SERVICE SUBSCRIPTIONS	21
1.11.1. Subscription-based access control	21
1.11.2. Subscription properties	22
1.11.3. Priority levels	22
1.11.4. Recommendations for priority level	22
1.11.5. Configuration guidance	23
1.11.6. Relationship with authorization policies	23
1.12. PREPARE FOR MODELS-AS-A-SERVICE TIER-TO-SUBSCRIPTION MIGRATION	24
1.13. MIGRATE MODELS-AS-A-SERVICE FROM TIERS TO SUBSCRIPTIONS	27
1.13.1. Clean up the old Models-as-a-Service tier-based configuration	34
1.14. MANAGE MODELS-AS-A-SERVICE SUBSCRIPTIONS	36
1.14.1. Monitor MaaS subscriptions	36
1.14.2. Create a subscription for Models-as-a-Service	37
1.14.3. Edit a subscription	39
1.14.4. Delete a subscription	40
1.15. MANAGE MODELS-AS-A-SERVICE AUTHORIZATION POLICIES	41
1.15.1. Models-as-a-Service authorization policies	41
1.15.2. View authorization policies	43
1.15.3. Create an authorization policy	44
1.15.4. Edit an authorization policy	45
1.15.5. Delete a Models-as-a-Service authorization policy	46
1.16. MANAGE API KEYS FOR USERS	47
1.16.1. View API keys	47
1.16.2. Create an API key for a user	48
1.16.3. Revoke user API keys	49
1.16.4. Configure the API key expiration limit	50
1.17. MANAGE MODELS-AS-A-SERVICE BY USING THE CLI AND API	52
1.17.1. Models-as-a-Service API	52
1.17.1.1. API structure	52
1.17.1.2. Authentication methods	52
1.17.2. MaaS custom resource workflow	53
1.18. MONITOR MODELS-AS-A-SERVICE USAGE BY USING THE OBSERVABILITY DASHBOARD	53
1.18.1. Models-as-a-Service usage monitoring	53
1.18.2. Enable Kuadrant observability for Models-as-a-Service	57
1.18.3. Enable telemetry for Models-as-a-Service	59
1.18.4. View the Models-as-a-Service observability dashboard	60
1.18.5. Export usage data for cost attribution	62
1.19. CONFIGURE EXTERNAL OIDC AUTHENTICATION FOR MODELS-AS-A-SERVICE	63

1.19.1. External OIDC authentication for Models-as-a-Service	63
1.19.1.1. Authentication flow	63
1.19.1.2. Group-based access control	63
1.19.1.3. API key lifecycle	64
1.19.1.4. Use cases	64
1.19.2. Configure Models-as-a-Service for external OIDC users	64
1.20. MANAGE MAAS MULTI-TENANCY	67
1.20.1. Models-as-a-Service multi-tenancy	67
1.20.2. Multi-tenancy prerequisites for Models-as-a-Service	69
1.20.3. Provision a MaaS tenant	71
1.20.4. Verify a multitenant MaaS deployment	74
1.20.5. Grant MaaS tenant access	77
1.20.6. Delete a MaaS tenant	80
1.20.7. AITenant custom resource reference	82
1.20.8. MaaS tenant RBAC reference	87
1.20.9. MaaS multi-tenancy known limitations	88
1.21. CONFIGURE EXTERNAL MODELS FOR MODELS-AS-A-SERVICE	90
1.21.1. External models for Models-as-a-Service	90
1.21.2. Multi-provider API passthrough for external models	92
1.21.2.1. When to use passthrough	93
1.21.2.2. How to enable passthrough	93
1.21.2.3. Vertex AI behavior differences	93
1.21.2.4. Limitations	94
1.21.3. Supported API formats and passthrough behavior for external models	94
1.21.3.1. Format detection	94
1.21.3.2. apiFormat field values	95
1.21.3.3. Passthrough decision matrix	96
1.21.3.4. Authentication headers per provider	96
1.21.3.5. ExternalProvider authentication types	97
1.21.3.6. Vertex AI provider configuration	97
1.21.3.7. Body-based model routing	98
1.21.3.8. Limitations	98
1.21.4. Configure routing to external model providers	99
1.21.5. Use single-URL passthrough with AI coding tools	107
1.22. MODELS-AS-A-SERVICE ADMINISTRATION TROUBLESHOOTING	110
1.22.1. Component enablement issues	110
1.22.2. Dashboard visibility issues	111
1.22.3. Model visibility issues	112
1.22.4. User access errors: 403 Forbidden	112
1.22.5. Subscription access control issues	113
1.22.6. Subscription management issues	114
1.22.7. Subscription phase shows Failed	114
CHAPTER 2. USE MODELS THROUGH MODELS-AS-A-SERVICE	116
2.1. FIND MODELS-AS-A-SERVICE MODELS IN THE DASHBOARD	116
2.2. ACCESS MODELS THROUGH MODELS-AS-A-SERVICE	117
2.2.1. Your token limits and access levels	117
2.2.2. View MaaS subscription limits	117
2.2.3. View your Models-as-a-Service subscriptions	118
2.2.4. Generate a temporary API key	119
2.2.5. Make API calls to models	120
2.2.6. Test models in a Jupyter notebook	123
2.2.7. Token limit responses	127

2.2.8. Test models in the playground	128
2.2.9. View your API keys	130
2.2.10. Create an API key	130
2.2.11. Revoke your API key	132
2.2.12. Best practices for Models-as-a-Service	133
2.2.12.1. API key security	133
2.2.12.2. Quota management	134
2.2.12.3. Token optimization	134
2.2.12.4. Performance and reliability	134
2.2.12.5. Testing and development	134
2.2.12.6. Multi-subscription usage	135
2.2.12.7. Endpoint selection	135
2.2.12.8. Production deployment	135
2.3. MODELS-AS-A-SERVICE USER ACCESS TROUBLESHOOTING	136
2.3.1. Authentication errors: 401 Unauthorized	136
2.3.2. Authorization errors: 403 Forbidden	136
2.3.3. Model not found: 404	137
2.3.4. Exceeded token limits	137
2.3.5. Persistent token limit errors	138

PREFACE

Deploy, manage, and govern access to large language models by using the Models-as-a-Service platform in OpenShift AI.

CHAPTER 1. DEPLOY AND MANAGE MODELS-AS-A-SERVICE

You can deploy Models-as-a-Service (MaaS) to provide subscription-based governance for large language model serving. With MaaS, you can define subscriptions that grant groups access to models with token limits, control access through API key authentication, and track resource consumption for cost allocation.

1.1. CONFIGURE MODELS-AS-A-SERVICE

In Red Hat OpenShift AI, Models-as-a-Service (MaaS) provides subscription-based governance for large language model (LLM) serving across your organization. This platform helps you manage resource consumption and governance challenges when you serve models to a large user base.



NOTE

In OpenShift AI 3.4, MaaS uses a subscription-based model for quota management. This replaces the tier-based model used in OpenShift AI 3.3. Subscriptions provide enhanced flexibility and are managed through custom resources instead of ConfigMaps.

As an administrator, you can use this subscription-based system to expose models through managed API endpoints. With this structure, you can enforce different consumption policies for different user groups and deliver AI models as shared resources with appropriate access levels.

The Models-as-a-Service platform acts as a governance layer between users and model serving infrastructure. You can enforce centralized policies without modifying the underlying model serving components.

Models-as-a-Service provides the following capabilities:

Subscription-based quota management

Define multiple subscriptions that grant specific groups quota for models with configurable token limits. Users can belong to multiple subscriptions, with priority levels determining which subscription is used when multiple options are available.

Self-service API key management

Users can create, list, and revoke their own API keys for model access. Administrators can also provision and manage API keys on behalf of users. API keys can be permanent or configured with custom expiration times, and individual keys can be revoked without affecting other keys for the same user.

OpenAI API compatibility

Call models by using the standard OpenAI **/v1/chat/completions** endpoint with the model name in the request body. This enables drop-in compatibility with OpenAI-compatible SDKs and clients such as the Python **openai** library, LangChain, and LlamaIndex without requiring custom URL paths. Legacy path-based routing is also supported for backwards compatibility.

Multi-runtime support

Expose models served with llm-d or vLLM runtimes through MaaS governance. You can apply consistent governance across different serving infrastructures. vLLM runtime support is a Technology Preview feature in Red Hat OpenShift AI.

Policy and quota management

Enforce token limit policies to prevent resource exhaustion.

Usage tracking and observability

Monitor subscription-level token consumption, request counts, and rate-limit violations through the MaaS observability dashboard. Track consumption metrics for cost allocation and billing. Export usage data in CSV format for cost attribution and showback reporting to finance teams. The MaaS observability dashboard is a Technology Preview feature in Red Hat OpenShift AI.

External models

Route inference requests to models hosted by external cloud providers such as AWS Bedrock, Azure OpenAI, or Google Vertex AI through the same MaaS gateway used for locally deployed models. External models is a Technology Preview feature in Red Hat OpenShift AI.

Multi-provider API passthrough

Route requests in native provider API formats such as the Anthropic Messages API or the OpenAI Responses API without format translation, preserving provider-specific features such as prompt caching and extended thinking. AI coding tools can connect to a single MaaS gateway URL and switch between models mid-session by using body-based model routing. Multi-provider API passthrough is a Technology Preview feature in Red Hat OpenShift AI.

Multi-tenancy

Provision isolated tenants for Models-as-a-Service where each tenant receives a dedicated gateway, identity realm, namespace, and API infrastructure. Multi-tenancy is a Technology Preview feature in Red Hat OpenShift AI.

External OpenID Connect (OIDC) authentication

Integrate with external OIDC identity providers for user authentication to provide enterprise-wide access without requiring OpenShift user accounts.

The following table summarizes when MaaS is the right choice and when standard model serving is sufficient.

Table 1.1. When to use MaaS compared to standard model serving

MaaS	Standard model serving
Centralized governance across multiple teams or projects is required.	You are deploying models for single-team or single-user use cases.
You need token limit enforcement and usage tracking for cost control.	You are prototyping or developing models in a single-user environment where additional governance is unnecessary.
You prefer declarative configuration management via GitOps.	Simplified deployment is preferred over centralized control.

MaaS administration is divided into initial configuration and ongoing management, with distinct responsibilities for cluster administrators and OpenShift AI administrators.

Table 1.2. MaaS administrator responsibilities

Phase	Cluster administrators	OpenShift AI administrators
-------	------------------------	-----------------------------

Phase	Cluster administrators	OpenShift AI administrators
Initial configuration	• Enable MaaS in the OpenShift AI operator • Configure the underlying cluster infrastructure to support model serving	• Define the initial governance structure by creating subscriptions and authorization policies • Assign users to groups • Configure model quota and token limits for each subscription • Validate that users can successfully access models through MaaS
Ongoing operations	• Scale MaaS components to handle increased load • Apply software updates • Troubleshoot infrastructure performance issues	• Monitor usage metrics to track costs • Adjust subscription configurations and user group assignments • Modify token limits based on demand patterns • Manage API keys for external consumers or assist users with key lifecycle management • Troubleshoot authentication and authorization issues

Models-as-a-Service custom resources:

Models-as-a-Service (MaaS) uses Kubernetes custom resources for declarative configuration management. You can integrate MaaS with GitOps workflows and version control. The platform uses the following custom resource types:

AITenant

Bootstraps an isolated MaaS tenant from the infrastructure namespace. Each **AITenant** creates a dedicated tenant namespace, deploys a per-tenant **maas-api** instance, configures gateway-scoped authentication policies, and creates tenant-admin RBAC Roles. **AITenant** is a Technology Preview feature.

MaasTenantConfig

Stores per-tenant runtime configuration including API key expiration settings, telemetry options, and resolved gateway and OIDC references. The controller creates one **MaasTenantConfig** CR named **default-tenant** in each tenant namespace.

Tenant

Deprecated. Configures tenant-specific settings including API key expiration limits, external OIDC authentication, telemetry options, and gateway references. The **Tenant** CR is being replaced by **AITenant** and **MaasTenantConfig**.

MaaSModelRef

References inference servers served through OpenShift AI. Models can be served using llm-d distributed inference, vLLM runtimes, or external LLM providers.

ExternalProvider

Configures an external LLM provider endpoint, authentication method, and credentials. Multiple **ExternalModel** resources can reference the same **ExternalProvider**. The controller automatically creates the required networking resources: **Service**, **ServiceEntry**, and **DestinationRule**.

ExternalModel

Defines a client-facing model name mapped to one or more external providers through **externalProviderRefs**. Each provider reference specifies the target model identifier, **apiFormat** for request translation or passthrough, and an outgoing request path. You can apply MaaS governance to third-party LLM services.

MaaSSubscription

Defines subscription-based quota by specifying which groups have quota for which models with configurable token rate limits. Subscriptions include priority levels for users belonging to multiple groups and optional metadata for cost allocation.

MaaSAuthPolicy

Authorizes groups to access model endpoints through the API gateway. Subscriptions control token limits, while authorization policies grant API gateway access.

With these custom resources, administrators can manage MaaS configurations by using standard Kubernetes tools and GitOps workflows. Changes to custom resources are automatically reconciled by the platform controllers.

You can view and manage these custom resources by using the OpenShift console or OpenShift CLI (**oc**).

Using the console:

Navigate to **Administration** → **CustomResourceDefinitions** and search for the resource name.

Using the CLI:

List resource instances:

```
$ oc get maassubscriptions -n models-as-a-service
$ oc get maasmodelrefs -n <namespace>
$ oc get externalproviders.inference.opendatahub.io -n <namespace>
$ oc get externalmodels.inference.opendatahub.io -n <namespace>
$ oc get aitenants -n ai-tenants
$ oc get maastenantconfigs.maas.opendatahub.io -n <tenant_namespace>
$ oc get tenants.maas.opendatahub.io -n models-as-a-service
$ oc get externalproviders.inference.opendatahub.io -n <namespace>
$ oc get externalmodels.inference.opendatahub.io -n <namespace>
```

View the YAML configuration of a specific resource:

```
$ oc get maassubscription <subscription_name> -n models-as-a-service -o yaml
$ oc get maastenantconfigs.maas.opendatahub.io default-tenant -n <tenant_namespace> -o yaml
$ oc get tenants.maas.opendatahub.io default-tenant -n models-as-a-service -o yaml
$ oc get externalprovider <provider_name> -n <namespace> -o yaml
$ oc get externalmodel <model_name> -n <namespace> -o yaml
```

Additional resources

- [Models-as-a-Service multi-tenancy overview](#)

1.2. PLATFORM AND OPERATOR PREREQUISITES FOR MODELS-AS-A-SERVICE

Before deploying Models-as-a-Service (MaaS) in Red Hat OpenShift AI, verify that your cluster has the required platform components, Operators, and infrastructure resources.

Platform and access requirements:

- You have a cluster with OpenShift version 4.19.9 or later.
- You have cluster administrator access to install Operators and create cluster-scoped resources.
- Your cluster has a functional ingress controller with valid TLS certificates for external access.
- You have installed OpenShift CLI (**oc**).

Operator requirements:

- You have installed Red Hat OpenShift AI 3.4 or later.
- You have enabled distributed inference with llm-d, including authentication for LLM Inference Service.
- You have installed the Red Hat Connectivity Link Operator version 1.4.x to the **openshift-operators** namespace and created a **Kuadrant** custom resource in the **kuadrant-system** namespace with ready status.
- If you plan to use vLLM runtime with Models-as-a-Service, you have set **spec.dashboardConfig.vLLMDeploymentOnMaaS** to **true** in the **OdhDashboardConfig** custom resource. vLLM runtime for Models-as-a-Service is available as a Technology Preview feature. For more information, see [Technology Preview Features Support Scope](#).

To install the required operator subscriptions using example manifests and scripts, see [Prerequisites](#) in the Models-as-a-Service companion guide.

Infrastructure requirements:

- You have created a **DataScienceCluster** resource with the **kserve** component set to **Managed**.
- You have enabled User Workload Monitoring on your OpenShift cluster. User Workload Monitoring is required for MaaS to collect and expose usage metrics for token consumption, request counts, and rate limiting. Without User Workload Monitoring enabled, the MaaS installation shows a **Degraded** status. For information about enabling User Workload Monitoring, see [Enabling monitoring for user-defined projects](#).
- You have deployed a PostgreSQL database instance, version 14 or later, that is reachable from the OpenShift cluster network. This database is required for API key lifecycle management. OpenShift AI does not provide a PostgreSQL database. You must provision and manage your own PostgreSQL instance.

To configure Kuadrant, Authorino, and User Workload Monitoring using example manifests and scripts, see [Platform configuration](#) in the Models-as-a-Service companion guide.

Additional resources

- [Gateway requirements for Models-as-a-Service](#)
- [Models-as-a-Service companion guide](#)
- [Enabling monitoring for user-defined projects](#)

1.3. GATEWAY REQUIREMENTS FOR MODELS-AS-A-SERVICE

Models-as-a-Service (MaaS) requires a configured Gateway API infrastructure for routing and securing model endpoints. You must create the required gateway resources and configure TLS before deploying MaaS.

GatewayClass and Gateway:

- You have created a **GatewayClass** resource configured for the OpenShift Gateway Controller (**openshift.io/gateway-controller/v1**), or you are using the pre-existing **openshift-default** GatewayClass.
- You have created a **Gateway** named **maas-default-gateway** in the **openshift-ingress** namespace. The **Gateway** resource must include the following annotations:
 - **opendatahub.io/managed: "false"** - Prevents the ODH Model Controller from overriding MaaS-managed authorization policies.
 - **security.opendatahub.io/authorino-tls-bootstrap: "true"** - Enables TLS communication between the **Gateway** and **Authorino**.

To configure **GatewayClass**, **Gateway**, and Authorino by using example manifests and scripts, see [Platform configuration](#) in the Models-as-a-Service companion guide.



NOTE

In multitenant deployments, each tenant requires a dedicated Gateway with a unique hostname. The **maas-default-gateway** serves only the default tenant. Each additional tenant Gateway must include the same required annotations and use the **openshift-default** GatewayClass. For per-tenant Gateway configuration, see [Multi-tenancy prerequisites for Models-as-a-Service](#).

Additional resources

- [Configure TLS for Models-as-a-Service](#)
- [Models-as-a-Service companion guide](#)
- [Enabling the Gateway API](#)

1.4. CONFIGURE TLS FOR MODELS-AS-A-SERVICE

To enable secure authentication and authorization for model endpoints, you must configure TLS communication between **Authorino** and the Models-as-a-Service (MaaS) API service.

Prerequisites

- You have installed the Red Hat Connectivity Link Operator and created a **Kuadrant** custom resource.
- You have access to OpenShift CLI (**oc**).
- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.

Procedure

1. Annotate the **Authorino** service to enable service serving certificate generation in OpenShift:

```
$ oc annotate service authorino-authorino-authorization \
  -n kuadrant-system \
  service.beta.openshift.io/serving-cert-secret-name=authorino-server-cert \
  --overwrite
```

The service-ca-operator generates a TLS certificate signed by the cluster service CA and stores it in the **authorino-server-cert** secret.

2. Patch the **Authorino** custom resource to enable the TLS listener:

```
$ oc patch authorino authorino -n kuadrant-system --type=merge --patch '
{
  "spec": {
    "listener": {
      "tls": {
        "enabled": true,
        "certSecretRef": {
          "name": "authorino-server-cert"
        }
      }
    }
  }
}'
```

Authorino uses the generated certificate for inbound TLS communication.

3. Configure the **Authorino** deployment with environment variables for TLS certificate validation:

```
$ oc -n kuadrant-system set env deployment/authorino \
  SSL_CERT_FILE=/etc/ssl/certs/openshift-service-ca/service-ca-bundle.crt \
  REQUESTS_CA_BUNDLE=/etc/ssl/certs/openshift-service-ca/service-ca-bundle.crt
```

The cluster CA bundle is automatically populated by the service-ca-operator in OpenShift.

4. Annotate your **Gateway** resource to enable automatic TLS configuration:

```
$ oc annotate gateway maas-default-gateway \
  -n openshift-ingress \
```

```
security.openshift.io/authorino-tls-bootstrap="true" \
--overwrite
```

The MaaS controller detects this annotation and creates an **EnvoyFilter** resource that configures the Envoy proxy to use TLS when communicating with **Authorino**.

Verification

- Verify that the **Authorino** service has the serving certificate annotation:

```
$ oc get service authorino-authorino-authorization \
-n kuadrant-system \
-o jsonpath='{.metadata.annotations.service\beta.openshift.io/serving-cert-secret-name}'
```

Expected output:

```
authorino-server-cert
```

- Verify that the **authorino-server-cert** secret exists:

```
$ oc get secret authorino-server-cert -n kuadrant-system
```

Expected output:

NAME	TYPE	DATA	AGE
authorino-server-cert	kubernetes.io/tls	2	5m

- Verify that the **Authorino** CR has TLS enabled:

```
$ oc get authorino authorino -n kuadrant-system -o jsonpath='{.spec.listener.tls.enabled}'
```

Expected output:

```
true
```

- Verify that the **Authorino** deployment has the TLS certificate environment variables configured:

```
$ oc get deployment/authorino -n kuadrant-system -o
jsonpath='{.spec.template.spec.containers[0].env[?(@.name=="SSL_CERT_FILE")].value}'
```

Expected output:

```
/etc/ssl/certs/openshift-service-ca/service-ca-bundle.crt
```

- Verify that the **Gateway** has the TLS bootstrap annotation:

```
$ oc get gateway maas-default-gateway -n openshift-ingress \
-o jsonpath='{.metadata.annotations.security\openshift.io/authorino-tls-bootstrap}'
```

Expected output:

■

■ true

Next steps

- [Configure the database secret for Models-as-a-Service](#)

1.5. CONFIGURE THE DATABASE SECRET FOR MODELS-AS-A-SERVICE

You must create a secret that contains your PostgreSQL database connection details. This database is required for API key lifecycle management in Models-as-a-Service.

Prerequisites

- You have deployed a PostgreSQL database instance, version 14 or later, that is reachable from the OpenShift cluster network. OpenShift AI does not provide a PostgreSQL database. You must provision and manage your own PostgreSQL instance before proceeding.
- You have access to OpenShift CLI (**oc**).
- You have permissions to create secrets in the infrastructure namespace. The default infrastructure namespace is **redhat-ai-gateway-infra** for OpenShift AI 3.5 and later.



NOTE

In OpenShift AI 3.5 and later, the **maas-db-config** secret is stored in the infrastructure namespace instead of the **{dbd-config-default-namespace}** namespace. To determine the infrastructure namespace for your deployment, query the **MaasTenantConfig** status:

```
$ oc get maastenantconfig default-tenant -n models-as-a-service \
  -o jsonpath='{.status.infraNamespace}'
```

Procedure

1. Create the **maas-db-config** secret in the infrastructure namespace:

```
$ oc create secret generic maas-db-config \
  -n <infrastructure_namespace> \
  --from-literal=DB_CONNECTION_URL=postgresql://<username>:<password>@<hostname>:<port>/<database>?sslmode=require
```

where:

<infrastructure_namespace>

Specifies the infrastructure namespace. The default is **redhat-ai-gateway-infra** for OpenShift AI 3.5 and later.

<username>

Specifies the PostgreSQL database username.

<password>

Specifies the PostgreSQL database password.

<hostname>

Specifies the hostname or IP address of the PostgreSQL server.

<port>

Specifies the port number for the PostgreSQL server, typically **5432**.

<database>

Specifies the name of the PostgreSQL database.

The following example shows a complete connection string:

```
postgresql://maasadmin:XXXXX@pg.example.com:5432/maasdb?sslmode=require
```

- Optional: Restart the **maas-api** deployment to apply the configuration if **modelsAsService** is already set to **Managed** in the **DataScienceCluster** resource:

```
$ oc rollout restart deployment/maas-api -n <infrastructure_namespace>
```

This step is not required if the secret exists before you enable **modelsAsService** in the **DataScienceCluster** resource.

Verification

- Verify that the **maas-db-config** secret exists in the infrastructure namespace:

```
$ oc get secret maas-db-config -n <infrastructure_namespace>
```

Expected output:

NAME	TYPE	DATA	AGE
maas-db-config	Opaque	1	5s

Next steps

- [MaaS configuration](#)

1.6. MAAS CONFIGURATION

To enable Models-as-a-Service (MaaS), you must configure the **DataScienceCluster** custom resource.

- Set **spec.components.kserve.modelsAsService.managementState** to **Managed** in the **DataScienceCluster** custom resource.

Additional resources

- [Dashboard configuration for Models-as-a-Service](#)

1.7. DASHBOARD CONFIGURATION FOR MODELS-AS-A-SERVICE

To access Models-as-a-Service (MaaS) features in the OpenShift AI dashboard, you must enable the required dashboard configuration flags in the **OdhDashboardConfig** custom resource.

- Set **spec.dashboardConfig.modelAsService** to **true** in the **OdhdashboardConfig** custom resource.

To access MaaS user-facing features in the dashboard:

- Set **spec.components.ogx.managementState** to **Managed** in the **DataScienceCluster** custom resource.
- Set **spec.dashboardConfig.genAiStudio** to **true** in the **OdhdashboardConfig** custom resource.

To access MaaS administrative features in the dashboard:

- Set **spec.dashboardConfig.maasAuthPolicies** to **true** in the **OdhdashboardConfig** custom resource.

To view external model endpoints in the dashboard:

- Set **spec.dashboardConfig.externalModels** to **true** in the **OdhdashboardConfig** custom resource. External models is a Technology Preview feature.

Additional resources

- [Observability configuration for Models-as-a-Service](#)

1.8. OBSERVABILITY CONFIGURATION FOR MODELS-AS-A-SERVICE

To enable the MaaS observability dashboard for usage monitoring, you must configure the following settings.

- Set **spec.dashboardConfig.observabilityDashboard** to **true** in the **OdhdashboardConfig** custom resource.
- Configure observability for OpenShift AI, including metrics storage in the **DSCInitialization** resource and the Cluster Observability Operator. For complete setup instructions, see [Managing observability](#).

To install the Observability Operators and configure gateway telemetry using example manifests and scripts, see [Observability](#) in the Models-as-a-Service companion guide.



NOTE

The MaaS observability dashboard is a Technology Preview feature in Red Hat OpenShift AI. For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

1.9. VERIFY MODELS-AS-A-SERVICE DEPLOYMENT

After you deploy Models-as-a-Service, you can run a series of checks to confirm that the required custom resources, monitoring components, and tenant configuration are in place.

Prerequisites

- You have deployed Models-as-a-Service.

- You have access to OpenShift CLI (**oc**).
- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.

Procedure

1. Verify that the Models-as-a-Service (MaaS) custom resource definitions (CRDs) are installed:

```
$ oc get crd | grep -E 'maas.opendatahub.io|aitenants'
```

Expected output shows the following CRDs:

```
aitenants.maas.opendatahub.io
maasauthpolicies.maas.opendatahub.io
maasmodelrefs.maas.opendatahub.io
maassubscriptions.maas.opendatahub.io
maastenantconfigs.maas.opendatahub.io
tenants.maas.opendatahub.io
```

2. Verify that User Workload Monitoring is enabled on the cluster:

```
$ oc get namespace openshift-user-workload-monitoring
```

If the namespace exists, User Workload Monitoring is enabled. If the namespace is not found, User Workload Monitoring is not enabled and the MaaS deployment might show as **Degraded**.

3. Verify that the **Tenant** custom resource exists in the **models-as-a-service** namespace:

```
$ oc get tenants.maas.opendatahub.io -n models-as-a-service
```

Expected output shows at least one **Tenant** resource:

NAME	READY	REASON	AGE
default-tenant	True	Reconciled	5m

4. Check the status of the **Tenant** custom resource:

```
$ oc get tenants.maas.opendatahub.io default-tenant -n models-as-a-service -o
jsonpath='{.status.conditions[?(@.type=="Ready")].status}'
```

Expected output:

```
True
```

The following values indicate the deployment status:

True

Indicates that the MaaS deployment is successful and all prerequisites are met.

False or Degraded

Indicates missing prerequisites or configuration issues. Check the condition message for details:

```
$ oc get tenants.maas.opendatahub.io default-tenant -n models-as-a-service -o
jsonpath='{.status.conditions[?(@.type=="Ready")].message}'
```

5. Verify that the **Tenant** custom resource shows a configured state:

```
$ oc get tenants.maas.opendatahub.io default-tenant -n models-as-a-service
```

Expected output:

NAME	READY	REASON	AGE
default-tenant	True	Reconciled	5m

The **READY** column shows **True** and the **REASON** column shows **Reconciled**, confirming that the tenant is fully configured.

6. Optional: If you plan to use external models, verify that the **ExternalModel** CRD is available:

```
$ oc get crd externalmodels.maas.opendatahub.io
```

Expected output shows the CRD details:

NAME	CREATED AT
externalmodels.maas.opendatahub.io	2026-04-28T10:15:30Z

Verification

- All MaaS CRDs are deployed and available.
- User Workload Monitoring is enabled on the cluster.
- The **Tenant** custom resource shows a **Ready** status with reason **Reconciled**.

Troubleshooting

If the **Tenant** status shows **False** or **Degraded**:

- Verify that User Workload Monitoring is enabled on the cluster.
- Verify that all platform, Operator, gateway, and configuration prerequisites are met. For more information, see [Platform and Operator prerequisites for Models-as-a-Service](#).
- Check that the PostgreSQL database secret **maas-db-config** exists in the infrastructure namespace. In OpenShift AI 3.5 and later, the default infrastructure namespace is **redhat-ai-gateway-infra**. To determine the infrastructure namespace for your deployment, run **oc get maas-tenant-config default-tenant -n models-as-a-service -o jsonpath='{.status.infraNamespace}'**.
- Verify that Red Hat Connectivity Link is installed and the **Kuadrant** custom resource is ready.
- Review the **Tenant** condition messages for specific error details.

Next steps

- To optionally test your MaaS deployment using a model simulator, see [MaaS models](#) and [Verification](#) in the Models-as-a-Service companion guide.

Additional resources

- [Enabling monitoring for user-defined projects](#)
- [Models-as-a-Service companion guide](#)

1.10. DEPLOY MODELS WITH MODELS-AS-A-SERVICE

In Red Hat OpenShift AI, you can deploy generative AI models through the dashboard wizard and publish them to Models-as-a-Service (MaaS) so that administrators can enforce subscription-based quota and token limits.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- You have administrator access to a project in OpenShift AI.
- Your cluster administrator has installed the required Operators and infrastructure for Models-as-a-Service. For more information, see [Platform and Operator prerequisites for Models-as-a-Service](#).
- If you plan to use distributed inference with llm-d, your cluster administrator has enabled distributed inference, including authentication for LLM Inference Service.
- If you plan to use vLLM runtime with Models-as-a-Service, your cluster administrator has enabled the vLLM deployment on MaaS feature flag:
spec.dashboardConfig.vLLMDeploymentOnMaaS: true in the **OdhDashboardConfig** custom resource. vLLM runtime support for Models-as-a-Service is a Technology Preview feature.

Procedure

1. In the left navigation menu, click **Projects**.
2. Click the name of the project where you want to deploy the model.
3. Click the **Deployments** tab.
4. Click **Deploy model** to open the wizard.
5. In the **Model details** section, specify your storage connection and model path.
6. Complete the model configuration based on whether **vLLMDeploymentOnMaaS** is enabled in the **OdhDashboardConfig** custom resource:

If vLLMDeploymentOnMaaS is enabled

- a. In the **Model details** section, select **Generative AI model (Example, LLM)** as the model type.
- b. Make sure that the **Use legacy deployment method** checkbox is unchecked.

- c. Click **Next**.
- d. In the **Model deployment** section, enter a unique model deployment name using lowercase letters, numbers, and hyphens.
- e. Select an appropriate hardware profile for your model.
- f. Select one of the following deployment resources:
 - **Distributed inference with llm-d** for distributed inference support.
 - A vLLM-based **LLMInferenceServiceConfig**, such as **vLLM NVIDIA CUDA GPU LLMInferenceServiceConfig**, for vLLM-based serving as a Technology Preview feature in Red Hat OpenShift AI.

If vLLMDeploymentOnMaaS is not enabled

- a. In the **Model details** section, select **Generative AI model (Example, LLM)** as the model type.
- b. Click **Next**.
- c. In the **Model deployment** section, enter a unique model deployment name using lowercase letters, numbers, and hyphens.
- d. Select an appropriate hardware profile for your model.
- e. Select **Distributed inference with llm-d** as the deployment resource.



NOTE

Models deployed for MaaS use the **LLMInferenceService** architecture, which is designed for large language models and integrates with the MaaS gateway for subscription-based quota enforcement. The legacy deployment method uses traditional KServe **InferenceService** resources with serving runtimes.

7. In the **Model deployment** section, in the **Number of replicas** field, enter the number of replicas to deploy. The default is 1. For production workloads, consider deploying at least 2 replicas for high availability.
8. Click **Next**.
9. In the **Advanced settings** section, configure MaaS publishing:
 - a. Under **Model availability**, select **Publish as MaaS** to make the model accessible to users through the MaaS gateway.



NOTE

Publishing as MaaS creates a **MaaSModelRef** object that registers the model with MaaS for subscription assignment. After publishing, an administrator must create a subscription and add this model to make it accessible to user groups.

- b. Optional: Select **Add custom runtime arguments** or **Add custom runtime environment variables** to customize model behavior.
 - c. Click **Next**.
10. In the **Review** section, verify your configuration settings:
- a. Review the model details, deployment configuration, and advanced settings.
 - b. Click **Deploy model**.

Verification

- Verify that the model appears on the **Deployments** tab with a checkmark in the **Status** column.
- Verify that the model was published to MaaS:

```
$ oc get maasmodelref -n <your-project-namespace>
```

You should see a **MaaSModelRef** object for your deployed model.



NOTE

Models published to MaaS require subscription and authorization policy configuration before users can access them.

Additional resources

- [Create a subscription](#)
- [Create an authorization policy](#)
- [Models-as-a-Service administration troubleshooting](#)

1.11. MODELS-AS-A-SERVICE SUBSCRIPTIONS

In Red Hat OpenShift AI, you can use Models-as-a-Service (MaaS) subscriptions to manage quotas and token limits for AI model serving. With subscriptions, you can grant specific groups quotas for models with configurable token limits based on user group membership.



NOTE

In OpenShift AI 3.3, MaaS used a tier-based model for access control. Starting with OpenShift AI 3.4, tiers have been replaced with subscriptions. The subscription model provides more flexibility by allowing users to belong to multiple subscriptions and uses custom resource definition (CRD)-based configuration for improved GitOps compatibility.

1.11.1. Subscription-based access control

When multiple teams share large language models, you can use subscriptions to perform the following tasks:

- Prevent resource exhaustion by enforcing token limits per model

- Provide different access levels for different user groups
- Track and allocate costs based on team consumption
- Control which teams can access high-cost or sensitive models
- Allow users to belong to multiple subscriptions based on their group memberships

MaaS assigns users to subscriptions based on their OpenShift group membership. When a user belongs to multiple groups with different subscriptions, the system uses the subscription with the highest priority level.

1.11.2. Subscription properties

MaaS subscriptions are defined as **MaaSSubscription** custom resources in the cluster. Each subscription has the following properties:

Name

A unique identifier for the subscription that becomes the Kubernetes resource name.

Description

An optional human-readable description shown in the dashboard.

Groups

One or more groups whose members can access this subscription. Groups can come from OpenShift Group objects or external OIDC providers. Users can belong to multiple groups and therefore have access to multiple subscriptions.

Priority level

A numeric value that determines subscription precedence when creating an API key without specifying a subscription. Higher numbers indicate higher priority, with 0 as the lowest. Priority only applies during API key creation.

Models

A list of models that this subscription gives quota for, with configurable token limits for each model.

1.11.3. Priority levels

Priority levels determine which subscription is selected when creating an API key without explicitly specifying a subscription. When a user belongs to multiple groups with different subscriptions, the subscription with the highest priority is selected as the default.

For example, if a user belongs to both the **analytics-team** group with priority 1 and the **production-apps** group with priority 2, creating a key without specifying a subscription selects the **production-apps** subscription because it has the higher priority.

When creating API keys, specifying the subscription explicitly bypasses priority selection.

1.11.4. Recommendations for priority level

Use a consistent priority numbering scheme to make subscription precedence clear and maintainable.

The recommended priority scheme is as follows:

Production workloads: 100

Use this priority for customer-facing applications, production APIs, and critical business processes.

Staging and pre-production: 50

Use this priority for QA testing, user acceptance testing, and performance testing.

Development and experimentation: 0

Use this priority for exploratory data science work, prototype development, and learning. This is the default value.

Personal and sandbox: -10

Use this priority for individual experimentation, tutorials, and non-business use.

Here are common use cases:

Separate production and development resources

Create a production subscription with priority 100 for stricter quotas billed to production cost centers, and a development subscription with priority 0 for generous quotas billed to R&D. Production applications automatically use the production subscription, while developers must explicitly select the development subscription for testing.

Team-based access with overlapping membership

When users belong to multiple teams, assign higher priority to broader access. For example, set "ML Platform Team" to priority 10 for access to all models and "Analytics Team" to priority 5 for analytics-focused models. Users in both teams default to the ML Platform Team subscription.

Cost-tiered model access

Create a "Standard Models" subscription with priority 10 for cheaper models with higher quotas, and a "Premium Models" subscription with priority 0 for expensive models with limited quotas. Users consume cheaper resources by default and must explicitly select the premium subscription when needed.

1.11.5. Configuration guidance

- Use incremental priority values with reasonable gaps such as 10, 20, 30 rather than 0, 1000, 10000. This makes it easier to insert intermediate priorities later.
- Avoid setting all subscriptions to the same priority, which creates unpredictable behavior.
- Use priority for convenience and defaults, not for access control. Use authorization policies to restrict which users can access which models.

Token limits

Tokens are the basic units of text processing in large language models. Token limits control the maximum number of tokens that can be consumed per request or time period for a specific model.

Configure token limits for each model when you create or edit a subscription through the dashboard.

Each model in a subscription can have different token limit configurations, allowing administrators to provide varying levels of access to different models within the same subscription.

1.11.6. Relationship with authorization policies

Subscriptions and authorization policies work together to control model access in the following ways:

- **Subscriptions** give groups quota for specific models with token rate limits.
- **Authorization policies** grant groups access to model endpoints through the API gateway.

Both are required for users to access models through MaaS. A subscription defines quota limits for model usage, while an authorization policy enables API gateway access.

When you create a subscription, you can optionally create a matching authorization policy by selecting the **Create matching authorization policy** checkbox. The authorization policy uses the same groups and models as the subscription, so users can access the models as soon as the subscription is created.

Additional resources

- [Manage Models-as-a-Service authorization policies](#)

1.12. PREPARE FOR MODELS-AS-A-SERVICE TIER-TO-SUBSCRIPTION MIGRATION

If you used Models-as-a-Service (MaaS) as a Technology Preview feature in OpenShift AI 3.3, you must prepare your environment before upgrading to OpenShift AI 3.4 or later. Starting with OpenShift AI 3.4, tiers are replaced by subscriptions. Complete this procedure before you upgrade the OpenShift AI Operator.



IMPORTANT

This procedure disables MaaS, which causes downtime. Users cannot access models through MaaS after you complete this procedure. Plan a maintenance window before you begin.

Prerequisites

- You have cluster administrator access to your OpenShift cluster.
- You have OpenShift AI 3.3 installed with MaaS enabled as a Technology Preview feature.
- You have the OpenShift command-line interface (**oc**) installed.
- You have tested this procedure in a non-production environment.

Procedure

1. Document your current tier configuration by creating backups of the existing resources. If a resource does not exist in your environment, skip the corresponding backup step.

- a. Create a backup directory by running the following command:

```
$ mkdir -p migration-backup
```

- b. Back up the **tier-to-group-mapping ConfigMap** by running the following command:

```
$ oc get configmap tier-to-group-mapping -n <infrastructure_namespace> -o yaml > migration-backup/tier-to-group-mapping.yaml
```

where:

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

If the **tier-to-group-mapping ConfigMap** is not found in your infrastructure namespace, search all namespaces to locate it:

+

```
$ oc get configmap tier-to-group-mapping --all-namespaces
```

- a. Back up the gateway authentication policy by running the following command:

```
$ oc get authpolicy gateway-auth-policy -n openshift-ingress -o yaml > migration-backup/gateway-auth-policy.yaml
```

- b. Back up the **TokenRateLimitPolicy** resources by running the following command:

```
$ oc get tokenratelimitpolicy -n openshift-ingress -o yaml > migration-backup/gateway-rate-limits.yaml
```

- c. Back up the **LLMInferenceService** resources by running the following command:

```
$ oc get llminferenceservice -n <model_namespace> -o yaml > migration-backup/llm-models.yaml
```

where:

<model_namespace>

Specifies the namespace where your models are deployed. If your models are deployed across multiple namespaces, repeat this command for each namespace and save each backup to a separate file.

2. Record the mapping between your tiers and their associated groups, models, and rate limits. Save this mapping to a file such as **migration-backup/tier-mapping.txt**. You will use this mapping to create subscriptions after the upgrade.

To extract this information from your backups:

- Review **migration-backup/tier-to-group-mapping.yaml** for tier names and their associated groups.
- Review **migration-backup/gateway-rate-limits.yaml** for the token rate limits configured for each tier.
- Review **migration-backup/llm-models.yaml** for the **alpha.maas.opendatahub.io/tiers** annotation on each model to determine which tiers have access to which models.

The following is an example mapping table:

Tier name	Groups	Models	Rate limit: tokens per minute
free	system:authenticated	simulator, qwen3	100
premium	premium-users	simulator, qwen3, llama	50000

Tier name	Groups	Models	Rate limit: tokens per minute
enterprise	enterprise-users	all models	100000

3. Disable the MaaS component in OpenShift AI:

- a. Go to the **Installed Operators** page in the OpenShift web console. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
- b. Click the **Red Hat OpenShift AI Operator**.
- c. Click the **Data Science Cluster** tab, and then click the **default-dsc** object.
- d. Click the **YAML** tab and set the **spec.components.kserve.modelsAsService.managementState** field to **Removed**:

```
spec:
  components:
    kserve:
      modelsAsService:
        managementState: Removed
```

- e. Click **Save** and wait for the Operator to reconcile.
- f. Verify that the MaaS pods are removed by running the following command:

```
$ oc get pods -n <infrastructure_namespace> -l app=maas-controller
```

where:

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

The command should return no results.

Verification

- Verify that the backup files exist in the **migration-backup** directory:

```
$ ls migration-backup/
```

The output should list **tier-to-group-mapping.yaml**, **gateway-auth-policy.yaml**, **gateway-rate-limits.yaml**, and **llm-models.yaml**.

- Verify that MaaS is disabled by confirming that no MaaS pods are running:

```
$ oc get pods -n <infrastructure_namespace> -l app=maas-controller
```

where:

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

The command should return no results.

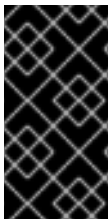
Next steps

- Upgrade the OpenShift AI Operator to version 3.5.
- After the upgrade, complete the migration by following [Migrate from tiers to subscriptions](#).

1.13. MIGRATE MODELS-AS-A-SERVICE FROM TIERS TO SUBSCRIPTIONS

After you upgrade the OpenShift AI Operator to version 3.5, complete this procedure to migrate your Models-as-a-Service (MaaS) configuration from the tier-based model to the subscription-based model. Starting with OpenShift AI 3.4, tiers are replaced by **MaaSSubscription**, **MaaSAuthPolicy**, and **MaaSModelRef** custom resources.

There is no automated migration tool for this process. You must manually re-create your access policies as subscriptions by using the tier mapping table that you recorded before the upgrade.



IMPORTANT

The authentication model changed between OpenShift AI 3.3 and OpenShift AI 3.4. In 3.3, MaaS used service account tokens for authentication. In OpenShift AI 3.4, MaaS uses subscription-bound API keys. After migration, all users must generate new API keys through the MaaS dashboard or API.

Prerequisites

- You have cluster administrator access to your OpenShift cluster.
- You have upgraded the OpenShift AI Operator to version 3.5.
- You have completed the pre-upgrade preparation procedure described in [Prepare for Models-as-a-Service tier-to-subscription migration](#), including backing up your tier configuration and disabling MaaS.
- You have the OpenShift command-line interface (**oc**) installed.
- You have the tier mapping table that you recorded before the upgrade.

Procedure

1. Re-enable the MaaS component in OpenShift AI:
 - a. Go to the **Installed Operators** page in the OpenShift web console. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.

- b. Click the **Red Hat OpenShift AI** Operator.
- c. Click the **Data Science Cluster** tab, and then click the **default-dsc** object.
- d. Click the **YAML** tab and set the **spec.components.kserve.modelsAsService.managementState** field to **Managed**:

```
spec:
  components:
    kserve:
      modelsAsService:
        managementState: Managed
```

- e. Click **Save** and wait for the Operator to reconcile.
- f. Verify that the MaaS controller created the default gateway policies by running the following commands:

```
$ oc get authpolicy gateway-default-auth -n openshift-ingress
$ oc get tokenratelimitpolicy gateway-default-deny -n openshift-ingress
```

Both commands should return a resource. If either policy is missing, check the MaaS controller logs for errors:

```
$ oc logs -n <infrastructure_namespace> -l app=maas-controller --tail=50
```

where:

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

2. Configure a PostgreSQL secret for MaaS. MaaS in OpenShift AI 3.4 and later requires a PostgreSQL database to store subscription, API key, and usage data. This is a new requirement that did not exist in OpenShift AI 3.3. You must provision a new PostgreSQL database for this purpose; there is no data to migrate from 3.3. For more information, see [Configure a PostgreSQL secret for Models-as-a-Service](#).
3. Verify that MaaS is installed correctly. For more information, see [Verify the Models-as-a-Service installation](#).
4. Publish your models to MaaS. For more information, see [Deploy models by using the dashboard wizard](#) or [Publish a model to Models-as-a-Service using YAML](#).
5. Create **MaaSSubscription** resources to replace each tier. Map the groups and rate limits from your tier mapping table to the subscription configuration.



NOTE

Both a **MaaSSubscription** and a **MaaSAuthPolicy** are required for each model to be accessible through MaaS. A subscription defines token rate limits, and an authorization policy grants API gateway access. If you create a **MaaSAuthPolicy** without a corresponding **MaaSSubscription**, users receive a **429 Too Many Requests** response because no token quota is defined. If you create a **MaaSSubscription** without a **MaaSAuthPolicy**, users receive a **403 Forbidden** response because they have no gateway access.

Apply a YAML file with the following structure for each tier:

```
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSSubscription
metadata:
  name: <subscription_name>
  namespace: models-as-a-service
spec:
  owner:
    groups:
      - name: <group_name>
    users: []
  modelRefs:
    - name: <model_name>
      namespace: <model_namespace>
      tokenRateLimits:
        - limit: <token_limit>
          window: 1m
    - name: <additional_model_name>
      namespace: <model_namespace>
      tokenRateLimits:
        - limit: <token_limit>
          window: 1m
```

In the YAML file, set the following values:

<subscription_name>

Specifies a name for the subscription, such as **premium-models-subscription**.

<group_name>

Specifies the OpenShift group that maps to the original tier. Add multiple **name** entries under **groups** for tiers that mapped to multiple groups.

<model_name>, <additional_model_name>

Specifies the name of each model that this subscription grants quota for. Add an entry under **modelRefs** for each model in the subscription.

<model_namespace>

Specifies the namespace where the **MaaSModelRef** resource is deployed.

<token_limit>

Specifies the maximum number of tokens per window for this model.

For more information, see [Manage Models-as-a-Service subscriptions](#).

6. Create **MaaSAuthPolicy** resources to grant groups access to model endpoints. Without an authorization policy, users receive a **403 Forbidden** response even if they have a valid subscription.

Apply a YAML file with the following structure for each access mapping:

```
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSAuthPolicy
metadata:
  name: <policy_name>
  namespace: models-as-a-service
spec:
  modelRefs:
    - name: <model_name>
      namespace: <model_namespace>
  subjects:
    groups:
      - name: <group_name>
    users: []
```

In the YAML file, set the following values:

<policy_name>

Specifies a name for the authorization policy, such as **premium-models-access**.

<model_name>

Specifies the name of the model to grant access to.

<model_namespace>

Specifies the namespace where the **MaaSModelRef** resource is deployed.

<group_name>

Specifies the OpenShift group that maps to the original tier.

For more information, see [Manage Models-as-a-Service authorization policies](#).

Verification

1. Verify that all **MaaSModelRef** resources have a **Ready** status. **MaaSModelRef** resources are created automatically when you publish models and reside in the model's project namespace, not in **models-as-a-service**. To find all **MaaSModelRef** resources across namespaces, run **oc get maasmodelref --all-namespaces**.

```
$ oc get maasmodelref -n <model_namespace>
```

where:

<model_namespace>

Specifies the project namespace where the model is deployed.

2. Verify that all **MaaSSubscription** resources are created:

```
$ oc get maassubscription -n models-as-a-service
```

3. Verify that all **MaaSAuthPolicy** resources are created:

```
$ oc get maasauthpolicy -n models-as-a-service
```

4. Verify that the MaaS controller generated the gateway authentication and rate limit policies for each model:

```
$ oc get authpolicy -n <model_namespace>
$ oc get tokenratelimitpolicy -n <model_namespace>
```

where:

<model_namespace>

Specifies the project namespace where the model is deployed.

Each model should have a corresponding **AuthPolicy** and **TokenRateLimitPolicy** created by the controller. If these policies are missing, check the MaaS controller logs for errors:

+

```
$ oc logs -n <infrastructure_namespace> -l app=maas-controller --tail=50
```

+ where:

+

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

5. Test inference as an authorized user by using a subscription-bound API key. Run the following commands in the same terminal session so that the **HOST** variable is available for subsequent verification steps.

```
$ HOST="maas.$(oc get ingresses.config.openshift.io cluster -o jsonpath='{.spec.domain}')"

$ curl -H "Authorization: Bearer <api_key>" \
  "https://${HOST}/llm/<model_name>/v1/chat/completions" \
  -H "Content-Type: application/json" \
  -d '{"model": "<model_name>", "messages": [{"role": "user", "content": "test"}], "max_tokens": 10}'
```

In the command, set the following values:

<api_key>

Specifies a MaaS API key generated for a user in one of the configured subscription groups. For more information about creating API keys, see [Manage API keys](#).

<model_name>

Specifies the name of a model registered with MaaS.

A successful response returns an HTTP **200** status code.

6. Optional: Test rate limiting by sending rapid requests to a model endpoint that has a low token limit, such as a subscription configured with a limit of 100 tokens per minute:

```
$ for i in $(seq 1 20); do
  curl -s -o /dev/null -w "%{http_code}\n" \
```

```
-H "Authorization: Bearer <api_key>" \
"https://${HOST}/llm/<model_name>/v1/chat/completions" \
-H "Content-Type: application/json" \
-d '{"model": "<model_name>", "messages":
[{"role": "user", "content": "test"}], "max_tokens": 50}'
done | sort | uniq -c
```

The output should include a mix of **200** and **429** status codes, confirming that token rate limits are enforced. If your subscriptions have high token limits, this test may not trigger rate limiting. In that case, verify the **TokenRateLimitPolicy** exists for the model instead:

```
$ oc get tokenratelimitpolicy -n <model_namespace>
```

where:

<model_namespace>

Specifies the project namespace where the model is deployed.

7. Verify that unauthorized requests are rejected. Send a request with an invalid API key:

```
$ curl -s -o /dev/null -w "%{http_code}\n" \
-H "Authorization: Bearer invalid-key" \
"https://${HOST}/llm/<model_name>/v1/chat/completions" \
-H "Content-Type: application/json" \
-d '{"model": "<model_name>", "messages": [{"role": "user", "content": "test"}], "max_tokens": 10}'
```

The command should return **401** or **403**.



NOTE

If any verification step fails, see the [Section 1.13, “Migrate Models-as-a-Service from tiers to subscriptions”](#) section to restore your previous configuration.

Rollback

If the migration fails or models are not accessible after the upgrade, you can restore the previous configuration from the backups created during the pre-upgrade preparation.



NOTE

If only specific models have issues, you can perform a partial rollback by deleting only the **MaaSModelRef**, **MaaSAuthPolicy**, and **MaaSSubscription** resources for that model and restoring its tier annotation from the backup. You do not need to roll back the entire migration.

1. Delete the MaaS custom resources created during the migration:

```
$ oc delete maasmodelref <model_name> -n <model_namespace>
$ oc delete maasauthpolicy <policy_name> -n models-as-a-service
$ oc delete maassubscription <subscription_name> -n models-as-a-service
```

where:

<model_name>

Specifies the name of the model.

<model_namespace>

Specifies the project namespace where the model is deployed.

<policy_name>

Specifies the name of the authorization policy.

<subscription_name>

Specifies the name of the subscription.

Repeat these commands for each resource created during the migration. To list all resources, run **oc get maasmodelref -n <model_namespace>**, **oc get maasauthpolicy -n models-as-a-service**, and **oc get maassubscription -n models-as-a-service**.

2. Restore the backed-up resources. Apply each backup file that you created during the pre-upgrade preparation:

```
$ oc apply -f migration-backup/gateway-auth-policy.yaml
$ oc apply -f migration-backup/gateway-rate-limits.yaml
$ oc apply -f migration-backup/tier-to-group-mapping.yaml
```

If you skipped any backup steps because the resource did not exist in your environment, skip the corresponding restore command.

3. Re-add tier annotations to the **LLMInferenceService** resources from the backup:

```
$ oc apply -f migration-backup/llm-models.yaml
```

4. Restart the MaaS API to reload the tier configuration:

```
$ oc rollout restart deployment/maas-api -n <infrastructure_namespace>
```

where:

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

5. Verify that the rollback succeeded by confirming that the tier-based resources are restored:

```
$ oc get configmap tier-to-group-mapping -n <infrastructure_namespace>
$ oc get authpolicy gateway-auth-policy -n openshift-ingress
$ oc get llminferenceservice -n <model_namespace> -o
jsonpath='{.items[0].metadata.annotations}'
```

where:

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

<model_namespace>

Specifies the project namespace where the model is deployed.

The first two commands should return the restored resources. The third command should show the **alpha.maas.opendatahub.io/tiers** annotation on your models.

1.13.1. Clean up the old Models-as-a-Service tier-based configuration

After you verify that the Models-as-a-Service (MaaS) migration from tiers to subscriptions is complete, remove the old tier-based resources from your cluster and inform users about the new authentication model.

Prerequisites

- You have completed the migration procedure described in [Migrate Models-as-a-Service from tiers to subscriptions](#) and verified that all models are accessible through MaaS.
- You have the OpenShift command-line interface (**oc**) installed.

Procedure

1. Remove the tier annotations from your **LLMInferenceService** resources by running the following command for each model:

```
$ oc annotate llminferenceservice <model_name> -n <model_namespace>
alpha.maas.opendatahub.io/tiers- --ignore-not-found
```

where:

<model_name>

Specifies the name of the model to update.

<model_namespace>

Specifies the project namespace where the model is deployed.

2. Delete the old gateway authentication policy by running the following command:

```
$ oc delete authpolicy gateway-auth-policy -n openshift-ingress --ignore-not-found
```

3. Delete the old tier-based **TokenRateLimitPolicy** resources. The **gateway-default-deny** policy created by the MaaS controller replaces them. First, list the existing policies to identify the ones to delete:

```
$ oc get tokenratelimitpolicy -n openshift-ingress
```

Delete each old tier-based policy, but do not delete **gateway-default-deny**:

```
$ oc delete tokenratelimitpolicy <policy_name> -n openshift-ingress --ignore-not-found
```

where:

<policy_name>

Specifies the name of each old tier-based rate limit policy.

4. Delete the **tier-to-group-mapping ConfigMap** by running the following command:

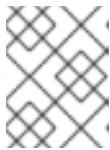
```
$ oc delete configmap tier-to-group-mapping -n <infrastructure_namespace> --ignore-not-found
```

where:

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

5. Inform users that they must generate new API keys. OpenShift AI 3.3 used service account tokens for authentication, but OpenShift AI 3.4 and later requires subscription-bound API keys. Inference endpoints accept only API keys with the **sk-oai-** prefix. Users can generate API keys from the MaaS dashboard by navigating to **Models-as-a-Service** → **API Keys**. For more information about creating and managing API keys, see [Use Models-as-a-Service](#).



NOTE

Store API keys securely. Do not store API keys in shell history, environment variables, or version control.

Verification

- Verify that the old tier annotations are removed from your models by running the following command:

```
$ oc get llminferenceservice -n <model_namespace> -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.metadata.annotations}{"\n"}{end}'
```

where:

<model_namespace>

Specifies the project namespace where the model is deployed.

The output should not contain the **alpha.maas.opendatahub.io/tiers** annotation on any model.

- Verify that the old gateway authentication policy is deleted by running the following command:

```
$ oc get authpolicy gateway-auth-policy -n openshift-ingress
```

The command should return **not found**.

- Verify that the **tier-to-group-mapping ConfigMap** is deleted by running the following command:

```
$ oc get configmap tier-to-group-mapping -n <infrastructure_namespace>
```

where:

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

The command should return **not found**.

Additional resources

- [Prepare for Models-as-a-Service tier-to-subscription migration](#)
- [Models-as-a-Service subscriptions](#)
- [Manage Models-as-a-Service authorization policies](#)
- [Manage API keys](#)
- [Models-as-a-Service administration troubleshooting](#)

1.14. MANAGE MODELS-AS-A-SERVICE SUBSCRIPTIONS

You can create and manage Models-as-a-Service (MaaS) subscriptions to control group access to models and configure token limits.



NOTE

In OpenShift AI 3.4, MaaS uses subscriptions instead of tiers.

1.14.1. Monitor MaaS subscriptions

In Red Hat OpenShift AI, you can use the **MaaS governance** page to monitor service subscriptions, verify their status, and review associated models, token limits, and priority levels.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- You have administrator access to the OpenShift AI dashboard.

Procedure

1. In the OpenShift AI dashboard, click **Settings** → **MaaS governance**.
The **Subscriptions** tab is displayed by default.
2. Review the information in the Subscriptions table:

Subscription

The unique identifier for the subscription.

Status

The current status of the subscription. Possible values: **Active**, **Failed**.

Groups

The number of user groups assigned to the subscription. Click the count to expand the row and view individual group names.

Models

The count of Models-as-a-Service (MaaS) model references included in the subscription. Click the count to expand the row and view each model name and its token limits.

Priority

The priority level assigned to the subscription. Higher numbers indicate higher priority.

3. Optional: To filter and organize the view:

- Use the **Keyword** dropdown to select a filter criterion.
 - Enter text in the **Filter by name or description** field to search for specific subscriptions.
 - Click the column headers to sort by Subscription, Status, Groups, Models, or Priority.
4. Click a subscription name to open the details page.
 5. In the details view, review the subscription configuration including groups, models, token limits, and synchronization status.

Verification

- Verify that the MaaS governance page displays all configured subscriptions in the system.
- Verify that subscriptions show the correct status: **Active** or **Failed**.
- Click a subscription name and verify that you can view its complete configuration.

1.14.2. Create a subscription for Models-as-a-Service

In Red Hat OpenShift AI, you can create a Models-as-a-Service (MaaS) subscription to grant user groups quota for specific models with configurable token limits.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- You have administrator access to the OpenShift AI dashboard.
- You have published at least one model to Models-as-a-Service (MaaS).

Procedure

1. In the OpenShift AI dashboard, click **Settings → MaaS governance**.
2. Click **Create subscription**.
3. In the **Create subscription** form, configure the subscription:
 - a. **Name:** Enter a descriptive name for the subscription.
 - b. Optional: Click **Edit resource name** to set a custom internal identifier for the subscription. If not specified, the resource name is generated automatically from the display name.
 - c. Optional: **Description:** Provide a brief description of the purpose of the subscription.
 - d. **Priority:** Set the subscription priority level using a numeric value. Higher numbers indicate higher priority. When a user belongs to multiple groups with different subscriptions, the subscription with the highest priority level is used.
 - e. **Groups:** Select OpenShift groups or enter custom group names from your external OIDC provider. Users who are members of these groups can access the models included in this subscription.
4. Configure models and token limits:

- a. Click **Add models**.
- b. In the **Add models** dialog, review the available models and their associated projects, model IDs, existing subscriptions, and policies.
- c. Click **Add model** for each model you want to include in this subscription.
- d. Click **Add models** to confirm your selection.
- e. For each model in the subscription, click **Add token limit** to configure token consumption limits.
- f. In the **Edit subscription token limits** dialog, enter the number of tokens allowed.
- g. Enter the time period value.
- h. Select the time unit: hour, minute, or second.
- i. Optional: Click **Add token rate limit** to configure additional token limits with different time windows.
- j. Click **Save**.

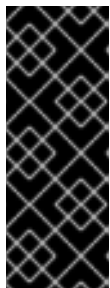
**NOTE**

At least one token limit is required for each model in the subscription.

5. Optional: Select **Create a matching authorization policy** to automatically create an authorization policy for this subscription.

**NOTE**

To consume model endpoints through the API gateway, users must have both a subscription and an authorization policy. A subscription defines quota for models. An authorization policy is a separate resource that authorizes specific groups to access model endpoints through the API gateway. While you can automatically create a matching authorization policy during subscription creation, creating authorization policies separately provides more flexibility and control.

**IMPORTANT**

The matching authorization policy feature creates an initial authorization policy with the same groups and models as the subscription. However, changes made to the subscription later such as adding or removing groups or models require manual updates to the authorization policy. When you modify a subscription, you must manually update the corresponding authorization policy to keep them synchronized.

6. Click **Create subscription**.

Verification

- In the OpenShift AI dashboard, navigate to **Settings → MaaS governance** and verify that the new subscription appears in the list with a status of **Active**.

- Click the subscription name to view its details and confirm the groups, models, token limits, and priority level are configured correctly.

Next steps

- If you did not select **Create a matching authorization policy**, [Create an authorization policy](#)
- [Create an API key for a user](#)

1.14.3. Edit a subscription

In Red Hat OpenShift AI, you can edit a Models-as-a-Service (MaaS) subscription to add or remove models, change token limits, or update group assignments. Changes modify the **MaaSSubscription** custom resource.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- You have administrator access to the OpenShift AI dashboard.
- At least one MaaS subscription exists.



IMPORTANT

Editing a **MaaSSubscription** resource takes effect when you save the changes:

- Changing token limits affects all users in the subscription groups.
- Adding or removing groups grants or revokes subscription quota for those groups.
- Adding or removing models changes which models are available to users in the subscription groups.

Procedure

1. In the OpenShift AI dashboard, click **Settings → MaaS governance**.
2. Locate the subscription you want to modify and click its name.
3. Click the action menu (⋮), and then select **Edit**.
4. Modify the subscription properties:
 - a. Update the name or description if needed.
 - b. Adjust the priority level.
 - c. Add or remove groups.
 - d. Add or remove models by clicking **Add models** or using the action menu (⋮) next to each model.
 - e. Modify token limits for existing models by clicking **Add token limit** or editing existing limits.



IMPORTANT

The **metadata.name** of the **MaaSSubscription** custom resource cannot be changed after creation. If you need a different resource name, delete the subscription and create a new one.

5. Click **Save**.



IMPORTANT

If you created a matching authorization policy when you created this subscription, changes to the subscription such as adding or removing groups or models require manual updates to the authorization policy. After modifying a subscription, manually update the corresponding authorization policy to keep them synchronized.

Verification

- In the OpenShift AI dashboard, navigate to **Settings → MaaS governance**.
- Verify that the subscription shows the updated configuration.
- Optional: Check the **MaaSSubscription** custom resource to confirm the changes:

```
$ oc get maassubscription <subscription-name> -n models-as-a-service -o yaml
```

1.14.4. Delete a subscription

In Red Hat OpenShift AI, you can delete a Models-as-a-Service (MaaS) subscription to revoke quota for specific user groups. Deleting a subscription removes the quota limits for all groups included in that subscription.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.
- You have created at least one MaaS subscription.

Deleting a subscription revokes quota for all groups included in that subscription. Make sure that affected users have access to models through other subscriptions before deletion to avoid service disruption.

Procedure

1. In the OpenShift AI dashboard, click **Settings → MaaS governance**.
2. Locate the subscription you want to delete and click its name.
3. Click the action menu (⋮), and then select **Delete**.
4. In the **Delete subscription** dialog, review the impact of deletion:
 - Users in groups assigned to this subscription lose access to the included models.
 - All API keys bound to this subscription are invalidated.

- Authorization policies are not deleted automatically. After deletion, review and update any authorization policies that referenced this subscription's groups or models.

5. To confirm, type the subscription name and click **Delete**.

Verification

- In the OpenShift AI dashboard, navigate to **Settings → MaaS governance**.
- Verify that the deleted subscription no longer appears in the list.
- Optional: List the remaining subscriptions to confirm the expected state:

```
$ oc get maassubscription -n models-as-a-service
```

Next steps

- [View authorization policies](#) to review and update policies that referenced groups or models in the deleted subscription.

Additional resources

- [Models-as-a-Service administration troubleshooting](#)

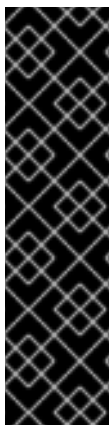
1.15. MANAGE MODELS-AS-A-SERVICE AUTHORIZATION POLICIES

You can create and manage authorization policies to control which groups can access AI model endpoints through the API gateway.

1.15.1. Models-as-a-Service authorization policies

In Red Hat OpenShift AI, you can use Models-as-a-Service (MaaS) authorization policies in combination with subscriptions to control user access to model endpoints through the API gateway.

A subscription gives groups quota for specific models with token rate limits. An authorization policy is required to authorize groups to access model endpoints through the API gateway.



IMPORTANT

Both a subscription and an authorization policy are required for users to access models through Models-as-a-Service:

- **Subscription:** Defines quota for models with token rate limits.
- **Authorization policy:** Authorizes groups to access model endpoints through the API gateway.

Without an authorization policy, users receive 403 Forbidden errors even if they have a valid subscription.

An authorization policy consists of the following components:

Name

A unique identifier for the policy.

Description

An optional description explaining the purpose of the policy.

Groups

The groups authorized to access model endpoints through this policy. Groups can come from OpenShift Group objects or external OIDC providers depending on the authentication method.

Models

The model endpoints that authorized groups can access.

Phase

The current status of the policy: **Pending**, **Active**, **Degraded**, **Failed**, or **Invalid**.

When you create a subscription, you can optionally create a matching authorization policy automatically by selecting the **Create matching authorization policy** checkbox. This creates an authorization policy with the same groups and models as the subscription, ensuring that users can immediately access the models included in their subscription.

Authorization policy lifecycle

- **Creating policies:** Create authorization policies manually or automatically when creating a subscription.
- **Active phase:** When a policy is in **Active** phase, the specified groups can access the configured model endpoints through the API gateway.
- **Failed phase:** If a policy enters **Failed** phase, check the policy status conditions for error messages.
- **Updating policies:** You can add or remove groups and models from existing policies. Changes take effect after the Authorino cache expires. The cache time-to-live (TTL) is configured by the controller.
- **Deleting policies:** Deleting an authorization policy revokes API gateway access for all groups in that policy after the Authorino cache expires.

Gateway-scoped authentication architecture:

In OpenShift AI 3.5 and later, MaaS uses a single gateway-scoped **AuthPolicy** named **maas-gateway-auth** per tenant gateway. This **AuthPolicy** aggregates authentication and authorization rules for all models served through the tenant's gateway, replacing the per-model **AuthPolicy** pattern used in earlier releases.

The gateway-scoped **AuthPolicy** enforces the following rules through CEL expressions:

- API key validation and OIDC token verification
- Subscription checks for model access
- Group membership validation
- Tenant-gateway isolation to prevent cross-tenant access

Model identity is resolved dynamically at the gateway level. End-user authentication behavior is unchanged: a valid API key or OIDC token, an active subscription, and an authorized group are required to access a model.

**NOTE**

Legacy per-model **AuthPolicy** resources are automatically cleaned up by the controller on the first reconciliation after upgrade to OpenShift AI 3.5. The gateway-scoped **AuthPolicy** requires Kuadrant v1.4.2 or later for **spec.defaults.rules** support.

**NOTE**

Authorization policies are managed as Models-as-a-Service (MaaS) custom resources in OpenShift. You can manage them through the OpenShift AI dashboard or using OpenShift CLI (**oc**).

1.15.2. View authorization policies

In Red Hat OpenShift AI, you can view all Models-as-a-Service (MaaS) authorization policies on the **MaaS governance** page to see which user groups can access specific model endpoints.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.
- At least one MaaS authorization policy exists.

Procedure

1. In the OpenShift AI dashboard, click **Settings → MaaS governance**.
2. Click the **Authorization policies** tab.
The authorization policies table displays with the following columns:

Authorization policy

The name of the authorization policy. Policies created automatically from subscriptions include the subscription name in the policy name.

Status

The current status of the policy. Possible values: **Active**, **Failed**, **Unknown**.

Groups

The number of OpenShift user groups authorized by this policy. Click the count to expand the row and view individual group names.

Models

The number of model endpoints included in this policy. Click the count to expand the row and view individual model names.

3. Optional: Filter the list of authorization policies:
 - To filter by keyword, click the **Keyword** dropdown and select a filter option.
 - To search by name or description, enter text in the search field.
4. Optional: Sort the table by clicking any column header.
5. To view details of a specific authorization policy, click the authorization policy name.

6. On the details page, review the policy name, description, status, resource name, creation date, authorized groups, and included models along with the namespace each model is deployed in.

Verification

- The authorization policies table displays all policies with their current status.
- When you click a policy name, the groups and models configuration is displayed.

1.15.3. Create an authorization policy

In Red Hat OpenShift AI, you can create authorization policies to control which groups can access AI model endpoints through the API gateway.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.
- You have published at least one model to Models-as-a-Service (MaaS).

Procedure

1. In the OpenShift AI dashboard, click **Settings** → **MaaS governance** and click the **Authorization policies** tab.
2. Click **Create authorization policy**.
3. In the **Create authorization policy** dialog, configure the following settings:
 - a. In the **Name** field, enter a unique name for the authorization policy.



NOTE

By default, the resource name matches the policy name. To customize the resource name, click **Edit resource name** and enter a different value. The resource name identifies the underlying **MaaSAuthPolicy** custom resource in OpenShift.

- b. Optional: In the **Description** field, enter a description explaining the purpose of the policy.
- c. From the **Groups** dropdown, select the groups to authorize for API gateway access. You can select multiple groups or type to add a new group name. Groups can come from OpenShift Group objects, API key group snapshots, or OIDC token claims depending on how users authenticate.
- d. In the **Models** section, click **Add models**.
- e. In the **Add models to authorization policy** dialog, browse the list of available models or use the search field to filter by name or description.
- f. Review the **Subscriptions** and **Policies** columns to see which subscriptions and policies already include each model.
- g. For each model you want to add, click **Add model**.

- h. Click **Add models** to add the selected models to the policy.
The **Models** section displays a table with the selected models and their project namespaces.

4. Click **Create authorization policy**.



NOTE

If you create a subscription with the **Create matching authorization policy** option selected, an authorization policy is created automatically with the same groups and models as the subscription. You only need to create authorization policies manually when you want to configure API gateway access independently of subscriptions.

Verification

- The new authorization policy appears in the Authorization policies table with an **Active** phase.

Next steps

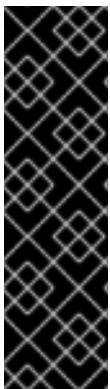
- [Create an API key for a user](#)

1.15.4. Edit an authorization policy

In Red Hat OpenShift AI, you can use the dashboard to edit Models-as-a-Service (MaaS) authorization policies to add or remove authorized groups and model endpoints.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.
- At least one authorization policy exists.



IMPORTANT

Editing an authorization policy takes effect when you save your changes:

- Adding groups grants API gateway access to users in those groups.
- Removing groups revokes API gateway access for users in those groups, including users with active sessions.
- Users and applications might experience access changes or receive 403 Forbidden errors based on the new configuration.

Procedure

1. In the OpenShift AI dashboard, click **Settings** → **MaaS governance** and click the **Authorization policies** tab.
2. In the row for the authorization policy you want to edit, click the action menu (⋮), and then select **Edit**.
3. In the **Edit authorization policy** dialog, modify the following settings as needed:
 - a. Update the **Name** field to change the policy display name.

- b. Update the **Description** field to change the policy description.
- c. From the **Groups** dropdown, add or remove groups:
 - To add groups, select additional groups from the dropdown or type to add a new group name.
 - To remove a group, click the remove icon (×) next to the group name.

**NOTE**

The available groups depend on your authentication configuration: OpenShift groups when using OpenShift authentication, or OIDC group claims when using external OIDC.

- d. In the **Models** section, add or remove models:
 - To add models, click **Add models** to open the **Add models to authorization policy** dialog. Select the models to add, and then click **Add models** in the dialog to confirm.
 - To remove a model, click the action menu (⋮) for the model, and then select **Remove**.
4. Click **Save** to apply your changes.

Verification

- The updated authorization policy appears in the Authorization policies table. To confirm specific changes, click the policy name to open the detail view.
- Confirm the updated configuration on the **MaaSAuthPolicy** resource:

```
$ oc get maasauthpolicy <policy-name> -n models-as-a-service -o yaml
```

1.15.5. Delete a Models-as-a-Service authorization policy

In Red Hat OpenShift AI, you can delete Models-as-a-Service (MaaS) authorization policies to revoke API gateway access for groups included in the policy. If another authorization policy grants the same groups access to the same models, those groups retain access through the remaining policy.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.
- At least one **MaaSAuthPolicy** resource exists.

Procedure

1. In the OpenShift AI dashboard, click **Settings** → **MaaS governance** and click the **Authorization policies** tab.
2. In the row for the authorization policy you want to delete, click the action menu (⋮), and then select **Delete**.
3. In the **Delete policy** dialog, enter the authorization policy name to confirm deletion.

4. Click **Delete**.**IMPORTANT**

Deleting an authorization policy revokes API gateway access for all groups in that policy. Users and applications using models covered by this policy receive 403 Forbidden errors even if they have a valid subscription.

If you delete an authorization policy that was automatically created with a subscription, the subscription remains active and continues to enforce token limits. API gateway access requires a new authorization policy.

When removing a user from a group, you must manually revoke all associated API keys to immediately revoke access. Consider setting up automation to revoke API keys when users are removed from groups.

Verification

- The deleted authorization policy no longer appears in the Authorization policies table.
- Confirm the **MaaSAuthPolicy** resource was deleted:

```
$ oc get maasauthpolicy <policy-name> -n models-as-a-service
```

Expected output:

```
Error from server (NotFound): maasauthpolicies.maas.opendatahub.io "<policy-name>" not found
```

Next steps

- [Create an authorization policy](#)

Additional resources

- [Subscriptions](#)
- [Models-as-a-Service administration troubleshooting](#)

1.16. MANAGE API KEYS FOR USERS

You can create and manage API keys on behalf of users to provide them with programmatic access to models through Models-as-a-Service subscriptions.

1.16.1. View API keys

In Red Hat OpenShift AI, you can view and filter all API keys created by users in your organization to monitor key usage and identify keys that might need to be revoked.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.

Procedure

1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
The **API keys** page displays a table with the following columns:
 - **Name:** The name assigned to the API key
 - **Status:** The current state of the key. Possible values: **Active, Expired, Revoked**.
 - **Subscription:** The subscription that scopes the key's access to models
 - **Owner:** The username of the key owner
 - **Created:** The date when the key was created
 - **Last used:** The date when the key was last used to access a model
 - **Expires:** The expiration date for the key
2. Optional: Filter the list of API keys:
 - To filter by status, click the **Status** dropdown and select **Active, Expired, or Revoked**.
 - To filter by username, enter a username in the search field.
3. Optional: Sort the table by clicking any column header.
4. Optional: If the list contains more keys than fit on one page, use the pagination controls at the bottom of the table to navigate between pages.

Verification

- The **API keys** table lists at least one key with a **Status** value of **Active, Expired, or Revoked**.
- Each key displays information including owner, subscription, creation date, last used date, and expiration.

1.16.2. Create an API key for a user

In Red Hat OpenShift AI, you can create API keys on behalf of users to provide them with programmatic access to models through Models-as-a-Service subscriptions.

Prerequisites

- You have access to the OpenShift AI dashboard with administrator privileges.
- You have created at least one Models-as-a-Service (MaaS) subscription.

Procedure

1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
2. Click **Create API key**.
3. In the **Create API key** dialog, configure the following settings:
 - a. In the **Name** field, enter a descriptive name for the API key.

- b. Optional: In the **Description** field, enter additional details about what the key is for.
- c. From the **Subscription** dropdown, select the subscription that determines which models the key can access and the applicable token limits.
The **Models** section displays the models included in the selected subscription and their configured token limits.
- d. From the **Expiration** dropdown, select when the key expires: 30 days, 60 days, 90 days (default), 180 days, 1 year, or **Custom (days)**. If you select **Custom (days)**, enter a value between 1 and 365.

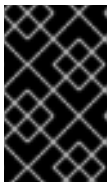


NOTE

As an administrator, you can set a maximum expiration limit in the **Tenant** custom resource. If not set, the default maximum is 90 days.

4. Click **Create**.

The **API key created** dialog displays the generated key with a prefix of **sk-oai-**.



IMPORTANT

The plaintext key is displayed only during creation and cannot be retrieved later. Save the key in a secrets manager before closing the **API key created** dialog. If you lose the key, you must revoke it and create a new one.

5. Click the copy icon next to the API key field to copy the key, and then provide it to the user through a secure channel.
6. Click **Close**.

Verification

- The new API key appears in the API keys table with an **Active** status.

Next steps

- [Make API calls to models](#)

1.16.3. Revoke user API keys

As an administrator, you can revoke individual Models-as-a-Service (MaaS) API keys, or all the keys belonging to a specific user.



IMPORTANT

API keys retain a snapshot of user group memberships from when the key was created. If a user is removed from a group, their existing API keys continue to grant access until you revoke them. To immediately revoke access after a group change, revoke the API keys for that user.

Prerequisites

- You have administrator privileges for OpenShift AI.

- You have access to the OpenShift AI dashboard with administrator privileges.
- At least one API key exists.

Procedure

To revoke an individual API key:

1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
2. In the row for the API key you want to revoke, click the action menu (⋮) and select **Revoke**.
3. In the **Revoke API key?** dialog, enter the API key name to confirm revocation.
4. Click **Revoke**.

To revoke all API keys for a single user:

1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
2. Click the action menu (⋮) in the table header and select **Revoke user API keys**.
3. In the **Revoke user API keys** dialog, enter the username in the **Username** field.
4. Click the search icon to display the keys for that user.
5. Click **Revoke all keys** to revoke all API keys for the user.

Verification

- Verify that the revoked API key or keys display a **Revoked** status in the API keys table.
- Verify that the revoked API key cannot make API calls:

```
$ curl -H "Authorization: Bearer <revoked-api-key>" \
https://<maas-gateway-url>/maas-api/v1/models
```

Expected output: **401 Unauthorized** error indicating the key is no longer valid.

1.16.4. Configure the API key expiration limit

In Red Hat OpenShift AI, you can set a maximum expiration period for API keys to enforce security policies and prevent users from creating keys with extended expiration periods.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have deployed Models-as-a-Service.
- The **Tenant** custom resource exists in the **models-as-a-service** namespace.



NOTE

Changes to **maxExpirationDays** apply only to API keys created after the change. Existing keys retain their original expiration dates.

Procedure

- Edit the **Tenant** custom resource to configure the API key expiration limit, using one of the following methods:

Using the OpenShift console

- In the OpenShift console, navigate to **Administration** → **CustomResourceDefinitions**.
- Search for **Tenant** and click the resource name.
- Click the **Instances** tab.
- Click **default-tenant**.
- Click the **YAML** tab.
- In the YAML editor, locate the **spec** section and add or update the **apiKeys** configuration:

```
spec:
  apiKeys:
    maxExpirationDays: 90
```

The YAML file uses the following field:

maxExpirationDays

Specifies the maximum allowed expiration in days for API keys. Common values: 30 for one month, 90 for three months, or 365 for one year. If not set, the default is 90 days.

- Click **Save**.

Using the OpenShift CLI

Run the following command:

```
$ oc patch tenants.maas.opendatahub.io default-tenant -n models-as-a-service \
--type merge \
-p '{"spec":{"apiKeys":{"maxExpirationDays":90}}}'
```

Verification

- Confirm that the **maxExpirationDays** value is set on the **Tenant** resource:

```
$ oc get tenants.maas.opendatahub.io default-tenant -n models-as-a-service -o
jsonpath='{.spec.apiKeys.maxExpirationDays}'
```

The output is the value you configured, for example, **90**.

- Optional: Test that the limit is enforced by attempting to create an API key with an expiration period that exceeds the configured maximum. The request receives a 400 Bad Request response with an error message indicating the expiration exceeds the configured maximum.

Additional resources

- [Subscriptions](#)
- [Models-as-a-Service administration troubleshooting](#)

1.17. MANAGE MODELS-AS-A-SERVICE BY USING THE CLI AND API

In Red Hat OpenShift AI, you can manage Models-as-a-Service (MaaS) configurations by using the MaaS API and custom resources. This approach is useful for the following scenarios:

- External OpenID Connect (OIDC) users who cannot access the dashboard
- Automating repetitive configuration tasks
- Integrating MaaS management into GitOps workflows and CI/CD pipelines

1.17.1. Models-as-a-Service API

In Red Hat OpenShift AI, you can use the Models-as-a-Service (MaaS) API to manage subscriptions, authorization policies, and API keys, and to call model endpoints programmatically. You can access the API directly through HTTP clients such as **curl** or integrate it with automation tools and GitOps workflows.

1.17.1.1. API structure

MaaS provides two APIs:

Management API

The MaaS management API (**/maas-api/v1**) provides endpoints for management operations such as creating API keys, listing models, and managing user access. This API accepts both OIDC tokens and API keys for authentication.

Inference API

Model inference endpoints use OpenAI-compatible request and response formats and require API key authentication. MaaS supports two routing modes:

- **Body-based routing (recommended):** Use the standard **/v1/chat/completions** or **/v1/completions** endpoint with the **model** field in the request body to specify which model receives the request. This mode enables drop-in compatibility with any OpenAI-compatible SDK or client.
 - **Path-based routing (legacy):** Use per-model endpoints at **/llm/<model-name>/v1** with the model name in the URL path. This mode is supported for backwards compatibility with existing integrations.
- Both routing modes apply the same subscription, token limit, and rate-limiting policies.

1.17.1.2. Authentication methods

The MaaS API supports two authentication methods:

OIDC tokens

Users authenticated through an external OIDC provider authenticate with JWTs obtained from their identity provider. OIDC tokens are required for management API operations that use external OIDC authentication.

API keys

Users authenticate all management API operations and all inference operations by using API keys with the **sk-oai-** prefix. API keys can be created through the MaaS API or, for OpenShift-authenticated users, through the dashboard.

Additional resources

- [Models-as-a-Service API endpoints reference](#)

1.17.2. MaaS custom resource workflow

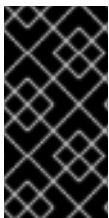
To make a model available to your users through Models-as-a-Service (MaaS), you can create and apply custom resources by using YAML and OpenShift CLI (**oc**) in the following order:

- Publish a model by creating a **MaaSModelRef** resource that references your deployed inference server.
- Create a subscription by defining a **MaaSSubscription** resource that assigns groups or users to the published model with token quota.
- Create an authorization policy by defining a **MaaSAuthPolicy** resource that grants API gateway access to the same groups or users.

For a complete list of MaaS custom resources, see [Models-as-a-Service custom resources](#).

1.18. MONITOR MODELS-AS-A-SERVICE USAGE BY USING THE OBSERVABILITY DASHBOARD

You can use the MaaS observability dashboard to monitor subscription-level usage metrics for cost attribution and showback reporting to finance teams.



IMPORTANT

The MaaS observability dashboard is a Technology Preview feature in Red Hat OpenShift AI. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production.

1.18.1. Models-as-a-Service usage monitoring

In Red Hat OpenShift AI, you can view Models-as-a-Service (MaaS) subscription-level usage metrics in the observability dashboard, including token consumption, request counts, and rate limit violations.

The dashboard is embedded in the OpenShift AI console by using Perses and queries metrics from Prometheus. Access to the dashboard is restricted to cluster administrators.

**NOTE**

The observability dashboard is designed for showback reporting, not as a billing-grade metering system. For production chargeback workflows that require precise billing data, access the **Limitador** metrics endpoint directly rather than using Prometheus or the dashboard CSV export.

The dashboard provides the following capabilities:

Overview metrics

View high-level statistics including total tokens consumed, total requests, total errors, success rate percentage, and active users.

Filtering

Filter metrics by user, subscription, and model to analyze specific usage patterns.

Time range selection

View metrics for configurable time periods ranging from the last 5 minutes to the last 14 days, or specify a custom date range.

Token consumption details

View a detailed table showing token consumption by user, subscription, and model, including request counts and rate limit violations.

Data export

Export usage data in CSV format for cost attribution and showback reporting to finance teams.

Dashboard metrics:

The observability dashboard displays the following overview metrics:

Table 1.3. Overview metrics

Metric	Description
Total Tokens	The total number of tokens consumed across all requests during the selected time period. This includes both input tokens (from user prompts) and output tokens (from model responses).
Total Requests	The total number of API requests made to MaaS models during the selected time period. Each API call to a model endpoint counts as one request.
Total Errors	The total number of failed requests during the selected time period. This includes requests that failed due to model errors, timeout errors, or other server-side issues.
Success Rate	The percentage of successful inference requests out of all requests made during the selected time period. The dashboard calculates this metric from vLLM success and latency counters. Rate-limited requests (HTTP 429) are excluded from the calculation.
Active Users	The number of unique users who made at least one request during the selected time period. Users are identified by their username from API key ownership or OIDC authentication.

The Token Consumption by User table displays detailed, per-user usage data with the following columns:

Table 1.4. Token consumption table columns

Column	Description
User	The username of the user who made the requests. For API key-based requests, this is the user who created the API key. For OIDC-authenticated requests, this is the user's OIDC identity.
Subscription	The subscription used for the requests. If a user belongs to multiple subscriptions, separate rows appear for each subscription.
Model	The model accessed by the user's requests. The format is <endpoint-name>/<model-id> for MaaS models.
Tokens	The total number of tokens consumed by this user for this subscription and model combination. Click the column header to sort the table by token consumption.
Requests	The number of API requests made by this user for this subscription and model combination.
Rate Limited	The number of requests that were rejected due to rate limiting (HTTP 429 responses). Rate-limited requests count toward the user's request total without consuming tokens.

Underlying Prometheus metrics:

The observability dashboard queries the following Prometheus metrics collected from **Kuadrant** and MaaS components:

Table 1.5. MaaS Prometheus metrics

Metric	Description
authorized_hits_total	Total number of tokens consumed. Labeled by subscription, model, and limitador_namespace. This metric is collected from Limitador and represents the actual token usage for successful requests.
authorized_calls_total	Total number of API requests made to models. Labeled by subscription and limitador_namespace. This metric counts all successful requests that passed rate limiting and authentication.
limited_calls_total	Total number of requests rejected due to rate limiting (HTTP 429 responses). Labeled by limitador_namespace. This metric indicates when users exceed their subscription token limits.

Metric	Description
istio_request_duration_milliseconds_bucket	Request latency at the API gateway. Tagged with subscription dimension for performance analysis. This metric helps identify performance issues by subscription.
auth_server_authconfig_duration_seconds	Time spent in Authorino authentication and authorization. Labeled by authorization policy. This metric helps troubleshoot authentication performance.



NOTE

The **user** label is disabled by default in MaaS metrics (**captureUser: false**). To enable per-user metrics collection, configure the **captureUser** setting in your **MaasTenantConfig**. The **model** label displays only on the **authorized_hits_total** metric due to Kuadrant wasm-shim limitations.

These metrics are scraped by Prometheus from the Limitador, Authorino, and gateway components. The observability dashboard aggregates and visualizes these metrics to provide usage insights for cost attribution and capacity planning.

Per-tenant observability:

In multitenant deployments, the **maas-api** instance for each tenant exposes Prometheus metrics with a **tenant_name** label, enabling per-tenant analysis of request rates, error rates, and latency.

Table 1.6. maas-api per-tenant Prometheus metrics

Metric	Description
maas_api_http_requests_total	Total number of HTTP requests processed by the per-tenant maas-api instance. Labeled by tenant_name , method , path , and status .
maas_api_http_request_duration_seconds	Histogram of HTTP request durations in seconds. Labeled by tenant_name , method , and path .
maas_api_key_validation_total	Total number of API key validation attempts. Labeled by tenant_name and result .
maas_api_token_mint_total	Total number of API key creation events. Labeled by tenant_name .

The default tenant uses **models-as-a-service** as its **tenant_name** label value.

To query per-tenant metrics, filter by the **tenant_name** label. The following PromQL examples show per-tenant request rate analysis:

```
# Request rate for a specific tenant
```

```
rate(maas_api_http_requests_total{tenant_name="red-team"}[5m])

# Error rate by tenant
sum by (tenant_name) (rate(maas_api_http_requests_total{status=~"5.."}[5m]))
```

Multitenant observability includes three pillars:

Prometheus metrics

Per-tenant **maas-api** metrics with **tenant_name** labels, as described in the preceding table. The Grafana Per-Tenant Metrics dashboard provides a **\$tenant** template variable for filtering metrics by tenant.

OTEL traces

When OTEL tracing is enabled, traces include span attributes for **tenant.name**, **tenant.namespace**, **gateway.name**, and **gateway.namespace**. OTEL tracing is disabled by default. To enable it, set the **OTEL_EXPORTER_OTLP_ENDPOINT** environment variable on the **maas-api** deployment.

Structured JSON logs

The **maas-api** structured JSON logs include **tenant_name**, **tenant_namespace**, **gateway_name**, and **gateway_namespace** fields in each log entry, enabling per-tenant log filtering.

1.18.2. Enable Kuadrant observability for Models-as-a-Service

In Red Hat OpenShift AI, you can enable observability in the **Kuadrant** custom resource to collect rate-limiting metrics for Models-as-a-Service usage tracking and monitoring.

Kuadrant uses **Limitador**, a rate-limiting service, to enforce the token limits defined in Models-as-a-Service (MaaS) subscriptions. When observability is enabled, Kuadrant creates a **PodMonitor** that configures Prometheus to scrape metrics from **Limitador**. These metrics track token consumption, request counts, and rate-limit violations, which are displayed in the MaaS observability dashboard for cost attribution and usage monitoring.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have installed Red Hat OpenShift AI.
- You have configured the observability stack for OpenShift AI. For more information, see [Observability configuration for Models-as-a-Service](#).

Procedure

1. Enable Kuadrant observability using the OpenShift console:
 - a. In the OpenShift console, navigate to **Administration** → **CustomResourceDefinitions**.
 - b. Search for **Kuadrant** and click the resource name.
 - c. Click the **Instances** tab.
 - d. Click **kuadrant**.
 - e. Click the **YAML** tab.

- f. In the YAML editor, locate the **spec** section and add or update the observability configuration:

```
spec:
  observability:
    enable: true
```

The patch uses the following fields:

enable: true

Enables the **Kuadrant** observability stack, which creates a **PodMonitor** resource that configures Prometheus to scrape rate-limiting metrics from **Limitador**. These metrics are used by the MaaS observability dashboard to track token consumption, request counts, and rate-limit violations.

- g. Click **Save**.

Alternatively, you can configure the resource using the command line:

```
$ oc patch kuadrant kuadrant -n kuadrant-system \
  --type merge \
  -p '{"spec":{"observability":{"enable":true}}}'
```

Verification

- Verify that the **Limitador PodMonitor** was created:

```
$ oc get podmonitor kuadrant-limitador-monitor -n kuadrant-system
```

Expected output shows the **PodMonitor** resource:

```
NAME                      AGE
kuadrant-limitador-monitor 2m
```

- Verify that the **Kuadrant** custom resource shows observability as enabled:

```
$ oc get kuadrant kuadrant -n kuadrant-system -o jsonpath='{.spec.observability.enable}'
```

Expected output:

```
true
```

- Verify that Prometheus is scraping metrics from **Limitador**:

- In the OpenShift console, navigate to **Observe → Metrics**.

- Run the following query to verify rate-limiting metrics are available:

```
limited_calls
```

You should see metrics showing rate-limit violations by user and subscription. If no data appears, this is expected if no models have been accessed yet. The metric appears once users begin making requests to MaaS models.

Next steps

- [Enable telemetry for Models-as-a-Service](#)

1.18.3. Enable telemetry for Models-as-a-Service

In Red Hat OpenShift AI, you can enable telemetry in the **Tenant** custom resource to collect usage metrics from Models-as-a-Service (MaaS) inference requests.

Telemetry configures the MaaS gateway to generate Prometheus metrics about model usage, including token consumption, request counts, and model-specific usage patterns. These metrics are displayed in the MaaS observability dashboard.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have installed Red Hat OpenShift AI.
- You have deployed Models-as-a-Service.

Procedure

1. Enable telemetry using the OpenShift console:
 - a. In the OpenShift console, navigate to **Administration** → **CustomResourceDefinitions**.
 - b. Search for **Tenant** and click the resource name.
 - c. Click the **Instances** tab.
 - d. Click **default-tenant**.
 - e. Click the **YAML** tab.
 - f. In the YAML editor, locate the **spec** section and add or update the **telemetry** configuration:

```
spec:
  telemetry:
    enabled: true
    metrics:
      captureOrganization: true
      captureUser: false
      captureGroup: false
      captureModelUsage: true
```

The patch uses the following fields:

enabled: true

Activates TelemetryPolicy and Istio Telemetry to collect MaaS usage metrics.

captureOrganization

Includes organization identifiers in metrics. Default is **true**.

captureUser

Includes user labels in metrics. Default is **false** due to privacy and cardinality considerations. Enabling this option with a large number of users can significantly increase Prometheus database size.

captureGroup

Includes group labels in metrics. Default is **false** to reduce metric cardinality.

captureModelUsage

Tracks model-specific usage patterns. Default is **true**.

- g. Click **Save**.

Alternatively, you can configure the resource using the command line:

```
$ oc patch tenants.maas.opendatahub.io default-tenant -n models-as-a-service \
--type merge \
-p '{
  "spec": {
    "telemetry": {
      "enabled": true,
      "metrics": {
        "captureOrganization": true,
        "captureUser": false,
        "captureGroup": false,
        "captureModelUsage": true
      }
    }
  }
}'
```

Verification

1. In the OpenShift console, navigate to **Observe → Metrics**.
2. In the query field, enter the following metric name:

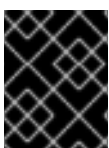
```
authorized_calls
```

3. Click **Run queries**.

If telemetry is enabled, the query returns MaaS usage metrics with labels such as **subscription**, **limitador_namespace**, and optionally **model** depending on your telemetry configuration.

1.18.4. View the Models-as-a-Service observability dashboard

In Red Hat OpenShift AI, you can use the Models-as-a-Service (MaaS) observability dashboard to monitor token consumption, request counts, and rate-limit violations across subscriptions and users.



IMPORTANT

The MaaS observability dashboard is intended for internal usage tracking and showback reporting. The metrics are not suitable for billing-grade metering or external invoicing.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have installed Red Hat OpenShift AI.
- You have configured observability for Models-as-a-Service. For more information, see [Observability configuration for Models-as-a-Service](#).
- You have enabled Kuadrant observability. For more information, see [Enable Kuadrant observability for Models-as-a-Service](#).
- You have enabled telemetry for Models-as-a-Service. For more information, see [Enable telemetry for Models-as-a-Service](#).

Procedure

1. In the OpenShift AI dashboard, click **Observe & monitor** → **Dashboard** in the left navigation menu.
The Observability dashboard page displays three tabs: **Cluster**, **Models**, and **Usage**.
2. Click the **Usage** tab to view Models-as-a-Service usage metrics.
3. Optional: To change the time range, select a value from the **Time period** dropdown. Options range from the last 5 minutes to the last 14 days. You can also specify a custom date range.
4. Optional: Filter the metrics by user, subscription, or model by selecting values from the **User**, **Subscription**, or **Model** dropdowns. Select **All** in any dropdown to view metrics for all items in that category.
5. Review the **Overview** section, which displays summary metrics including **Total Tokens**, **Total Requests**, **Total Errors**, **Success Rate**, and **Active Users**.
6. Review the **Token Consumption by User** table, which shows detailed per-user usage data.
7. Optional: Click column headers in the table to sort by that column.
8. Optional: Use the pagination controls at the bottom of the table to navigate through multiple pages of results or adjust the number of rows displayed per page.

Verification

- The **Overview** section shows non-zero values for users with recent activity.
- The **Token Consumption by User** table lists users with token consumption in the selected time period.
- Changing the time period updates the metrics to reflect the new range.

Additional resources

- [Models-as-a-Service observability overview](#)

Next steps

- [Export usage data](#)

- [Managing subscriptions for Models-as-a-Service](#)

1.18.5. Export usage data for cost attribution

In Red Hat OpenShift AI, you can export Models-as-a-Service usage data in CSV format for cost attribution and showback reporting to finance teams.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- The Cluster Observability Operator is installed and configured on your cluster.
- You have access to the OpenShift AI dashboard with administrator privileges.

Procedure

1. In the OpenShift AI dashboard, click **Observe & monitor** → **Dashboard**.
2. Click the **Usage** tab.
3. Optional: Configure filters to export specific usage data:
 - a. Select a time period from the **Time period** dropdown to export metrics for a specific timeframe.
 - b. Select filters for **User**, **Subscription**, or **Model** to export metrics for specific resources.
4. Hover over the Token Consumption by User table.
5. Click **Export as CSV** to download the usage data.
The system generates a CSV file containing the filtered usage data.
6. Save the CSV file to your local system.



NOTE

The exported CSV file contains subscription-level usage data for the selected time period and filters. This data is suitable for showback reporting but might not be billing-grade accurate. For production chargeback workflows, configure external metering and billing tools to consume this data.

Verification

- The CSV file downloads to your local system and contains usage data matching your selected filters and time period.

Next steps

- Provide the exported usage data to your finance team for cost attribution and showback reporting.

Additional resources

- [Subscriptions](#)
- [Models-as-a-Service administration troubleshooting](#)
- [Technology Preview Features Support Scope](#)

1.19. CONFIGURE EXTERNAL OIDC AUTHENTICATION FOR MODELS-AS-A-SERVICE

You can configure Models-as-a-Service (MaaS) to authenticate users with an external OpenID Connect (OIDC) identity provider, enabling enterprise-wide access without requiring OpenShift accounts for every user. This allows organizations to integrate MaaS with existing identity providers such as Keycloak and map external user groups to MaaS subscriptions for access control and quota enforcement.

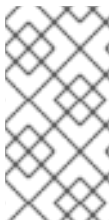
1.19.1. External OIDC authentication for Models-as-a-Service

In Red Hat OpenShift AI, you can authenticate Models-as-a-Service users through an external OpenID Connect (OIDC) identity provider, allowing them to use their existing corporate credentials without requiring OpenShift user accounts.

1.19.1.1. Authentication flow

Models-as-a-Service (MaaS) uses a two-tier authentication approach with external OIDC providers as follows:

1. **MaaS platform access:** Users retrieve an OIDC token from the external OIDC provider to access platform APIs. MaaS validates the OIDC token.
2. **Model access:** Users create API keys through the MaaS API by using their OIDC token. These API keys are used for programmatic access to models through the MaaS API gateway.



NOTE

When using external OIDC authentication, users create API keys through the MaaS API by using **curl** or other HTTP clients. The OpenShift AI dashboard does not support API key creation for external OIDC users. The dashboard uses OpenShift OAuth for authentication.

This approach provides industry-standard OIDC authentication for user login while maintaining centralized API key management for model access.

1.19.1.2. Group-based access control

MaaS validates external identity provider groups directly from OIDC tokens to determine user access. The OIDC token must include group claims for authorization to work. The validation process follows these steps:

1. **OIDC provider** defines user groups.
2. **OIDC token** includes group claims when a user authenticates. For example, a token might include **groups**: **["data-scientists", "ml-engineers"]**. The **groups** claim is required for MaaS authorization.

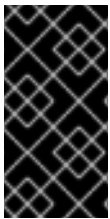
3. **MaaS subscriptions** define which groups have access to specific models. For example, a subscription might grant access to the **data-scientists** group.
4. **Authorization policies** validate the group claims in the user's OIDC token against the groups defined in subscriptions. If the token includes **data-scientists** and the subscription grants access to that group, the user is authorized.

Group names in MaaS subscriptions and authorization policies must match the group names in the OIDC token claims exactly. MaaS validates groups directly from the token without requiring OpenShift group creation or synchronization.

1.19.1.3. API key lifecycle

The API key creation process for OIDC-authenticated users follows these steps:

1. Users retrieve an OIDC token from the external OIDC provider.
2. Users call the MaaS API with their OIDC token to create an API key.
3. MaaS validates the OIDC token.
4. MaaS generates an API key with an expiration period specified in the API request, up to the maximum limit configured in the **MaasTenantConfig** custom resource.
5. Users use this API key for model access.



IMPORTANT

API keys capture the user's group memberships at the time of creation. If a user is removed from a group in the external OIDC provider, their existing API keys retain the original group associations and continue to work until revoked or expired. To immediately revoke access, administrators must manually revoke the user's API keys.

1.19.1.4. Use cases

Enterprise deployment: Organizations with existing identity providers such as Keycloak can integrate MaaS without creating OpenShift accounts for every user, reducing the overhead of managing a large user base.

Service provider deployment: Service providers offering MaaS to external customers can authenticate users through a centralized OIDC provider while maintaining subscription-based isolation and quota enforcement.

Regulated industries: Organizations with compliance requirements for centralized authentication and audit logging can use external OIDC integration while maintaining MaaS governance features.

1.19.2. Configure Models-as-a-Service for external OIDC users

In Red Hat OpenShift AI, you can configure Models-as-a-Service (MaaS) to authenticate users through an external OpenID Connect (OIDC) identity provider to enforce group-based access control without requiring OpenShift accounts for every user.

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have installed Red Hat OpenShift AI.
- You have deployed Models-as-a-Service.
- You have an external OIDC provider.
- You have registered a client application in your OIDC provider and obtained the issuer URL and client ID.
- You have created user groups in your external OIDC provider.
- Your external OIDC provider is configured to include group claims in ID tokens.

Procedure

1. Edit the **AITenant** custom resource to add the OIDC configuration, using one of the following methods:

Using the OpenShift console

- a. In the OpenShift console, navigate to **Administration → CustomResourceDefinitions**.
- b. Search for **AITenant** and click the resource name.
- c. Click the **Instances** tab.
- d. Click **models-as-a-service**.
- e. Click the **YAML** tab.
- f. In the YAML editor, locate the **spec** section and add or update the **oidc** configuration:

```
spec:
  oidc:
    clientId: <oidc-client-id>
    issuerUrl: <oidc-provider-issuer-url>
    ttl: <cache-duration>
```

where:

<oidc-client-id>

Specifies the client ID for your MaaS application registered with the OIDC provider.

<oidc-provider-issuer-url>

Specifies the OIDC issuer URL for your external identity provider.

<cache-duration>

Specifies how long, in seconds, the system caches your identity provider's signing keys before refreshing them. The default value is **300**. The minimum value is **30**.

- a. Click **Save**.

Using the OpenShift CLI

Run the following command:

```
$ oc patch aitenants.maas.opendatahub.io models-as-a-service -n ai-tenants \
--type merge \
-p { "spec": { "oidc": { "clientId": "<oidc-client-id>", "issuerUrl": "<oidc-provider-issuer-url>", "ttl": <cache-duration> } } }
```

2. Create MaaS subscriptions that include the groups from your OIDC provider.



NOTE

Group names in subscriptions must match the group names in the OIDC token claims exactly. MaaS validates group memberships directly from the OIDC token.

When configuring subscriptions, enter group names exactly as they appear in your OIDC provider's group claims. For example, if your OIDC token includes **groups: ["data-scientists"]**, enter **data-scientists** in the subscription.

For information about creating subscriptions, see [Managing subscriptions for Models-as-a-Service](#).

Verification

To verify external OIDC authentication, obtain a user access token from your identity provider by using an interactive flow such as authorization code or device grant.

1. Use the OIDC token to list available models:

```
$ curl -H "Authorization: Bearer <oidc-token>" \
https://<maas-gateway-url>/maas-api/v1/models
```

where:

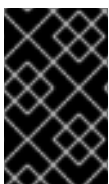
<oidc-token>

Specifies the user access token obtained from your OIDC provider.

<maas-gateway-url>

Specifies your MaaS gateway URL.

If authentication is successful, the API returns a list of models available to the groups in your token. An empty list means the token is valid but the user has no matching subscription groups.



IMPORTANT

API keys capture group memberships at creation time. If a user is removed from a group in the external OIDC provider, their existing API keys continue to work until revoked or expired. Administrators must manually revoke API keys to immediately revoke access.



NOTE

If you are upgrading from a previous version that used the **Tenant** custom resource for OIDC configuration, existing **Tenant/default-tenant externalOIDC** values are migrated to the **AITenant** custom resource automatically during upgrade.

Next steps

- [Create an API key](#)

Additional resources

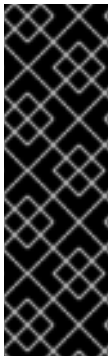
- [Manage API keys using the MaaS API](#)
- [Create a subscription using YAML](#)

1.20. MANAGE MAAS MULTI-TENANCY

You can provision isolated tenants for Models-as-a-Service so that multiple teams or business units share a single MaaS platform without accessing each other's resources. Each tenant receives a dedicated gateway, identity realm, namespace, and API infrastructure.

1.20.1. Models-as-a-Service multi-tenancy

You can provision isolated tenants for Models-as-a-Service (MaaS) so that multiple teams or business units share a single MaaS platform without accessing each other's resources. Each tenant receives a dedicated gateway, identity realm, namespace, and API infrastructure. Multi-tenancy is a Technology Preview feature in Red Hat OpenShift AI.



IMPORTANT

MaaS multi-tenancy is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

In a multitenant deployment, you create an **AITenant** custom resource for each organizational team. The controller automatically bootstraps the tenant infrastructure: a dedicated namespace, a per-tenant **maas-api** instance, gateway-scoped authentication and rate-limit policies, and tenant-admin RBAC Roles. Each tenant operates as an independent MaaS environment from the user's perspective. Users within a tenant create API keys, manage subscriptions, and access models through the tenant's dedicated gateway endpoint.

Default tenant and additional tenants:

When you deploy MaaS, the controller automatically creates a default tenant named **models-as-a-service**. This default tenant uses the namespace **models-as-a-service** and references the **maas-default-gateway** Gateway. The controller migrates the previous **Tenant** resource to a **MaasTenantConfig** resource, preserving existing API keys, subscriptions, and models.

Additional tenants are provisioned by creating **AITenant** custom resources in the management namespace, which is **ai-tenants** by default. For each additional tenant, the controller creates a namespace named **ai-tenant-`<aitenant_name>`**, deploys a per-tenant **maas-api** instance, and configures gateway-scoped authentication policies.

Tenant isolation boundaries:

Each tenant has isolated access to the following resources:

- **Identity realm:** Each tenant can use a separate OIDC identity provider or the OpenShift **TokenReview** identity. API keys are scoped to the tenant where they were created and cannot authenticate against other tenants.
- **Gateway:** Each tenant requires a dedicated Gateway API Gateway. The gateway enforces tenant-specific authentication and authorization rules.
- **Namespace:** Tenant-scoped MaaS resources such as subscriptions, authorization policies, and model references are created in the tenant namespace.
- **API keys and JWT tokens** Tokens minted for one tenant are rejected by other tenants' gateways.
- **Subscriptions and authorization policies** Each tenant manages its own subscriptions and authorization policies independently.
- **Usage data:** Prometheus metrics, OpenTelemetry (OTEL) traces, and structured logs include the tenant name, enabling per-tenant observability filtering.

The following resources are shared across all tenants:

- **GPU compute and model weights** Model serving infrastructure is shared across tenants.
- **PostgreSQL database:** All tenants share the same database instance for API key storage.
- **Cluster infrastructure:** OpenShift control plane, networking, and storage.
- **Authorino and Limitador:** Platform-level authentication and rate-limiting components.
- **Observability stack:** Prometheus, Grafana, and OpenTelemetry collectors.

Layered access control model:

Multi-tenancy uses three layers of access control to enforce tenant boundaries:

Namespace-scoped Kubernetes RBAC

Controls which users can create and manage MaaS custom resources such as subscriptions, authorization policies, and model references within a tenant namespace. The controller creates tenant-admin and object-admin Roles, but platform administrators must create **RoleBindings** to grant access.

Gateway-scoped Kuadrant AuthPolicies

A single **maas-gateway-auth AuthPolicy** per tenant gateway aggregates authentication and authorization rules for all models served through that tenant. The **AuthPolicy** enforces API key validation, OIDC token verification, subscription checks, and tenant-gateway isolation through Common Expression Language (CEL) expressions.

MaaS subscription-based access control

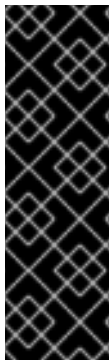
Within each tenant, subscriptions define which groups have quota for which models with configurable token rate limits. Authorization policies authorize groups to access model endpoints through the gateway. Both a subscription and an authorization policy are required for users to access models.

Additional resources

- [Multi-tenancy prerequisites for Models-as-a-Service](#)
- [Provision a MaaS tenant](#)
- [AITenant custom resource reference](#)
- [MaaS tenant RBAC reference](#)

1.20.2. Multi-tenancy prerequisites for Models-as-a-Service

Before you provision additional MaaS tenants, verify that the base MaaS installation and per-tenant infrastructure prerequisites are in place. Missing prerequisites cause **AITenant** creation to fail or remain in **Pending** status.



IMPORTANT

MaaS multi-tenancy is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Platform prerequisites:

- Base MaaS installation is deployed and the default tenant is in **Ready** state:

```
$ oc get aitenant models-as-a-service -n ai-tenants
```

The **READY** column shows **True**.

- Gateway API CRDs and the **openshift-default GatewayClass** are available:

```
$ oc get gatewayclass openshift-default
```

- cert-manager is installed on the cluster for TLS certificate provisioning.
- Red Hat Connectivity Link is installed with Kuadrant v1.4.2 or later. The gateway-scoped **AuthPolicy** requires **spec.defaults.rules** support introduced in Kuadrant v1.4.2.
- You have cluster administrator privileges or equivalent access to the **ai-tenants** management namespace.

Per-tenant infrastructure prerequisites:

Each additional tenant requires the following infrastructure before you create the **AITenant** custom resource:

Dedicated Gateway

A Gateway API Gateway with a unique hostname and the following required annotations:

- **opendatahub.io/managed: "false"** – Prevents the ODH Model Controller from overriding MaaS-managed authorization policies.
- **security.opendatahub.io/authorino-tls-bootstrap: "true"** – Enables TLS communication between the Gateway and Authorino.
The Gateway must use the **openshift-default GatewayClass** and a listener on port 443 with HTTPS protocol. Each **AITenant** must reference a unique Gateway. Two **AITenant** resources cannot share the same Gateway.

OpenShift Route

If your cluster does not support **LoadBalancer** services, such as bare metal or restricted environments, you might need to create an OpenShift Route for external access if the Gateway controller does not auto-provision one. Check for an auto-provisioned Route before creating one manually:

```
$ oc get route -n openshift-ingress -l gateway.networking.k8s.io/gateway-name=<tenant_name>
```

AITenant naming rules:

- Must be a valid DNS-1123 label: lowercase alphanumeric characters and hyphens, starting and ending with an alphanumeric character.
- Maximum 41 characters. This limit exists because the controller appends the tenant name as a suffix to per-tenant resource names, and the combined name must fit within the Kubernetes 63-character resource name limit.
- Must not conflict with existing **AITenant** names.

AITenant placement:

- **AITenant** resources must be created in the management namespace, which is **ai-tenants** by default.
- The admission webhook rejects **AITenant** creation in any other namespace.

Optional prerequisites:

External OIDC identity provider

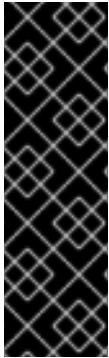
If you want to configure Bring Your Own Identity Provider (BYOIDP) for a tenant, you need the OIDC issuer URL and client ID for the external identity provider. OIDC configuration is optional. When omitted, the tenant uses OpenShift **TokenReview** for authentication.

Additional resources

- [Gateway requirements for Models-as-a-Service](#)
- [Platform and Operator prerequisites for Models-as-a-Service](#)

1.20.3. Provision a MaaS tenant

You can provision an isolated MaaS tenant by creating a dedicated Gateway and an **ALTenant** custom resource. The controller bootstraps the tenant namespace, per-tenant API service, authentication policies, and RBAC Roles.



IMPORTANT

MaaS multi-tenancy is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Prerequisites

- Base MaaS installation is deployed and the default tenant is in **Ready** state.
- Gateway API CRDs and the **openshift-default GatewayClass** are available on the cluster.
- cert-manager is installed on the cluster.
- You have cluster administrator privileges for the OpenShift cluster.
- You have access to OpenShift CLI (**oc**).
- You have reviewed the [multi-tenancy prerequisites](#).

Procedure

1. Set environment variables for the tenant name and cluster domain:

```
$ TENANT_NAME="<tenant_name>"
$ CLUSTER_DOMAIN=$(oc get ingresses.config.openshift.io cluster -o
jsonpath={.spec.domain})
$ GATEWAY_HOSTNAME="${TENANT_NAME}-maas.${CLUSTER_DOMAIN}"
$ GATEWAY_NAMESPACE="openshift-ingress"
```

where:

<tenant_name>

Specifies a unique name for the tenant. Must be a valid DNS-1123 label with a maximum of 41 characters, such as **red-team** or **finance-dept**.

2. Label the infrastructure namespace for Gateway access so that **HTTPRoutes** can attach to the tenant Gateway:

```
$ oc label namespace <infrastructure_namespace> \
"maas.opendatahub.io/gateway-access-${TENANT_NAME}=true" --overwrite
```

where:

<infrastructure_namespace>

Specifies the infrastructure namespace where the **maas-api** deployment runs. The default is **redhat-ai-gateway-infra** for OpenShift AI.

If you have model namespaces where **LLMInferenceService** resources run, label those namespaces as well:

```
$ oc label namespace <model_namespace> \
  "maas.opendatahub.io/gateway-access-${TENANT_NAME}=true" --overwrite
```

3. Create a dedicated Gateway for the tenant by using the following YAML and running **oc apply -f <filename>**:

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: ${TENANT_NAME}
  namespace: ${GATEWAY_NAMESPACE}
  annotations:
    opendatahub.io/managed: "false"
    security.opendatahub.io/authorino-tls-bootstrap: "true"
spec:
  gatewayClassName: openshift-default
  listeners:
    - name: https
      hostname: "${GATEWAY_HOSTNAME}"
      port: 443
      protocol: HTTPS
      allowedRoutes:
        namespaces:
          from: Selector
          selector:
            matchLabels:
              maas.opendatahub.io/gateway-access-${TENANT_NAME}: "true"
      tls:
        mode: Terminate
        certificateRefs:
          - group: ""
            kind: Secret
            name: router-certs-default
```

4. Verify that the OpenShift Gateway controller auto-provisioned a Route for external access:

```
$ oc get route -n ${GATEWAY_NAMESPACE} \
  -l gateway.networking.k8s.io/gateway-name=${TENANT_NAME}
```

Depending on the cluster configuration, the Gateway controller might not auto-provision a Route. If no Route exists, verify the Gateway service name and create a Route manually:

```
$ oc get svc -n ${GATEWAY_NAMESPACE} \
  -l gateway.networking.k8s.io/gateway-name=${TENANT_NAME}
```

Note the service name from the output and use it in the following **Route** definition:

■

```

apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: ${TENANT_NAME}-gateway
  namespace: ${GATEWAY_NAMESPACE}
spec:
  host: "${GATEWAY_HOSTNAME}"
  to:
    kind: Service
    name: <gateway_service_name>
    weight: 100
  port:
    targetPort: https
  tls:
    termination: reencrypt
    insecureEdgeTerminationPolicy: Redirect
    wildcardPolicy: None

```

5. Verify that the Gateway is Programmed:

```
$ oc get gateway ${TENANT_NAME} -n ${GATEWAY_NAMESPACE}
```

The **PROGRAMMED** column shows **True**.

6. Create the **AITenant** custom resource in the **ai-tenants** namespace:

```

apiVersion: maas.opendatahub.io/v1alpha1
kind: AITenant
metadata:
  name: ${TENANT_NAME}
  namespace: ai-tenants
spec:
  gateway:
    name: ${TENANT_NAME}

```

To configure an external OIDC identity provider for the tenant, add the **oidc** section:

```

apiVersion: maas.opendatahub.io/v1alpha1
kind: AITenant
metadata:
  name: <tenant_name>
  namespace: ai-tenants
spec:
  gateway:
    name: <tenant_name>
  oidc:
    issuerUrl: "https://<oidc_provider>/realms/<tenant_realm>"
    clientId: <client_id>
    ttl: 300

```

where:

issuerUrl

Specifies the OIDC issuer URL for the external identity provider.

clientId

Specifies the OIDC client ID registered with the identity provider.

ttl

Specifies the cache time-to-live in seconds for OIDC token validation. The default is 300 seconds.

- Wait for the **AITenant** to reach the **Ready** status:

```
$ oc get aitenant ${TENANT_NAME} -n ai-tenants -w
```

The **READY** column shows **True** when the controller has bootstrapped all tenant resources.

Verification

- Verify that the tenant namespace was created with the expected labels:

```
$ oc get namespace ai-tenant-${TENANT_NAME} --show-labels
```

The output includes the labels **ai-gateway.opendatahub.io/tenant=<tenant_name>** and **maas.opendatahub.io/managed-by-aitenant=true**.

- Verify that the **MaasTenantConfig** CR exists in the tenant namespace:

```
$ oc get maastenantconfig default-tenant -n ai-tenant-${TENANT_NAME}
```

- Verify that the per-tenant **maas-api** deployment is running:

```
$ oc get deployment maas-api-${TENANT_NAME} -n <infrastructure_namespace>
```

where:

<infrastructure_namespace>

Specifies the infrastructure namespace where the **maas-api** deployment runs. The default is **redhat-ai-gateway-infra** for OpenShift AI.

The **READY** column shows **1/1**.

Next steps

- [Grant MaaS tenant access](#)
- [Verify a multitenant MaaS deployment](#)

1.20.4. Verify a multitenant MaaS deployment

After you provision a MaaS tenant, you can verify that the controller bootstrapped all required resources and that tenant isolation is working correctly.

Prerequisites

- You have provisioned at least one additional **AITenant** and it shows a **Ready** status.

- You have access to OpenShift CLI (**oc**).
- You have cluster administrator privileges for the OpenShift cluster.

Procedure

1. Set environment variables for the tenant you want to verify:

```
$ TENANT_NAME=<tenant_name>
$ TENANT_NS="ai-tenant-{$TENANT_NAME}"
```

2. Check the **AITenant** status and conditions:

```
$ oc get aitenant {$TENANT_NAME} -n ai-tenants
```

The **READY** column shows **True**. If the status is not **True**, inspect the conditions for details:

```
$ oc get aitenant {$TENANT_NAME} -n ai-tenants \
-o jsonpath='{.status.conditions}' | jq .
```

3. Verify that the tenant namespace exists with the expected labels:

```
$ oc get namespace {$TENANT_NS} -o jsonpath='{.metadata.labels}' | jq .
```

The output includes:

- **ai-gateway.opendatahub.io/tenant:** <tenant_name>
- **maas.opendatahub.io/managed-by-aitenant:** "true"

4. Confirm that the **MaasTenantConfig** CR exists in the tenant namespace:

```
$ oc get maastenantconfig default-tenant -n {$TENANT_NS}
```

5. Verify that the per-tenant **maas-api** deployment is running and ready:

```
$ oc get deployment maas-api-{$TENANT_NAME} -n <infrastructure_namespace>
```

where:

<infrastructure_namespace>

Specifies the infrastructure namespace where the **maas-api** deployment runs. The default is **redhat-ai-gateway-infra** for OpenShift AI.

The **READY** column shows **1/1**.

6. Retrieve the gateway name and hostname for the tenant:

```
$ GATEWAY_NAME=$(oc get aitenant {$TENANT_NAME} -n ai-tenants \
-o jsonpath='{.spec.gateway.name}')
$ GATEWAY_HOST=$(oc get gateway {$GATEWAY_NAME} -n openshift-ingress \
-o jsonpath='{.spec.listeners[0].hostname}')
```

If **GATEWAY_HOST** is empty, retrieve the hostname from the **Route** instead:

```
$ GATEWAY_HOST=$(oc get route -n openshift-ingress \
-l gateway.networking.k8s.io/gateway-name=${GATEWAY_NAME} \
-o jsonpath='{.items[0].spec.host}')
```

7. Test model listing through the tenant gateway endpoint:

```
$ TOKEN=$(oc whoami -t)
$ curl -sSk "https://${GATEWAY_HOST}/v1/models" \
-H "Authorization: Bearer ${TOKEN}" \
-H "Content-Type: application/json" | jq .
```

A successful response returns **HTTP 200** with the models configured for this tenant.

8. Test that unauthenticated requests are rejected:

```
$ curl -sSk -o /dev/null -w "%{http_code}\n" \
"https://${GATEWAY_HOST}/v1/models"
```

The expected response depends on your Authorino gateway configuration. Typical codes include **401**, which indicates an authentication requirement, or **403**, which indicates denied access.

9. Test tenant isolation by using one tenant's API key against another tenant's gateway:

```
$ DEFAULT_HOST=$(oc get gateway maas-default-gateway -n openshift-ingress \
-o jsonpath={.spec.listeners[0].hostname})
$ curl -sSk -o /dev/null -w "%{http_code}\n" \
-H "Authorization: Bearer <tenant_api_key>" \
"https://${DEFAULT_HOST}/v1/models"
```

where:

<tenant_api_key>

Specifies an API key created in the additional tenant. To generate an API key, see [Generate a temporary API key](#) or [Manage API keys using the Models-as-a-Service API](#).

The expected response code is **404**, confirming that cross-tenant access is blocked. The **404** response prevents information disclosure about models in other tenants.

Troubleshooting

If the **AITenant** is stuck in **Pending** status, check the following common causes:

- **Gateway not found or not Programmed:** Verify that the Gateway exists in the **openshift-ingress** namespace and shows **PROGRAMMED: True**.
- **Webhook rejection:** The **AITenant** must be created in the **ai-tenants** namespace. Creating it in any other namespace is rejected by the admission webhook.
- **Gateway uniqueness violation:** Each **AITenant** must reference a unique Gateway. The webhook rejects creation if the Gateway is already claimed by another **AITenant**.

- **Database secret missing:** Verify that the **maas-db-config** secret exists in the infrastructure namespace.
- **Authorino not deployed:** Verify that Red Hat Connectivity Link is installed and the **Kuadrant** custom resource is ready.

If **MaaSSubscription** or **MaaSAuthPolicy** creation is rejected, verify that the **MaasTenantConfig** CR exists in the target namespace. The admission webhook requires a **MaasTenantConfig** CR before these resources can be created.

1.20.5. Grant MaaS tenant access

After you provision a MaaS tenant, the controller creates Roles but does not create **RoleBindings**. You must create **RoleBindings** to grant users, groups, or service accounts access to manage tenant resources.

Prerequisites

- You have provisioned an **AITenant** and it shows **Ready** status.
- You have cluster administrator privileges for the OpenShift cluster.
- You have access to OpenShift CLI (**oc**).

Procedure

1. Identify the tenant-admin Role name in the tenant namespace.

For a tenant named **red-team**, the Role name is **aitenant-red-team-tenant-admin**.

For long tenant names, the controller truncates the Role name and appends a hash. To find the exact Role name, list Roles by tenant label:

```
$ oc get roles -n ai-tenant-<tenant_name> \
-l ai-gateway.opendatahub.io/tenant=<tenant_name>
```

2. Create a **RoleBinding** to grant tenant-admin access.

To grant access to a group:

```
$ oc create rolebinding <tenant_name>-tenant-admins \
--role=aitenant-<tenant_name>-tenant-admin \
--group=<group_name> \
-n ai-tenant-<tenant_name>
```

where:

<tenant_name>

Specifies the name of the tenant.

<group_name>

Specifies the OpenShift group or OIDC group to grant access.

To grant access to an individual user:

```
apiVersion: rbac.authorization.k8s.io/v1
```

```

kind: RoleBinding
metadata:
  name: <tenant_name>-tenant-admin
  namespace: ai-tenant-<tenant_name>
subjects:
  - kind: User
    apiGroup: rbac.authorization.k8s.io
    name: <username>
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: aitenant-<tenant_name>-tenant-admin

```

To grant access to a **ServiceAccount**:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: <tenant_name>-admin-serviceaccount
  namespace: ai-tenant-<tenant_name>
subjects:
  - kind: ServiceAccount
    name: <serviceaccount_name>
    namespace: <serviceaccount_namespace>
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: aitenant-<tenant_name>-tenant-admin

```

- Optional: Create an object-admin **RoleBinding** in the infrastructure namespace to grant read access to the **AITenant** object:

```

$ oc create rolebinding <tenant_name>-aitenant-reader \
  --role=aitenant-<tenant_name>-object-admin \
  --group=<group_name> \
  -n ai-tenants

```

This **RoleBinding** allows tenant administrators or dashboards to view the **AITenant** bootstrap status.

Verification

- Verify tenant namespace permissions.
If you granted access to a user:

```

$ oc auth can-i create maassubscriptions.maas.opendatahub.io \
  --as=<username> \
  -n ai-tenant-<tenant_name>

```

If you granted access to a group:

```
$ oc auth can-i create maassubscriptions.maas.opendatahub.io \
--as=<username> --as-group=<group_name> \
-n ai-tenant-<tenant_name>
```

Expected output: **yes**

- Verify **AITenant** read access in the **ai-tenants** namespace.
If you created the object-admin **RoleBinding** for a user:

```
$ oc auth can-i get aitenants.maas.opendatahub.io/<tenant_name> \
--as=<username> \
-n ai-tenants
```

If you created the object-admin **RoleBinding** for a group:

```
$ oc auth can-i get aitenants.maas.opendatahub.io/<tenant_name> \
--as=<username> --as-group=<group_name> \
-n ai-tenants
```

Expected output: **yes**



WARNING

User-created **RoleBindings** are not deleted when you delete an **AITenant**. If you re-create a tenant with the same name, stale **RoleBindings** that reference the re-created Roles can re-enable access. Review and remove stale **RoleBindings** before or after deleting a tenant.

Next steps

After granting RBAC access, configure model access within the tenant:

- [Publish a model to Models-as-a-Service by using YAML](#)
- [Create a subscription using YAML](#)
- [Create an authorization policy using YAML](#)



NOTE

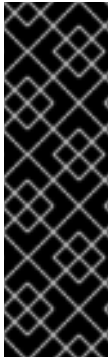
When creating **MaaSModelRef**, **MaaSSubscription**, and **MaaSAuthPolicy** resources for a non-default tenant, use the tenant namespace **ai-tenant-<tenant_name>** instead of **models-as-a-service**. The admission webhook requires a **MaasTenantConfig** resource in the namespace, which the controller creates automatically when the **AITenant** reaches **Ready** status.

Additional resources

- [MaaS tenant RBAC reference](#)

1.20.6. Delete a MaaS tenant

You can delete a MaaS tenant by removing the **AITenant** custom resource. The controller cleans up controller-managed resources but preserves the tenant namespace and user-created resources to protect user data.



IMPORTANT

MaaS multi-tenancy is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster.
- You have access to OpenShift CLI (**oc**).
- You have identified the tenant to delete.

Procedure

1. List existing **AITenant** resources to confirm the tenant to delete:

```
$ oc get aitenant -n ai-tenants
```

2. Optional: Review MaaS resources in the tenant namespace before deletion:

```
$ oc get maassubscription,maasauthpolicy,maasmodelref -n ai-tenant-<tenant_name>
```

3. Delete the **AITenant** custom resource:

```
$ oc delete aitenant <tenant_name> -n ai-tenants
```

where:

<tenant_name>

Specifies the name of the tenant to delete.

The controller finalizer runs an ordered cleanup sequence: revokes active API keys, deletes the **MaasTenantConfig** CR, removes controller-created Roles and **RoleBindings**, releases gateway authentication policies and claim resources, and removes the finalizer.

4. Monitor the deletion progress:

```
$ oc get aitenant <tenant_name> -n ai-tenants -w
```

The **AITenant** shows a **Terminating** phase during cleanup. Deletion might take several minutes while the controller completes API key revocation and resource cleanup. When the finalizer completes, the **AITenant** resource is removed.

5. Verify that controller-managed resources were cleaned up:

```
$ oc get aitenant <tenant_name> -n ai-tenants
```

Expected output: **Error from server (NotFound): aitenants.maas.opendatahub.io "<tenant_name>" not found**

```
$ oc get maastenantconfig default-tenant -n ai-tenant-<tenant_name>
```

Expected output: **Error from server (NotFound)** confirming the **MaasTenantConfig** was deleted.

6. Remove user-created **RoleBindings** in the preserved tenant namespace:



IMPORTANT

Stale **RoleBindings** that reference the deleted tenant's Roles can silently re-enable access for former authorized users if you re-create a tenant with the same name. Always review and remove stale **RoleBindings** before or after deleting a tenant.

```
$ oc get rolebindings -n ai-tenant-<tenant_name>
```

Delete any stale **RoleBindings** that reference the deleted tenant's Roles:

```
$ oc delete rolebinding <rolebinding_name> -n ai-tenant-<tenant_name>
```

7. Optional: Delete the Gateway and Route if they are no longer needed:

```
$ oc delete gateway <gateway_name> -n openshift-ingress
$ oc delete route <gateway_name> -n openshift-ingress
```

where:

<gateway_name>

Specifies the name of the Gateway associated with the tenant. The Gateway name defaults to the **AITenant** name but might differ if a custom name was specified in **spec.gateway.name**.

The controller does not manage or delete the Gateway or Route because they are external prerequisites. A Route exists only if one was manually created or auto-provisioned by the Gateway controller.

8. Optional: Delete the preserved tenant namespace if no user data needs to be retained:

```
$ oc delete namespace ai-tenant-<tenant_name>
```

The following table summarizes the cleanup behavior for tenant resources:

Table 1.7. Tenant deletion cleanup behavior

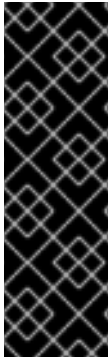
Resource	Cleanup behavior	Action required
AITenant CR	Deleted automatically	None
MaasTenantConfig CR	Deleted automatically	None
Controller-created Roles	Deleted automatically	None
Per-tenant maas-api Deployment	Deleted automatically	None
Gateway AuthPolicy	Deleted automatically	None
User-created RoleBindings	Preserved	Review and delete manually
HTTPRoute (internal API routing)	Deleted automatically	None
Gateway and OpenShift Route (external access)	Preserved	Delete manually if no longer needed
Tenant namespace	Preserved	Delete manually after data review

**WARNING**

If you re-create a tenant with the same name without removing stale **RoleBindings**, those **RoleBindings** reference the newly created Roles and can re-enable access for former authorized users. Always review **RoleBindings** in the tenant namespace before or after deleting a tenant.

1.20.7. AITenant custom resource reference

The **AITenant** custom resource bootstraps an isolated MaaS tenant from the **ai-tenants** management namespace. You can use this reference to understand the **AITenant** spec fields, status fields, validation rules, and deletion behavior.



IMPORTANT

MaaS multi-tenancy is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

API details:

- API group: **maas.opendatahub.io**
- API version: **v1alpha1**
- Kind: **AITenant**
- Short name: **ait**
- Scope: Namespaced

Spec fields:

Table 1.8. AITenantSpec fields

Field	Type	Required	Description
spec.gateway	AITenantGatewayRef	No	References the Gateway API Gateway for this tenant. If omitted, the Gateway name defaults to the AITenant name. The Gateway namespace is set by controller configuration, not by the AITenant spec.
spec.gateway.name	string	No	Name of the Gateway in the controller-configured Gateway namespace. Maximum 63 characters. Must be a valid DNS-1123 label.
spec.oidc	TenantExternalOIDCConfig	No	External OIDC identity provider settings for this tenant. When omitted, the tenant uses OpenShift TokenReview for authentication.
spec.oidc.issuerUrl	string	Yes, if oidc is set	OIDC issuer URL for the external identity provider.
spec.oidc.clientId	string	Yes, if oidc is set	OIDC client ID registered with the identity provider. Tokens must include an azp (authorized party) claim matching this value.
spec.oidc.ttl	integer	No	Cache time-to-live in seconds for OIDC token validation. Minimum: 30. Default: 300.

Field	Type	Required	Description
spec.rbac	AITenantRBACConfig	No	Deprecated. Accepted for schema compatibility but ignored by the controller. Create standard Kubernetes RoleBindings instead.

Status fields:

Table 1.9. AITenantStatus fields

Field	Type	Description
status.phase	string	High-level lifecycle phase. Values: Pending, Active, Failed, Terminating .
status.tenantNamespace	string	Reconciled tenant namespace name.
status.gatewayRef	TenantGatewayRef	Resolved reference to the tenant Gateway, including the Gateway name and namespace.
status.conditions	[]Condition	Latest observations of the tenant bootstrap state. The Ready condition indicates whether all tenant resources are reconciled.

Validation rules:

The following validation rules are enforced by CRD-level validation and the admission webhook:

Name constraints

The **AITenant** name must be a valid DNS-1123 label: lowercase alphanumeric characters and hyphens, starting and ending with an alphanumeric character. The maximum length is 41 characters. This limit exists because per-tenant resource names use the format **<base_name>-<tenant_name>**, and the combined name must fit within the Kubernetes 63-character resource name limit.

Namespace restriction

AITenant resources must be created in the management namespace, which is **ai-tenants** by default. The admission webhook rejects creation in any other namespace.

Gateway uniqueness

Each **AITenant** must reference a unique Gateway. The admission webhook rejects creation if the specified Gateway is already claimed by another **AITenant**. This validation runs on both create and update operations.

Namespace derivation:

The controller derives the tenant namespace from the **AITenant** name by using the pattern **ai-tenant-<aitenant_name>**. The default tenant **models-as-a-service** uses the namespace **models-as-a-service** for backward compatibility.

Bootstrapped resources:

When an **AITenant** reaches **Active** status, the controller has created the following resources:

Table 1.10. Resources bootstrapped by AITenant

Resource	Location	Name
Namespace	Cluster	ai-tenant-<code><tenant_name></code>
MaasTenantConfig CR	Tenant namespace	default-tenant
maas-api Deployment	Infrastructure namespace	maas-api-<code><tenant_name></code>
Gateway AuthPolicy	Gateway namespace	<code><tenant_name>-maas-auth</code>
Tenant-admin Role	Tenant namespace	aitenant-<code><tenant_name></code>-tenant-admin
Object-admin Role	ai-tenants	aitenant-<code><tenant_name></code>-object-admin

Deletion behavior:

Deleting an **AITenant** triggers the controller finalizer, which performs an ordered cleanup:

1. Revokes active API keys through the tenant **maas-api** instance.
2. Deletes the **MaasTenantConfig** CR and tenant-scoped MaaS resources.
3. Removes controller-created Roles and **RoleBindings**.
4. Removes tenant ownership metadata from the namespace.
5. Releases the gateway **AuthPolicy** and gateway claim resources.
6. Removes the finalizer.

The tenant namespace is preserved to protect user data. User-created **RoleBindings** are not deleted. The **HTTPRoute** (internal API routing) is deleted automatically. The Gateway and OpenShift Route (external access) are not modified or deleted because they are external prerequisites.

If cleanup exceeds the configured deletion timeout, the finalizer is forcibly removed.

MaasTenantConfig specification:

The **MaasTenantConfig** is a namespace-scoped singleton automatically created in each tenant namespace during **AITenant** reconciliation. The **MaasTenantConfig** must reach **Ready** state before the **AITenant** is marked **Active**. It stores per-tenant runtime configuration for API keys and telemetry. Platform context such as gateway and OIDC settings belongs to the **AITenant**, not the **MaasTenantConfig**.

Table 1.11. MaasTenantConfig spec fields

Field	Type	Description
spec.apiKeys.maxExpirationDays	Integer	The maximum API key expiration duration in days. Minimum: 1 .
spec.telemetry.enabled	Boolean	Enables or disables telemetry and metrics collection for this tenant. Default: true .
spec.telemetry.metrics.captureOrganization	Boolean	Adds an organization dimension to telemetry metrics. Default: true .
spec.telemetry.metrics.captureModelUsage	Boolean	Adds a model dimension to telemetry metrics. Default: true .
spec.telemetry.metrics.captureUser	Boolean	Adds a user dimension to telemetry metrics containing the authenticated user ID. Default: false . Enabling this setting might have privacy implications.
spec.telemetry.metrics.captureGroup	Boolean	Adds a group dimension to telemetry metrics. Default: false .

Examples:

AITenant with external OIDC identity provider

```

apiVersion: maas.opendatahub.io/v1alpha1
kind: AITenant
metadata:
  name: red-team
  namespace: ai-tenants
spec:
  gateway:
    name: red-team
  oidc:
    issuerUrl: "https://keycloak.example.com/realms/red-team"
    clientId: red-team-maas
    ttl: 300

```

AITenant using OpenShift TokenReview authentication

```

apiVersion: maas.opendatahub.io/v1alpha1
kind: AITenant
metadata:
  name: blue-team
  namespace: ai-tenants
spec:
  gateway:
    name: blue-team

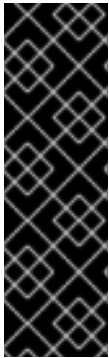
```

Additional resources

- [Provision a MaaS tenant](#)
- [Delete a MaaS tenant](#)
- [MaaS tenant RBAC reference](#)

1.20.8. MaaS tenant RBAC reference

When you create an **AITenant**, the controller creates two Roles for tenant administration. You can use this reference to understand the permissions granted by each Role and how to identify Role names for your tenants.



IMPORTANT

MaaS multi-tenancy is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Controller-created Roles:

The controller creates the following Roles for each **AITenant**:

Table 1.12. Roles created by the AITenant controller

Role	Namespace	Purpose
aitenant- <tenant_name>- tenant-admin	Tenant namespace	Manage tenant-scoped MaaS resources: subscriptions, authorization policies, tenant configuration, and model references.
aitenant- <tenant_name>- object-admin	ai-tenants infrastructure namespace	Read the specific AITenant object to view tenant bootstrap status.

For long tenant names, the controller truncates the Role name and appends a hash to fit within the Kubernetes 63-character name limit. To find the exact Role name, list **Roles** by **tenant_name**:

```
$ oc get roles -A -l ai-gateway.opendatahub.io/tenant=<tenant_name>
```

Tenant-admin Role permissions:

The tenant-admin Role in the tenant namespace grants the following permissions:

Table 1.13. Tenant-admin Role permissions

Resource	Allowed verbs
maasauthpolicies.maas.opendatahub.io	create, delete, get, list, patch, update, watch
maassubscriptions.maas.opendatahub.io	create, delete, get, list, patch, update, watch
maastenantconfigs.maas.opendatahub.io (restricted to default-tenant instance)	get, update, patch
maasmodelrefs.maas.opendatahub.io	get, list, watch

The object-admin Role in the **ai-tenants** infrastructure namespace grants **get** access to the specific **AITenant** object.

Role lifecycle:

Controller-created Roles and their controller-owned **RoleBindings** are deleted when the **AITenant** is deleted. User-created **RoleBindings** in the tenant namespace and infrastructure namespace are preserved.



WARNING

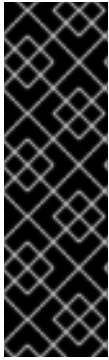
Review and remove stale user-created **RoleBindings** before deleting or re-creating a tenant with the same name. Stale **RoleBindings** that reference the re-created Roles can re-enable access for former authorized users.

Additional resources

- [Grant MaaS tenant access](#)
- [AITenant custom resource reference](#)

1.20.9. MaaS multi-tenancy known limitations

Multi-tenancy for Models-as-a-Service has the following known limitations in the current Technology Preview release. Review these limitations before you provision additional tenants.



IMPORTANT

MaaS multi-tenancy is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

External models are supported for the default tenant only:

ExternalModel custom resources are supported only for the default tenant in multitenant deployments. Non-default tenant instances have the **ExternalModel** controller disabled to prevent **HTTPRoute** conflicts between tenants.

When per-tenant instances set the **HTTPRoute parentRef** to different tenant gateways, the resulting conflict causes continuous flapping that breaks external model routing for all tenants. To prevent this, the controller injects **DISABLE_EXTERNAL_MODEL_CONTROLLER=true** for non-default tenants.

To use external models in a multitenant deployment, access them through the default tenant gateway.

Cross-tenant access prevention:

API keys, OIDC tokens, subscriptions, and model discovery are scoped per tenant. The following isolation mechanisms prevent cross-tenant access:

- **API key rejection:** An API key created in one tenant returns an HTTP 404 error with **{error: API key not found}** when used against another tenant's gateway. Each tenant's **maas-api** instance queries only its own tenant's API keys.
- **OIDC token rejection:** OIDC tokens are validated against the tenant-specific identity provider configuration. Tokens from a different tenant's identity provider are rejected by the gateway **AuthPolicy**.
- **Subscription scoping:** Each tenant manages its own subscriptions independently. The **GET /v1/subscriptions** endpoint returns only the requesting tenant's data.
- **Model discovery scoping:** The **GET /v1/models** endpoint returns only models associated with the tenant through **MaaSModelRef** resources in the tenant namespace or with a matching **tenantRef**.

Technology Preview limitations:

- The **spec.rbac** field on the **AITenant** custom resource is deprecated and ignored. Create standard Kubernetes **RoleBindings** to grant tenant access.
- Multi-tenancy is a Technology Preview feature and is not supported with Red Hat production service level agreements.

Additional resources

- [Models-as-a-Service multi-tenancy overview](#)
- [About external models for Models-as-a-Service](#)

1.21. CONFIGURE EXTERNAL MODELS FOR MODELS-AS-A-SERVICE



IMPORTANT

External models for Models-as-a-Service is currently available in Red Hat OpenShift AI as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

You can configure Models-as-a-Service (MaaS) to route inference requests to models hosted by external cloud providers such as OpenAI, Anthropic, AWS Bedrock, Azure OpenAI, or Google Vertex AI. This enables unified governance and authentication for both locally deployed models and external model endpoints through the same MaaS gateway.

1.21.1. External models for Models-as-a-Service

In Red Hat OpenShift AI, you can use Models-as-a-Service (MaaS) external models to route inference requests to large language models hosted outside the cluster, such as OpenAI, Anthropic, AWS Bedrock, Azure OpenAI, and Google Vertex AI, through the same MaaS gateway you use for locally deployed models.

You configure external models by creating two resources:

ExternalProvider

Defines the provider connection, including the endpoint URL, authentication type, and a reference to a Kubernetes secret containing the provider API key. Multiple models can share the same **ExternalProvider**, so you define provider credentials once and rotate them in a single operation.

ExternalModel

References one or more **ExternalProvider** resources and specifies the target model identifier and API format for request translation. Users reference the **ExternalModel** name when making inference requests. You can add optional annotations to control TLS origination and port configuration for the external endpoint. For more information, see [Configure routing to external model providers](#).



IMPORTANT

In multitenant deployments, **ExternalModel** custom resources are supported only for the default tenant. Non-default tenant instances have the **ExternalModel** controller disabled to prevent HTTPRoute conflicts between tenants. Access external models through the default tenant gateway.

External models appear in the MaaS dashboard alongside locally deployed models. Users access external models the same way as locally deployed models: administrators include the model in a subscription, users create a MaaS API key, and users send inference requests through the gateway. By default, the gateway translates all requests to the format expected by the external provider. With multi-provider API passthrough, users can also send requests in native provider formats such as the Anthropic Messages API or the OpenAI Responses API without translation.

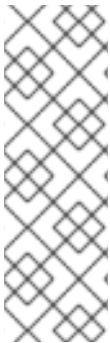
External models differ from locally deployed models in two ways: external models use a two-tier authentication pattern, and token limits at the external provider level are shared across all users.

Two-tier authentication

External model access uses a two-tier authentication pattern:

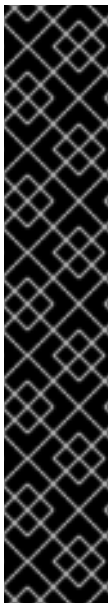
1. **User authentication:** Users authenticate to MaaS by using their MaaS API key. MaaS validates the user subscription and confirms access to the requested external model.
2. **Provider authentication:** MaaS automatically injects the provider API key from the Kubernetes secret referenced by the **ExternalProvider** resource when forwarding requests to the external provider. The authentication header varies by provider type: Anthropic providers use the **x-api-key** header, Azure providers use the **api-key** header, and OpenAI and other providers use the **Authorization: Bearer** header.

As a result, users need only their MaaS API key to access external models. The provider API key is managed by administrators through the **ExternalProvider** resource and shared across all users of that external model.



NOTE

When any **ExternalModel** in the cluster has **apiFormat: messages**, the MaaS gateway enables **x-api-key** header authentication cluster-wide. This allows Anthropic SDK clients to authenticate to any model in the cluster by using the **x-api-key** header. If you delete all **ExternalModel** resources that have **apiFormat: messages**, the gateway disables **x-api-key** authentication cluster-wide. Anthropic SDK clients that rely on the **x-api-key** header receive **401 Unauthorized** errors until an **ExternalModel** with **apiFormat: messages** is created again.



IMPORTANT

Token limits apply at two levels:

- **MaaS subscription level:** Token limits configured in MaaS subscriptions apply per-user within MaaS. These limits control individual user consumption.
- **External provider level:** Token limits imposed by the external provider on the API key apply to the aggregate consumption of all users of that external model. Because all users share the same provider API key, the provider-level limit is shared across the entire group of users.

Administrators must ensure that the token limit on the provider API key can handle the combined consumption of all users who access the external model. If the provider-level limit is exceeded, no users can access the model until the provider resets the limit, which is typically hourly, daily, or monthly depending on the provider. Individual MaaS subscription limits do not affect this provider-level restriction.

External models in the dashboard

The **External models** tab is a Technology Preview feature. To view the tab, your cluster administrator must set **spec.dashboardConfig.externalModels** to **true** in the **OdhdashboardConfig** custom resource. For more information, see [Dashboard configuration for Models-as-a-Service](#).

You can view registered external model endpoints by clicking **AI hub** → **Models** and selecting the **External models** tab on the **Model deployments** page. The table shows all **ExternalModel** resources in the selected project with the following columns:

Model

The name and description of the **ExternalModel** resource.

Provider

The external providers associated with the model, displayed as labels.

Status

The reconciliation status of the external model: **Ready**, **Pending**, or **Failed**.

You can use the **Project** selector in the toolbar to switch between projects, and the search field to filter by name, resource name, or description.

To view detailed provider information, expand an external model row. The sub-table shows the **ExternalProvider** resources associated with the model, with the following details:

External provider

The name of the external provider, displayed as a label.

Provider URL

Click **View URL** to open a dialog that shows the URL where requests are routed for the provider, along with the provider name and target model ID. Consumers use the gateway endpoint, not this URL directly.

Path

Click **View path** to open a dialog that shows the request path appended to the provider URL. If path variables are configured, the dialog shows them with resolved values.

Authentication

The authentication method used to connect to the external provider, such as **Signature Version 4**.

Credential secret

The name of the Kubernetes secret that contains the provider credentials.

API format

The API format used by the external provider, such as **messages**.

Target model ID

The upstream model identifier at the provider, such as **claude-sonnet-4-5-20241022**.

Weight

The routing weight assigned to the provider for traffic distribution.

Additional resources

- [Models-as-a-Service subscriptions](#)
- [Authorization policies](#)
- [Create a subscription for Models-as-a-Service](#)
- [Create an API key](#)
- [Configure routing to external model providers](#)
- [Multi-provider API passthrough for external models](#)
- [MaaS multi-tenancy known limitations](#)

1.21.2. Multi-provider API passthrough for external models



IMPORTANT

Multi-provider API passthrough is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

You can route inference requests through the Models-as-a-Service (MaaS) gateway by using native provider API formats such as the Anthropic Messages API or the OpenAI Responses API. When the client API format matches the external model's configured **apiFormat**, the gateway forwards requests without translation, preserving provider-specific features that would otherwise be stripped during format conversion.

1.21.2.1. When to use passthrough

By default, the MaaS gateway translates all incoming requests to the format expected by the external provider. For example, a request sent in OpenAI Chat Completions format to an Anthropic provider is automatically translated to the Anthropic Messages format before forwarding. If your developers use only the standard OpenAI SDK, you do not need passthrough and can use the default **openai-chat** format.

Use passthrough when your developers need provider-specific features that translation cannot preserve:

- **Anthropic SDK or Claude Code users:** Set **apiFormat: messages** on the **ExternalModel** to preserve Anthropic prompt caching, extended thinking, and beta flags such as **interleaved-thinking**.
- **OpenAI Responses API users:** Set **apiFormat: openai-responses** to forward the Responses API format as-is, preserving all response-format-specific capabilities.
- **Vertex AI with Anthropic SDK clients:** Set **apiFormat: messages** to route native Anthropic SDK requests through the Vertex AI Anthropic endpoint. The gateway automatically handles Vertex-specific adjustments such as injecting the **anthropic_version** body field and stripping unsupported headers.

Existing deployments that use OpenAI-format-only clients are unaffected because passthrough activates only when the client format matches the configured **apiFormat**.

1.21.2.2. How to enable passthrough

Set the **apiFormat** field on the **ExternalModel** custom resource to match the API format your developers use. The gateway compares the incoming request format with the configured **apiFormat**. If the two formats match, the gateway forwards the request unchanged.

The supported **apiFormat** values and their passthrough eligibility are listed in the Additional resources section of this topic.

1.21.2.3. Vertex AI behavior differences

When routing to Anthropic Claude models on Google Vertex AI, the gateway automatically handles several differences between the direct Anthropic API and the Vertex AI Anthropic endpoint:

- The model identifier is carried in the URL path rather than the request body.
- The **anthropic_version** field is injected into the request body automatically. The **anthropic-version** header is removed.
- Several Anthropic-API-only request fields are stripped because the Vertex AI endpoint rejects them: **context_management**, **betas**, **mcp_servers**, **service_tier**, **container**, and **stream_options**.
- The **anthropic-beta** header is removed.

Clients that send Anthropic Messages format to a Vertex AI provider do not need to modify their requests.

1.21.2.4. Limitations

Multi-provider API passthrough has limitations that affect streaming, response inspection, token rate limiting, and request body handling. For a complete list, see *Multi-provider API passthrough limitations* in Additional resources.

Additional resources

- [Supported API formats and passthrough behavior for external models](#)
- [Multi-provider API passthrough limitations](#)

1.21.3. Supported API formats and passthrough behavior for external models



IMPORTANT

Multi-provider API passthrough is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

You can configure the API format for external models in Models-as-a-Service (MaaS) to control whether the gateway translates or passes through inference requests. The passthrough behavior for the **messages** and **openai-responses** formats is a Technology Preview feature. Routing external models with the default **openai-chat** format uses translation and is not affected by the Technology Preview scope. The following tables describe the supported API formats, their passthrough eligibility, authentication requirements, and known limitations.

1.21.3.1. Format detection

The gateway detects the client's API format from the request path suffix. The following table maps request path suffixes to the detected format name used internally by the gateway.

Table 1.14. Request path to detected format mapping

Request path suffix	Detected format	API type
/v1/chat/completions	openai-chat	OpenAI Chat Completions
/v1/messages	messages	Anthropic Messages
/v1/responses	openai-responses	OpenAI Responses
Unrecognized path	Not detected	Returns 400 Bad Request

1.21.3.2. apiFormat field values

The **apiFormat** field on the **ExternalProviderRef** in the **ExternalModel** custom resource determines the output format for the provider. The following table lists the supported values and their passthrough eligibility.

Table 1.15. apiFormat values and passthrough eligibility

apiFormat value	Provider output format	Passthrough eligible	Notes
openai-chat	OpenAI Chat Completions	No	Always uses translation. The gateway performs path rewriting and request body normalization that are required for correct provider routing.
messages	Anthropic Messages	Yes	Passthrough activates when the client sends requests to /v1/messages .
openai-responses	OpenAI Responses	Yes	Passthrough activates when the client sends requests to /v1/responses .
vertex-messages	Vertex AI Anthropic	No (minimal translation)	When the client sends requests to /v1/messages , the gateway applies minimal Vertex-specific adjustments: injecting the anthropic_version body field, removing the anthropic-beta header, and stripping Vertex-unsupported fields. The request body structure is largely preserved but is not forwarded unchanged.

Vertex AI providers support two **apiFormat** values:

- **openai-chat**: Routes requests to the Vertex AI OpenAI-compatible endpoint. This is the default.
- **messages**: Routes requests to the Vertex AI Anthropic endpoint, enabling native Anthropic SDK clients to route through Vertex AI.

1.21.3.3. Passthrough decision matrix

The following table shows whether the gateway uses passthrough or translation for each combination of input and output format.

Table 1.16. Input format and output format passthrough behavior

Client input format	ExternalModel apiFormat	Behavior
openai-chat	openai-chat	Translation (path rewriting required)
openai-chat	messages	Translation (OpenAI to Anthropic)
openai-chat	vertex-messages	Translation (OpenAI to Vertex Anthropic)
messages	messages	Passthrough
messages	messages (Vertex)	Translation (Anthropic to Vertex Anthropic, with anthropic_version body injection, anthropic-beta header removal, and Vertex-unsupported field stripping)
messages	vertex-messages	Translation (Anthropic to Vertex Anthropic, with header injection and field stripping)
openai-responses	openai-responses	Passthrough
Other combinations	Any	400 Bad Request (unsupported format combination)

1.21.3.4. Authentication headers per provider

The gateway injects the correct authentication header based on the provider type specified in the **ExternalProvider** custom resource. The following table maps provider types to their default authentication headers.

Table 1.17. Provider authentication headers

Provider type	Default auth header	Value format
openai	Authorization	Bearer <token>
anthropic	x-api-key	<key> (no prefix)
azure	api-key	<key> (no prefix)
vertex	Authorization	Bearer <oauth2-token>
bedrock	Authorization	SigV4 request signing

The **provider** field on the **ExternalProvider** custom resource accepts any string value. The values listed above have built-in support for provider-specific authentication header format and request translation. Providers not listed in the default mapping fall back to the **Authorization** header with **Bearer** prefix.

1.21.3.5. ExternalProvider authentication types

The **auth.type** field in the **ExternalProvider** custom resource determines the authentication mechanism.

Table 1.18. ExternalProvider auth.type values

auth.type value	Description
apikey	Header-based API key authentication. The gateway reads the API key from the referenced Kubernetes secret and injects it in the provider-specific header. This is the most commonly used authentication type. The auth.type field is required and must be set explicitly.
sigv4	AWS Signature Version 4 request signing. Used for AWS Bedrock providers. The gateway signs the entire request using AWS credentials from the referenced secret.
oauth2	OAuth2 token generation. Used for Google Vertex AI providers. The gateway generates a short-lived OAuth2 bearer token from a Google Cloud Platform (GCP) service account JSON key stored in the referenced secret.

1.21.3.6. Vertex AI provider configuration

When configuring an external model for Anthropic Claude models hosted on Google Vertex AI, use the **messages** apiFormat for native Anthropic SDK clients or the **vertex-messages** apiFormat for the legacy Vertex-specific format. Provide the required configuration keys.

Table 1.19. Required Vertex AI configuration

Configuration key	Description
project	The GCP project ID where the Vertex AI endpoint is hosted.
location	The GCP region where the model is deployed, such as us-central1 .
anthropicVersion	The Anthropic API version to use, such as vertex-2023-10-16 .

The **ExternalProvider** for Vertex AI should use **auth.type: oauth2** with a Kubernetes secret containing the GCP service account JSON key for proper GCP authentication.

The gateway automatically strips the following Anthropic-API-only fields from requests sent to Vertex AI because the Vertex AI endpoint rejects them:

- **context_management**
- **betas**
- **mcp_servers**
- **service_tier**
- **container**
- **stream_options**

The gateway also removes the **anthropic-beta** header from requests sent to Vertex AI.

1.21.3.7. Body-based model routing

AI coding tools and SDK clients that connect to a single MaaS gateway URL can switch between models mid-session by specifying the model name in the request body. A pre-auth external processing filter extracts the model name from the request body and sets the **X-Gateway-Model-Name** header. The gateway then uses this header for routing, falling back to reading the **model** field from the request body if the header is not set.

The gateway resolves model names by checking the **X-Gateway-Model-Name** header first, then the **model** field in the request body. The model name is matched against **spec.modelName** if configured on the **ExternalModel**, otherwise against **metadata.name**.

1.21.3.8. Limitations



NOTE

If you optionally integrate NeMo Guardrails, response guards do not inspect responses in non-OpenAI formats. The response guard extracts content from the **choices** field in the response body, which is specific to the OpenAI response format. Responses in Anthropic Messages or OpenAI Responses format do not include a **choices** field, so the guard allows these responses through without content inspection.

If your deployment uses NeMo Guardrails and requires response content inspection, use the **openai-chat** apiFormat with translation instead of passthrough.

The following additional limitations apply to multi-provider API passthrough:

- **OpenAI Chat Completions exclusion** The **openai-chat** format always uses translation regardless of format match, because the gateway performs path rewriting and request body normalization that are required for correct provider routing.
- **Streaming buffered during cross-format translation** When the gateway translates between different API formats, such as a client sending OpenAI format to an Anthropic provider, the gateway buffers the entire provider response before delivering it to the client instead of streaming tokens incrementally. Real-time streaming is available when the client uses the provider's native format with passthrough.
- **Unsupported request paths** Requests with paths that do not map to a recognized format suffix are rejected with **400 Bad Request**.
- **Request body truncation with large payloads** Requests larger than 16 KB might experience body truncation when the Kuadrant wasm shim is active with the **allow_on_headers_stop_iteration** flag set. This issue is resolved in Red Hat OpenShift Service Mesh 3.3.1 and later.
- **Body-based model routing required for single-URL access** AI coding tools that connect to a single base URL rely on body-based model routing, which uses the pre-auth **ext_proc** filter to extract the model name from the request body. If the filter is not configured, the gateway reads the **model** field directly from the request body as a fallback.
- **Model name resolution** When a model name in the request body does not match any configured **ExternalModel**, the request is not handled by the external model routing pipeline. The request proceeds without provider resolution, which might result in a routing error or a timeout downstream rather than an immediate rejection with a clear error message.
- **Token rate limiting not enforced for passthrough formats** MaaS subscription-level token rate limits apply only to OpenAI Chat Completions format requests. Models configured with **apiFormat: messages** or **apiFormat: openai-responses** are not metered at the MaaS subscription level. The rate limiting integration relies on the **usage.total_tokens** response field, which is specific to the OpenAI Chat Completions format. Use provider-level rate limits to manage consumption for these models.

1.21.4. Configure routing to external model providers

In Red Hat OpenShift AI, you can configure Models-as-a-Service (MaaS) to route inference requests to large language models (LLMs) hosted by external cloud providers such as OpenAI, Anthropic, AWS Bedrock, Azure OpenAI, or Google Vertex AI, enabling unified governance for both locally deployed models and external models. Users access external models through MaaS by using their MaaS API keys. The provider API key is used by MaaS to authenticate requests to the external provider.

The external model configuration uses a two-resource architecture: an **ExternalProvider** custom resource defines the provider endpoint and credentials, and an **ExternalModel** custom resource maps a client-facing model name to one or more providers. The **apiFormat** field on the **ExternalModel** determines whether the gateway translates requests or passes them through in the provider's native format. For more information about passthrough mode and supported formats, see [Multi-provider API passthrough for external models](#).



NOTE

Token limits apply at two levels:

- **MaaS subscription level:** Token limits you configure in MaaS subscriptions control per-user consumption within MaaS.
- **External provider level:** Token limits on the provider API key, configured by the external provider, apply to the aggregate consumption of all users accessing the external model. All users share the same provider API key, so the provider-level limit is shared across all users.

Make sure that the token limit on the provider API key can handle the combined consumption of all users who access the external model. If the provider-level limit is exceeded, no users can access the model until the provider resets the limit, which is typically hourly, daily, or monthly depending on the provider.



IMPORTANT

In multitenant deployments, external models are supported for the default tenant only. Non-default tenant instances have the **ExternalModel** controller disabled. For more information, see [MaaS multi-tenancy known limitations](#).

Prerequisites

- You have cluster administrator privileges for the OpenShift cluster where OpenShift AI is installed.
- You have installed Red Hat OpenShift AI.
- You have deployed Models-as-a-Service.
- You have created at least one MaaS subscription to which you will add the external model.
- You have identified the external model provider, API endpoint hostname, and target model ID. For example, provider **openai**, endpoint **api.openai.com**, and target model **gpt-4o**.
- You have identified the **apiFormat** value for the provider. For supported values, see [Supported API formats and passthrough behavior for external models](#).
- You have an API key for the external model provider.
- You have created a model namespace, such as **llm**.

Procedure

1. Create a Kubernetes secret with the external provider API key:

```
$ oc create secret generic <provider-api-key-secret> \
  --from-literal=api-key=<provider-api-key> \
  -n <model-namespace>
$ oc label secret <provider-api-key-secret> \
  inference.llm-d.ai/ipp-managed=true \
  -n <model-namespace>
```

where:

<provider-api-key-secret>

Specifies the name of the secret containing the provider API key.

<provider-api-key>

Specifies the API key for the external provider.

<model-namespace>

Specifies the name of the model namespace you created, such as **llm**.

This secret stores the provider API key that MaaS uses to authenticate requests to the external model.

The resulting secret has the following structure:

```
apiVersion: v1
kind: Secret
metadata:
  name: <provider-api-key-secret>
  namespace: <model-namespace>
  labels:
    inference.llm-d.ai/ipp-managed: "true"
type: Opaque
data:
  api-key: <base64-encoded-api-key>
```

2. Create an **ExternalProvider** custom resource that defines the provider endpoint and authentication:

```
apiVersion: inference.opendatahub.io/v1alpha1
kind: ExternalProvider
metadata:
  name: <provider-name>
  namespace: <model-namespace>
spec:
  provider: <provider-type>
  endpoint: <external-provider-hostname>
  auth:
    type: apikey
    secretRef:
      name: <provider-api-key-secret>
```

where:

<provider-name>

Specifies a name for the **ExternalProvider** resource, such as **openai-provider** or **anthropic-provider**.

<model-namespace>

Specifies the name of the model namespace you created, such as **llm**.

<provider-type>

Specifies the provider type. The gateway has built-in support for the following values:

openai, **anthropic**, **azure**, **vertex**, **bedrock**. Other values are accepted but use the default **Authorization: Bearer** header format.

<external-provider-hostname>

Specifies the fully qualified domain name (FQDN) of the external provider without scheme or path, such as **api.openai.com** or **api.anthropic.com**.

<provider-api-key-secret>

Specifies the name of the secret created in the previous step.

The example uses **auth.type: apikey**. Other providers require different authentication types. For provider-specific authentication types, see [Supported API formats and passthrough behavior for external models](#).



NOTE

When the **ExternalProvider** resource is created, the controller automatically creates the required networking resources: **Service**, **ServiceEntry**, and **DestinationRule**. These resources enable routing from the MaaS gateway to the external provider endpoint. If external model routing fails, verify that these resources were created successfully in your model namespace.

Verify that the **ExternalProvider** status is **Ready**:

```
$ oc get externalprovider <provider-name> -n <model-namespace>
```

The following example shows the expected output for a provider in **Ready** status:

NAME	PROVIDER	ENDPOINT	PHASE	AGE
openai-provider	openai	api.openai.com	Ready	10s

3. Create an **ExternalModel** custom resource that maps a client-facing model name to the external provider:

```
apiVersion: inference.opendatahub.io/v1alpha1
kind: ExternalModel
metadata:
  name: <external-model-name>
  namespace: <model-namespace>
spec:
  externalProviderRefs:
  - ref:
      name: <provider-name>
      targetModel: <target-model-id>
      apiFormat: <api-format>
      path: <provider-path>
      weight: 100
```

where:

<external-model-name>

Specifies the client-facing model name. Clients use this name in the **model** field of inference requests.

<model-namespace>

Specifies the name of the model namespace you created, such as **llm**.

<provider-name>

Specifies the name of the **ExternalProvider** resource you created in the previous step.

<target-model-id>

Specifies the upstream model identifier at the provider, such as **gpt-4o** or **claude-sonnet-4-20250514**.

<api-format>

Specifies the API format for this provider. Use **openai-chat** for OpenAI Chat Completions, **messages** for Anthropic Messages or for Anthropic Claude models hosted on Google Vertex AI, **openai-responses** for OpenAI Responses, or **vertex-messages** for the legacy Vertex AI Anthropic format. For the complete list, see [Supported API formats and passthrough behavior for external models](#).

<provider-path>

Specifies the outgoing request path, such as **/v1/chat/completions** for OpenAI or **/v1/messages** for Anthropic.



NOTE

Set **weight** to control traffic distribution when an **ExternalModel** references multiple providers. A weight of **0** disables the provider so that no traffic is routed to it. When an **ExternalModel** references a single provider, set **weight** to **100**.

You can add optional annotations to the **ExternalModel** metadata to control TLS origination and port configuration for the external endpoint:

maas.opendatahub.io/tls

Specifies whether the Istio sidecar performs TLS origination to the external endpoint. The default value is **true**. When set to **true**, a **DestinationRule** with TLS mode **SIMPLE** is created and the **ServiceEntry** protocol is set to HTTPS. When set to **false**, no **DestinationRule** is created and the **ServiceEntry** protocol is set to HTTP.

maas.opendatahub.io/port

Specifies the port used for the external endpoint. The default value is **443**. Valid range: 1-65535. Values outside this range are rejected during reconciliation.



IMPORTANT

If you set the **maas.opendatahub.io/tls** annotation to **false** while the **ExternalProvider** resource references a **credentialRef**, the provider API key is sent in plain text between the gateway and the endpoint. Only disable TLS on a trusted, isolated network where credential exposure is not a concern.

The following example shows an **ExternalModel** that connects to an internal vLLM instance without TLS on port 8000:

```
apiVersion: inference.opendatahub.io/v1alpha1
kind: ExternalModel
metadata:
  name: internal-vllm
  namespace: llm
  annotations:
```

```

    maas.opendatahub.io/tls: "false"
    maas.opendatahub.io/port: "8000"
spec:
  externalProviderRefs:
  - ref:
      name: internal-vllm-provider
      targetModel: my-model
      apiFormat: openai-chat
      path: /v1/chat/completions
      weight: 100

```

The following examples show **ExternalProvider** and **ExternalModel** configurations for common providers:

OpenAI provider example:

```

apiVersion: inference.opendatahub.io/v1alpha1
kind: ExternalProvider
metadata:
  name: openai-provider
  namespace: llm
spec:
  provider: openai
  endpoint: api.openai.com
  auth:
    type: apikey
    secretRef:
      name: openai-api-key
---
apiVersion: inference.opendatahub.io/v1alpha1
kind: ExternalModel
metadata:
  name: gpt-4o
  namespace: llm
spec:
  externalProviderRefs:
  - ref:
      name: openai-provider
      targetModel: gpt-4o
      apiFormat: openai-chat
      path: /v1/chat/completions
      weight: 100

```

Anthropic provider example:

```

apiVersion: inference.opendatahub.io/v1alpha1
kind: ExternalProvider
metadata:
  name: anthropic-provider
  namespace: llm
spec:
  provider: anthropic
  endpoint: api.anthropic.com
  auth:
    type: apikey
    secretRef:

```

```

name: anthropic-api-key
---
apiVersion: inference.opendatahub.io/v1alpha1
kind: ExternalModel
metadata:
  name: claude-sonnet
  namespace: llm
spec:
  externalProviderRefs:
  - ref:
      name: anthropic-provider
      targetModel: claude-sonnet-4-20250514
      apiFormat: messages
      path: /v1/messages
      weight: 100

```



NOTE

When the **ExternalModel** resource is created, the controller automatically creates an **HTTPRoute** with per-provider routing rules. The **ExternalModel** must reference an **ExternalProvider** that is in **Ready** status.

4. Create a **MaaSModelRef** custom resource in the same namespace as the **ExternalModel** to publish the external model to MaaS:

```

$ cat <<EOF | oc apply -f -
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSModelRef
metadata:
  name: <external-model-name>
  namespace: <model-namespace>
spec:
  modelRef:
    kind: ExternalModel
    name: <external-model-name>
EOF

```

where:

- **<external-model-name>**: Specifies the name of the **ExternalModel** resource you created in the previous step.
- **<model-namespace>**: Specifies the namespace where you created the **ExternalModel** resource.

Verify that the **MaaSModelRef** resource was created:

```
$ oc get maasmodelref <external-model-name> -n <model-namespace>
```

If the resource is missing, the external model does not appear in MaaS subscriptions.

5. Add the external model to a MaaS subscription and authorization policy.
When creating or updating subscriptions, the external model is displayed in the model list alongside locally deployed models. Select the external model and configure token limits the

same way as for internal models. You must also create a matching authorization policy to grant user groups access to the external model.

For more information, see [Managing subscriptions for Models-as-a-Service](#) and [Create an authorization policy](#).

Verification

1. Verify that the **ExternalModel** resource has a **Ready** status:
 - a. In the OpenShift AI dashboard, click **AI hub** → **Models**.
 - b. On the **Model deployments** page, click the **External models** tab.
 - c. Select the project where you created the external model from the **Project** selector.
 - d. Verify that the external model is displayed in the list with a **Ready** status.
If the status is **Pending**, wait for the MaaS controller to finish reconciling the resource. If the status is **Failed**, verify that the networking resources were created successfully in your model namespace:


```
$ oc get service,httproute,serviceentry,destinationrule -n <model-namespace>
```
 - e. Click the expand arrow on the model row and verify that the provider details, such as the provider URL, authentication method, and target model ID, are correct.
2. Verify that the external model was added to the subscription:
 - a. In the OpenShift AI dashboard, navigate to **Settings** → **MaaS governance**.
 - b. Click the subscription name.
 - c. In the **Models** section, verify that the external model is displayed in the list.
3. Make a test inference request to verify end-to-end routing to the external provider:
 - a. Log in to the OpenShift AI dashboard as a user who belongs to a group included in the subscription.
 - b. Click **Gen AI studio** → **AI asset endpoints**.
 - c. Locate the external model and click **View** in the **Endpoints** column.
 - d. Copy the external API endpoint URL from the **Endpoints** dialog.
 - e. Generate a temporary API key or create a persistent API key. For more information, see [Generate a temporary API key](#) or [Create an API key](#).
 - f. Run the appropriate command for your model's API format:
The authentication header and endpoint path depend on the **apiFormat** configured for the model. For the Anthropic Messages format, use the **x-api-key** header. For OpenAI formats, use the **Authorization: Bearer** header. For the complete authentication header mapping, see [Supported API formats and passthrough behavior for external models](#).

For models configured with `apiFormat: openai-chat`

```
$ curl -X POST <external-api-endpoint>/v1/chat/completions \
-H "Authorization: Bearer <maas-api-key>" \
-H "Content-Type: application/json" \
-d { "model": "<external-model-name>", "messages": [{"role": "user", "content":
"Hello"}] }
```

For models configured with **apiFormat: messages** (Anthropic):

+

```
$ curl -X POST <external-api-endpoint>/v1/messages \
-H "x-api-key: <maas-api-key>" \
-H "Content-Type: application/json" \
-H "anthropic-version: 2023-06-01" \
-d { "model": "<external-model-name>", "max_tokens": 100, "messages": [{"role": "user",
"content": "Hello"}] }
```

For models configured with **apiFormat: openai-responses**:

+

```
$ curl -X POST <external-api-endpoint>/v1/responses \
-H "Authorization: Bearer <maas-api-key>" \
-H "Content-Type: application/json" \
-d { "model": "<external-model-name>", "input": "Hello" }
```

where:

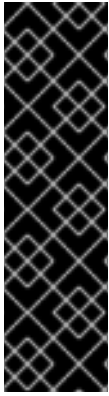
- **<external-api-endpoint>**: Specifies the external API endpoint URL you copied from the **Endpoints** dialog.
- **<maas-api-key>**: Specifies the API key you generated or created.
- **<external-model-name>**: Specifies the name of the external model resource you created.

A successful response confirms that MaaS routed the request to the external provider and returns the model completion in the provider's native format.

Next steps

- Provide your developers with the MaaS gateway URL and inform them of the authentication header for the model's API format:
 - For models with **apiFormat: messages**, developers use the **x-api-key** header with their MaaS API key.
 - For models with **apiFormat: openai-chat** or **apiFormat: openai-responses**, developers use the **Authorization: Bearer** header with their MaaS API key.
- [Use single-URL passthrough with AI tools](#)

1.21.5. Use single-URL passthrough with AI coding tools



IMPORTANT

Multi-provider API passthrough is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

You can configure AI coding tools such as Claude Code or OpenAI Codex to use a single MaaS gateway URL for inference requests. The gateway extracts the model name from the request body and routes to the correct external model, allowing tools to switch models mid-session without per-model URLs.

Prerequisites

- An administrator has configured at least one **ExternalModel** with a passthrough-eligible **apiFormat** value such as **messages** or **openai-responses**.
- Body-based model routing is enabled on the MaaS gateway. This is enabled by default as part of the MaaS deployment. For more information about body-based routing, see [Supported API formats and passthrough behavior for external models](#).
- You have a MaaS API key. For more information, see [Create an API key](#).
- You have the MaaS gateway URL.
- You have the external model name as configured in the **ExternalModel** custom resource.

Procedure

1. Obtain your MaaS API key from the OpenShift AI dashboard:
 - a. Navigate to **Gen AI studio** → **AI asset endpoints**.
 - b. Click **Create API key** and save the generated key.
2. Identify the MaaS gateway base URL. The gateway URL uses the format `https://<maas-gateway-url>`.
3. Configure your AI coding tool to use the MaaS gateway as its API endpoint. The configuration depends on the tool and the provider API format it uses.

For tools that use the Anthropic Messages API, such as Claude Code

Set the base URL to the MaaS gateway URL and authenticate with the **x-api-key** header by providing your MaaS API key.

The following example shows the environment variables for Claude Code:

```
$ export ANTHROPIC_BASE_URL=https://<maas-gateway-url>
$ export ANTHROPIC_API_KEY=<maas-api-key>
```

For tools that use the OpenAI Responses API

Set the base URL to the MaaS gateway URL and authenticate with the **Authorization: Bearer** header by providing your MaaS API key.

The following example shows the environment variables for an OpenAI SDK client:

```
$ export OPENAI_BASE_URL=https://<maas-gateway-url>
$ export OPENAI_API_KEY=<maas-api-key>
```

The tool specifies the model name in the request body. The gateway extracts the model name and routes the request to the matching **ExternalModel** resource.

4. To switch between models, change the model name in your tool's configuration or next request. The gateway resolves the model name and routes to the corresponding external provider without requiring a URL change.

Verification

1. Send a test request using **curl** to verify that single-URL body-based routing works correctly. The following example sends an Anthropic Messages format request to the single gateway URL with the model name in the request body:

```
$ curl -X POST https://<maas-gateway-url>/v1/messages \
-H "x-api-key: <maas-api-key>" \
-H "Content-Type: application/json" \
-H "anthropic-version: 2023-06-01" \
-d { "model": "<external-model-name>", "max_tokens": 100, "messages": [{"role": "user",
"content": "Hello"}] }
```

The command uses the following placeholders:

<maas-gateway-url>

Specifies your MaaS gateway URL.

<external-model-name>

Specifies the name of the external model resource as configured by your administrator.

<maas-api-key>

Specifies your MaaS API key.

2. Verify that the response is in the Anthropic Messages format. A successful response contains a **content** array with the model's reply:

```
{
  "id": "msg_abc123",
  "type": "message",
  "role": "assistant",
  "content": [
    {
      "type": "text",
      "text": "Hello! How can I help you today?"
    }
  ],
  "model": "claude-sonnet-4-20250514",
  "stop_reason": "end_turn",
  "usage": {
```

```

    "input_tokens": 10,
    "output_tokens": 12
  }
}

```



NOTE

The gateway resolves model names by checking the **X-Gateway-Model-Name** header first, then the **model** field in the request body. The model name is matched against **spec.modelName** if configured on the **ExternalModel**, otherwise against **metadata.name**. If the model name in the request body does not match any configured external model, the request is not handled by the external model routing pipeline and might result in a routing error downstream.

Additional resources

- [Technology Preview Features Support Scope](#)
- [Managing subscriptions for Models-as-a-Service](#)
- [Accessing models with Models-as-a-Service](#)

1.22. MODELS-AS-A-SERVICE ADMINISTRATION TROUBLESHOOTING

As a OpenShift AI administrator, you can diagnose and resolve common administrative issues with Models-as-a-Service (MaaS) deployment, configuration, and management.

1.22.1. Component enablement issues

If the **maas-api** pod fails to start or shows errors after enabling the MaaS component:

- Check the pod logs for error messages:

```
$ oc logs -n <infrastructure_namespace> -l app.kubernetes.io/name=maas-api
```

where:

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

- Verify that all prerequisites are met, especially:
 - **Kuadrant** is running in the **kuadrant-system** namespace
 - The **maas-default-gateway Gateway** exists in the **openshift-ingress** namespace
 - KServe component is set to **Managed** in the DataScienceCluster

If **Kuadrant** is not in a ready state:

- a. Check the **Kuadrant** Operator status:

```
$ oc get kuadrant -n kuadrant-system
```

- b. If the **Kuadrant** resource shows a non-ready status, restart the **Kuadrant** Operator:
 - i. In the OpenShift web console, go to the **Installed Operators** page. The navigation path depends on your OpenShift version:
 - On OpenShift 4.20 and later, click **Ecosystem** → **Installed Operators**.
 - On OpenShift 4.19, click **Operators** → **Installed Operators**.
 - ii. Select the **kuadrant-system** namespace.
 - iii. Click **Red Hat Connectivity Link**
 - iv. From the **Actions** menu, click **Restart**.
- c. Wait for the Operator to restart and verify that Kuadrant becomes ready:

```
$ oc wait Kuadrant -n kuadrant-system kuadrant --for=condition=Ready --
timeout=5m
```

- Check for events related to the MaaS deployment:

```
$ oc get events -n <infrastructure_namespace> --sort-by='.lastTimestamp' | grep maas
```

- Verify that the required RBAC resources were created:

```
$ oc get clusterrole | grep maas
$ oc get clusterrolebinding | grep maas
```

1.22.2. Dashboard visibility issues

If MaaS features do not appear in the dashboard:

- Verify that the MaaS API component is running:

```
$ oc get pods -n <infrastructure_namespace> -l app.kubernetes.io/name=maas-api
```

where:

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

- Check that the ODHDashboardConfig was updated correctly:

```
$ oc get odhdashboardconfig odh-dashboard-config -n redhat-ods-applications -o yaml | grep
-A 2 "dashboardConfig:"
```

Verify the following:

- **modelAsService: true** for admin features, including the MaaS governance page
- **genAiStudio: true** for user-facing features, including the Models tab in AI asset endpoints
- Clear your browser cache and hard refresh the dashboard (Ctrl+Shift+R or Cmd+Shift+R).

- Check the dashboard pod logs for errors:

```
$ oc logs -n redhat-ods-applications $(oc get pods -n redhat-ods-applications -o name | grep dashboard | head -1) --tail=50
```

- Verify that you have the required permissions to view MaaS features. Admin features require administrator access.

1.22.3. Model visibility issues

If a model is missing from the available models for MaaS:

- Verify that you selected **Publish as MaaS** in the Advanced settings during deployment.
- Check that the **MaaSModelRef** was created:

```
$ oc get maasmodelref -n <your-project-namespace>
```

where:

<your-project-namespace>

Specifies the project namespace where the model is deployed.

- Check that the MaaS API is running:

```
$ oc get pods -n <infrastructure_namespace> -l app.kubernetes.io/name=maas-api
```

where:

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

- Verify that the model deployment is in a Ready state:

```
$ oc get llminferenceservice -n <your-project-namespace>
```

1.22.4. User access errors: 403 Forbidden

If users receive 403 Forbidden errors when accessing models through MaaS:

- Verify that the user has both a subscription and an authorization policy:
 - A subscription grants quota for specific models with token limits.
 - An authorization policy is required to authorize groups to access model endpoints through the API gateway.
- Check that a subscription exists for the user's groups:

```
$ oc get maassubscriptions -n <infrastructure_namespace>
```

where:

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

- Verify that the model is included in the subscription:

```
$ oc get maassubscription <subscription-name> -n <infrastructure_namespace> -o yaml
```

where:

<subscription-name>

Specifies the name of the subscription.

- Check that an authorization policy exists:

```
$ oc get maasauthpolicies -n <infrastructure_namespace>
```

- Verify that the authorization policy includes the user's groups:

```
$ oc get maasauthpolicy <policy-name> -n <infrastructure_namespace> -o yaml
```

where:

<policy-name>

Specifies the name of the authorization policy.

1.22.5. Subscription access control issues

If users receive unexpected access denials:

- Verify that the subscription status is **Active**:

```
$ oc get maassubscription <subscription-name> -n <infrastructure_namespace> -o  
jsonpath='{.status.phase}'
```

where:

<subscription-name>

Specifies the name of the subscription.

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

- Check the subscription conditions for errors:

```
$ oc get maassubscription <subscription-name> -n <infrastructure_namespace> -o  
jsonpath='{.status.conditions}'
```

- Ensure the model deployment is ready:

```
$ oc get llminferenceservice -n <model-namespace>
```

where:

<model-namespace>

Specifies the project namespace where the model is deployed.

- Check the **Gateway** logs for authorization errors:

```
$ oc logs -n kuadrant-system -l app=authorino --tail=50
```

- Verify that the **MaaSModelRef** exists for the model:

```
$ oc get maasmodelref -n <model-namespace>
```

1.22.6. Subscription management issues

If users receive access errors when attempting to use models after creating a subscription:

- Verify that the user's OpenShift groups are listed in the subscription's groups.
- Verify that the model is included in the subscription's model list.
- Check that at least one token limit is configured for each model in the subscription.
- If multiple subscriptions apply to a user, verify that the correct subscription is being used based on priority level (higher numbers have higher priority).
- Check the MaaS API logs for subscription resolution errors:

```
$ oc logs -n <infrastructure_namespace> -l app.kubernetes.io/name=maas-api --tail=50 |  
grep subscription
```

where:

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

- Verify that a matching authorization policy was created if you selected that option during subscription creation.

1.22.7. Subscription phase shows Failed

If a subscription shows a **Failed** phase status:

- Check the subscription status conditions for the failure reason:

```
$ oc describe maassubscription <subscription-name> -n <infrastructure_namespace>
```

where:

<subscription-name>

Specifies the name of the subscription.

<infrastructure_namespace>

Specifies your MaaS infrastructure namespace. The default is **redhat-ai-gateway-infra**.

- Verify that all referenced models exist and have valid **MaaSModelRef** objects.

- Ensure that the groups specified in the subscription are valid OpenShift groups.
- Check that token limits are properly configured for all models.
- Review the MaaS API logs for detailed error messages:

```
$ oc logs -n <infrastructure_namespace> -l app.kubernetes.io/name=maas-api --tail=100
```

CHAPTER 2. USE MODELS THROUGH MODELS-AS-A-SERVICE

Use Models-as-a-Service (MaaS) to access large language models with subscription-based governance and self-service API key management. You can discover available models, create API keys, and integrate models into your applications using OpenAI-compatible APIs. For external models configured with multi-provider API passthrough, you can also send requests in native provider formats such as the Anthropic Messages API or the OpenAI Responses API.

2.1. FIND MODELS-AS-A-SERVICE MODELS IN THE DASHBOARD

In Red Hat OpenShift AI, models published to Models-as-a-Service (MaaS) are available through the **AI asset endpoints** page in the OpenShift AI dashboard.

Procedure

1. To view available models, log in to the OpenShift AI dashboard and click **Gen AI studio → AI asset endpoints**.

The **Models** tab displays a table with all available model deployments. Each row shows:

Model

The model deployment name and model ID.

Use case

The model type, such as LLM for large language models.

Status

The current operational state of the model: **Ready**, **Not ready**, or **Unknown**.

Endpoints

A **View** button that opens the endpoint details.

Playground

An **Add to playground** link for testing the model interactively.

2. To identify whether a model is published to Models-as-a-Service (MaaS), click the **View** button under Endpoints. Models published to MaaS display a **Model as a Service** badge at the top of the Endpoints dialog. The dialog shows:
 - The API endpoint URL for accessing the model through the MaaS gateway from outside the cluster
 - A subscription selector for choosing which subscription to use
 - A **Generate API key** button for creating 1-hour temporary API keys
 - A link to the **API keys** page for managing API keys
3. To manage your API keys for programmatic access to MaaS models, click **Gen AI studio → API keys**.

The **API keys** page has two tabs:

- **API keys:** Create, view, and revoke your MaaS API keys.
- **Subscriptions:** View all your subscription assignments, browse models by subscription or by model, and check token rate limits.

Additional resources

- [Access models through models-as-a-service](#)
- [Models-as-a-Service overview](#)
- [Create an API key](#)
- [Test models in the playground](#)
- [View subscription limits](#)
- [View your Models-as-a-Service subscriptions](#)

2.2. ACCESS MODELS THROUGH MODELS-AS-A-SERVICE

In Red Hat OpenShift AI, you can use Models-as-a-Service (MaaS) to access large language models with subscription-based governance and self-service API key management.

2.2.1. Your token limits and access levels

Before you begin working with models, it's helpful to understand how Models-as-a-Service (MaaS) uses subscriptions and authorization policies to control your quota and access.



NOTE

In OpenShift AI 3.4, MaaS uses subscriptions instead of tiers and API keys instead of service account tokens for authentication.

- **Subscription assignment:** You are automatically assigned to subscriptions based on your group membership in OpenShift. If you belong to multiple groups with different subscriptions, you can access all those subscriptions. When creating an API key without specifying a subscription, the system selects the subscription with the highest priority level.
- **Token limits:** Your subscription determines how many tokens you can consume per time period for each model.
- **Model access:** Authorization policies determine which models you can access through the API gateway. Your administrator creates authorization policies that grant your groups access to specific model endpoints. Both a subscription with quota and an authorization policy are required to access models through MaaS.
- **Authentication:** You must use an API key to access models. You can create and manage your own API keys through the dashboard.
- **Subscription visibility:** You can view all your subscription assignments, the models each subscription includes, and the applicable token rate limits from the **Subscriptions** tab on the **API keys** page. For more information, see [View your Models-as-a-Service subscriptions](#).

2.2.2. View MaaS subscription limits

In Red Hat OpenShift AI, you can see which Models-as-a-Service (MaaS) subscriptions you can use for a specific model from the **AI asset endpoints** page.

Prerequisites

- You have access to the OpenShift AI dashboard.
- At least one model has been published to Models-as-a-Service in your environment.
- You belong to a group that is assigned to at least one subscription.

Procedure

1. In the OpenShift AI dashboard, click **Gen AI studio** → **AI asset endpoints**.
2. On the **Models** tab, locate a model published to Models-as-a-Service (MaaS) and click **View** in the **Endpoints** column.
3. In the **Endpoints** dialog, view your subscription assignment from the **Subscription** dropdown. If you belong to multiple subscriptions that include this model, you can select which subscription to use for the model. The selected subscription's token limits apply to your inference requests.

TIP

If you frequently exceed token limits or need access to additional models, contact your administrator to request a subscription update.

TIP

To view all your subscriptions and models in a single view, use the **Subscriptions** tab on the **API keys** page. For more information, see [View your Models-as-a-Service subscriptions](#).

Verification

- The **Subscription** dropdown lists at least one subscription for which you have access to this model.

2.2.3. View your Models-as-a-Service subscriptions

You can view all your Models-as-a-Service (MaaS) subscription assignments, browse associated models, and check token rate limits from the **Subscriptions** tab on the **API keys** page. This centralized view helps you choose the right subscription when creating API keys and avoid unexpected rate-limiting errors.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- You are a member of at least one OpenShift group that is assigned to a MaaS subscription.

Procedure

1. In the OpenShift AI dashboard, click **Gen AI studio** → **API keys**.
2. Select the **Subscriptions** tab.
The **Subscriptions** tab shows models available to you through your subscriptions, with token limits.

3. Choose how to browse your subscriptions by using the **Subscription view** and **Model view** toggle buttons in the toolbar:
 - **Subscription view:** The default view. Groups the list by subscription. Each row shows the subscription name and an active API key count badge. Expand a subscription row to see the models included in that subscription. The expanded table shows the model name with token rate limits formatted as the number of tokens per time window.
 - **Model view:** Groups the list by model. Each row shows the model name. Expand a model row to see the subscriptions that include that model. The expanded table shows the subscription name, API key count, and token rate limits.
4. Optional: Click the question mark icon next to a model name to open a popover with the model ID and description. Click the copy icon in the popover to copy the model ID for use in API calls.
5. Optional: Filter the list by entering a name in the **Search** field. The search matches against subscription names and model names in both views.
6. Optional: Sort the list alphabetically by clicking the **Subscription** or **Model** column header, depending on the active view. Click the column header again to reverse the sort order.



NOTE

If no subscriptions are assigned to you, the tab displays a "No subscriptions" message. If your current search or filter criteria do not match any results, the tab displays a "No results found" message. Clear the filters to see all your subscriptions.

Verification

- The **Subscriptions** tab displays your subscriptions with the correct models and token rate limits.
- Expanding a subscription row shows all models included in that subscription with their token limits.

Next steps

- [Create an API key](#)

2.2.4. Generate a temporary API key

In Red Hat OpenShift AI, you can generate a temporary 1-hour API key directly from the **Endpoints** dialog for quick testing and prototyping of Models-as-a-Service (MaaS) endpoints.

Prerequisites

- You have access to the OpenShift AI dashboard.
- Your administrator has enabled Models-as-a-Service and assigned you to a subscription.

Procedure

1. In the OpenShift AI dashboard, click **Gen AI studio** → **AI asset endpoints**.
2. On the **Models** tab, locate the model you want to access and click **View** in the **Endpoints** column.

3. In the **Endpoints** dialog, verify that the model displays a **Model as a Service** badge at the top.
4. Copy the external API endpoint URL and store it securely. This URL is required for API calls to the model.
5. Under the **Authentication** section, select your subscription from the **Subscription** dropdown.
6. Click **Generate API key**.
7. Copy the generated API key immediately and store it securely.



IMPORTANT

The temporary API key is displayed only once and expires after 1 hour. Temporary API keys generated from the **Endpoints** dialog are scoped to the selected subscription. To create API keys with custom expiration dates, navigate to **Gen AI studio** → **API keys** in the OpenShift AI dashboard.

8. Click **Close**.

Verification

- The generated API key displays with the **sk-oai-** prefix.
- You have securely stored both the API key and the external API endpoint URL for use in API calls.

Next steps

- [Make API calls to models](#)

2.2.5. Make API calls to models

In Red Hat OpenShift AI, you can use your Models-as-a-Service (MaaS) API key to list models available to you and to make inference requests through the OpenAI-compatible API. MaaS supports two inference routing modes: body-based routing using the standard **/v1/chat/completions** endpoint, and legacy path-based routing using per-model URL paths. For external models configured with multi-provider API passthrough, you can also send requests in native provider formats such as the Anthropic Messages API or the OpenAI Responses API. For more information, see [Multi-provider API passthrough for external models](#) and [Use single-URL passthrough with AI tools](#).

Prerequisites

- You have generated a temporary API key or created an API key. For more information, see [Create an API key](#).
- You have copied the external API endpoint URL.
- At least one model is published to Models-as-a-Service and accessible through your subscription.

Procedure

1. Set your API key as an environment variable:

■

```
$ export MAAS_API_KEY="<your_api_key>"
```

The command uses the following placeholder:

<your_api_key>

Specifies the API key you generated or created, with the **sk-oai-** prefix.

2. Set the external API endpoint URL as an environment variable:

```
$ export MAAS_URL="<external_api_endpoint>"
```

The command uses the following placeholder:

<external_api_endpoint>

Specifies the external API endpoint URL you copied from the **Endpoints** dialog.

3. List available models using the **/v1/models** endpoint:

```
$ curl -X GET "${MAAS_URL}/v1/models" \
-H "Authorization: Bearer ${MAAS_API_KEY}"
```

Example response in OpenAI-compatible format:

```
{
  "object": "list",
  "data": [
    {
      "id": "facebook-opt-125m",
      "object": "model",
      "created": 1234567890,
      "owned_by": "llm",
      "ready": true
    },
    {
      "id": "llama-2-7b-chat",
      "object": "model",
      "created": 1234567890,
      "owned_by": "llm",
      "ready": true
    }
  ]
}
```

4. Call a model using the chat completions endpoint. You can use either body-based routing or legacy path-based routing:

Body-based routing (recommended)

Use the standard OpenAI-compatible **/v1/chat/completions** endpoint. The **model** field in the request body determines which model receives the request. This approach enables drop-in compatibility with any OpenAI-compatible SDK or client.

```
$ curl -X POST "${MAAS_URL}/v1/chat/completions" \
-H "Authorization: Bearer ${MAAS_API_KEY}" \
-H "Content-Type: application/json" \
```

```
-d { "model": "<model_name>", "messages": [ { "role": "user", "content": "Explain quantum computing in simple terms." } ], "max_tokens": 150, "temperature": 0.7 }
```

where:

<model_name>

Specifies the model name as returned by the **/v1/models** endpoint, such as **facebook-opt-125m** or **llama-2-7b-chat**.

Legacy path-based routing

Use a per-model URL path that includes the model name. This approach is supported for backwards compatibility with existing integrations.

```
$ curl -X POST "${MAAS_URL}/llm/<model_name>/v1/chat/completions" \
-H "Authorization: Bearer ${MAAS_API_KEY}" \
-H "Content-Type: application/json" \
-d { "model": "<model_name>", "messages": [ { "role": "user", "content": "Explain quantum computing in simple terms." } ], "max_tokens": 150, "temperature": 0.7 }
```

where:

<model_name>

Specifies the model name as returned by the **/v1/models** endpoint.

Example response:

```
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "created": 1234567890,
  "model": "llama-2-7b-chat",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Quantum computing is a type of computing that uses quantum mechanics..."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 12,
    "completion_tokens": 45,
    "total_tokens": 57
  }
}
```

- Optional: Use the Python **openai** SDK to call a model. Body-based routing enables standard SDK usage with no custom URL paths:

```
from openai import OpenAI

client = OpenAI(
```

```

    base_url="<maas_gateway_url>/v1",
    api_key="<your_api_key>",
)

response = client.chat.completions.create(
    model="<model_name>",
    messages=[
        {"role": "user", "content": "Explain quantum computing in simple terms."}
    ],
    max_tokens=150,
    temperature=0.7,
)
print(response.choices[0].message.content)

```

where:

<maas_gateway_url>

Specifies the MaaS gateway base URL, such as `\https://maas.apps.cluster.example.com`.

<your_api_key>

Specifies your MaaS API key with the **sk-oai-** prefix.

<model_name>

Specifies the model name as returned by the `/v1/models` endpoint.

Verification

- The `/v1/models` response includes the models you expect to access.
- Chat completion requests return responses in the OpenAI-compatible JSON format.

Troubleshooting

- If you receive **401 Unauthorized**, verify that your API key is valid and has not been revoked.
- If you receive **429 Too Many Requests**, your subscription's token limit has been reached. Wait for the token limit window to reset.

2.2.6. Test models in a Jupyter notebook

In Red Hat OpenShift AI, you can test Models-as-a-Service endpoints in a Jupyter notebook using Python code. This approach is useful for iterative testing, prototyping model integrations, and validating rate limiting behavior in an interactive environment before integrating models into applications.

Prerequisites

- You have generated a temporary API key or created a permanent API key for the model that you want to access.
- You have copied the external API endpoint URL from the **Endpoints** dialog. The procedure describes how to extract the base gateway URL.
- You have access to a Jupyter notebook environment in OpenShift AI.
- You have Python 3.9 or later available in your notebook environment.

Procedure

1. Create a new Jupyter notebook in your OpenShift AI workbench.
2. In the first code cell, configure the Models-as-a-Service (MaaS) gateway URL and API key:

```
DEMO_MAAS_BASE = "https://maas.example.com"
DEMO_API_KEY = "your-api-key-here"
```

The code uses the following variables:

DEMO_MAAS_BASE

Specifies the MaaS gateway base URL. Extract this from the external API endpoint URL you copied from the **Endpoints** dialog. For example, if the endpoint is `https://maas.apps.cluster.example.com/llm/facebook-opt-125m`, use `https://maas.apps.cluster.example.com`. The specific model URLs are retrieved from the API response in the next step.

DEMO_API_KEY

Specifies your API key. Use the temporary or permanent key you generated earlier.

3. In a new code cell, add the setup code to configure the HTTP client:

```
import json
import os
import ssl
import urllib.error
import urllib.request
from typing import Any, Dict, Optional

_mb = globals().get("DEMO_MAAS_BASE", "")
if isinstance(_mb, str) and _mb.strip():
    MAAS_BASE = _mb.strip().rstrip("/")
else:
    MAAS_BASE = os.environ.get("MAAS_BASE",
    "https://maas.YOUR_DOMAIN_HERE").strip().rstrip("/")

_ak = globals().get("DEMO_API_KEY", "")
if isinstance(_ak, str) and _ak.strip():
    API_KEY = _ak.strip()
else:
    API_KEY = (os.environ.get("MAAS_API_KEY") or os.environ.get("API_KEY") or "").strip()

VERIFY_TLS = os.environ.get("VERIFY_TLS", "").lower() in ("1", "true", "yes")
MODELS_URL = f"{MAAS_BASE}/maas-api/v1/models"

if not API_KEY:
    raise SystemExit(
        "Set DEMO_API_KEY in the quick-swap cell or MAAS_API_KEY / API_KEY in the
        environment."
    )

def http_json(
    method: str,
    url: str,
```

```

*,
token: Optional[str] = None,
data: Optional[Dict[str, Any]] = None,
):
    """Minimal JSON HTTP helper (stdlib only)."""
    headers = {"Content-Type": "application/json", "Accept": "application/json"}
    if token:
        headers["Authorization"] = f"Bearer {token}"
    body = None
    if data is not None:
        body = json.dumps(data).encode("utf-8")
    ctx = ssl.create_default_context()
    if not VERIFY_TLS:
        ctx.check_hostname = False
        ctx.verify_mode = ssl.CERT_NONE
    req = urllib.request.Request(url, data=body, headers=headers, method=method)
    try:
        with urllib.request.urlopen(req, context=ctx, timeout=120) as resp:
            raw = resp.read().decode("utf-8")
            return resp.status, json.loads(raw) if raw else {}
    except urllib.error.HTTPError as e:
        err_body = e.read().decode("utf-8", errors="replace")
        try:
            parsed = json.loads(err_body) if err_body else {}
        except json.JSONDecodeError:
            parsed = {"_raw": err_body}
        raise RuntimeError(f"HTTP {e.code}: {parsed}") from None

print("MAAS_BASE :", MAAS_BASE)
print("MODELS_URL:", MODELS_URL)
print("VERIFY_TLS:", VERIFY_TLS)
print("API key set:", bool(API_KEY))

```

This cell defines the **http_json** helper function and prints the configuration summary.

4. In a new code cell, list the available models:

```

_, models_body = http_json("GET", MODELS_URL, token=API_KEY)

data = models_body.get("data") or []
if not data:
    raise SystemExit("No models in response; deploy a model and check subscription binding.")

first = data[0]
MODEL_NAME = first.get("id") or first.get("name")
MODEL_URL = (first.get("url") or "").rstrip("/")
if not MODEL_NAME or not MODEL_URL:
    raise RuntimeError(f"Could not parse model id/url from: {first}")

print(f"MODEL_NAME: {MODEL_NAME}")
print(f"MODEL_URL: {MODEL_URL}")
print("\nAll models:")
print(json.dumps(models_body, indent=2))

```

This cell retrieves the list of models you can access and stores the first model's name and URL for testing.

5. In a new code cell, make a single inference request:

```
COMPLETIONS_URL = f"{MODEL_URL}/v1/completions"
inference_payload = {
    "model": MODEL_NAME,
    "prompt": "Hello from the notebook demo.",
    "max_tokens": 50,
}

status, completion = http_json("POST", COMPLETIONS_URL, token=API_KEY,
data=inference_payload)

usage = completion.get("usage") or {}
choices = completion.get("choices") or []
choice0 = choices[0] if choices and isinstance(choices[0], dict) else {}
completion_text = choice0.get("text") or ""

print(f"HTTP status: {status}")
print(f"\nTokens used:")
print(f" prompt_tokens: {usage.get('prompt_tokens', '—')}")
print(f" completion_tokens: {usage.get('completion_tokens', '—')}")
print(f" total_tokens: {usage.get('total_tokens', '—')}")
print(f"\nCompletion text:")
print(completion_text if completion_text else "(empty)")
```

This cell sends a completion request and displays the response with token usage information.

6. Optional: In a new code cell, test the rate limiting behavior:

```
import time

# Configuration
SLEEP_BETWEEN_REQUESTS_SEC = 1.0
CONSECUTIVE_429_TO_STOP = 3
MAX_REQUESTS = 64

inference_payload = {
    "model": MODEL_NAME,
    "prompt": "Rate limit probe.",
    "max_tokens": 50,
}

ctx = ssl.create_default_context()
if not VERIFY_TLS:
    ctx.check_hostname = False
    ctx.verify_mode = ssl.CERT_NONE

consecutive_429 = 0
last_code = None
for i in range(1, MAX_REQUESTS + 1):
    body = json.dumps(inference_payload).encode("utf-8")
    req = urllib.request.Request(
        COMPLETIONS_URL,
```

```

        data=body,
        headers={
            "Authorization": f"Bearer {API_KEY}",
            "Content-Type": "application/json",
        },
        method="POST",
    )
    try:
        with urllib.request.urlopen(req, context=ctx, timeout=120) as resp:
            last_code = resp.status
    except urllib.error.HTTPError as e:
        last_code = e.code

    print(f"{i:3d} HTTP {last_code}", end="")
    if last_code == 429:
        consecutive_429 += 1
        print(f" (429 streak {consecutive_429}/{CONSECUTIVE_429_TO_STOP})")
        if consecutive_429 >= CONSECUTIVE_429_TO_STOP:
            print(f"Stopping: {CONSECUTIVE_429_TO_STOP} consecutive 429 responses.")
            break
    else:
        consecutive_429 = 0
        print()

    if SLEEP_BETWEEN_REQUESTS_SEC > 0 and i < MAX_REQUESTS:
        time.sleep(SLEEP_BETWEEN_REQUESTS_SEC)
    else:
        if consecutive_429 < CONSECUTIVE_429_TO_STOP:
            print(f"Stopped: reached MAX_REQUESTS={MAX_REQUESTS} without
{CONSECUTIVE_429_TO_STOP} consecutive 429s.")

```

This cell sends repeated requests to test rate limiting. The loop stops after receiving 3 consecutive HTTP 429 responses or reaching the maximum request count.

Verification

- The setup cell prints your MaaS configuration without exposing the API key value.
- The model list cell displays at least one available model with its name and URL.
- The inference cell returns an HTTP 200 status with completion text and token usage statistics.
- The rate limit test demonstrates throttling behavior by receiving HTTP 429 responses when limits are exceeded.

Additional resources

- [Models-as-a-Service token limit responses](#)

2.2.7. Token limit responses

In Red Hat OpenShift AI, when you exceed token limits for your Models-as-a-Service (MaaS) subscription, the API returns specific error responses to help you manage your usage.

Token limit exceeded

If you exceed the maximum number of tokens allowed per time period for a model in your subscription:

Example error response

```
{
  "error": {
    "message": "Token limit exceeded. You have consumed the maximum number of tokens allowed for this model in your subscription.",
    "type": "token_limit_error",
    "code": 429
  }
}
```

Response headers for retry logic

```
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1234567890
Retry-After: 42
```

Handling token limits in Python

```
import time
import requests

def make_request_with_retry(url, headers, data, max_retries=3):
    for attempt in range(max_retries):
        response = requests.post(url, headers=headers, json=data)

        if response.status_code == 429:
            retry_after = int(response.headers.get('Retry-After', 60))
            print(f"Token limit exceeded. Retrying after {retry_after} seconds...")
            time.sleep(retry_after)
            continue

        return response

    raise Exception("Max retries exceeded")
```

Mitigation strategies

- Reduce **max_tokens** in your requests to consume fewer tokens per request
- Implement exponential backoff retry logic to handle temporary limit violations
- Batch requests with longer delays between them to spread token consumption over time
- Contact your administrator to request a subscription update with higher token limits if you consistently hit limits

2.2.8. Test models in the playground

In Red Hat OpenShift AI, the **Gen AI Studio playground** provides a chat-style interface for sending prompts to deployed models and reviewing the responses. You can test Models-as-a-Service (MaaS) endpoints in the playground using your existing subscriptions and token quotas.



IMPORTANT

Playground testing consumes tokens from your subscription's token limit. Heavy testing can deplete your quota and block subsequent production usage. Use a subscription dedicated to development if available.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- You are a member of at least one group that is granted access to a Models-as-a-Service (MaaS) subscription.
- At least one model has been published to MaaS and is included in your subscription.
- You have deployed an OGX server in your project. For more information, see [Deploying a OGX Server](#).

Procedure

1. From the OpenShift AI dashboard, click **Gen AI studio** → **AI asset endpoints**.
2. Locate the MaaS model that you want to test.
3. In the **Actions** column for that model, click the action menu (:) and then select **Try in playground**.
The playground interface opens in a new browser tab.
4. In the **Configure** panel, verify your MaaS settings:
 - a. From the **Model** dropdown, verify that the correct model is selected.
MaaS models are displayed in the format **<endpoint-name>/<model-id>**. For example, **maas-vllm-inference-1/facebook/opt-125m**.
 - b. From the **Subscription** dropdown, select the subscription to use for testing.
If you belong to multiple subscriptions, select the subscription you want to use for testing.
5. Test the model by entering a prompt in the message field and then clicking **Send**.
The model response appears in the chat interface.
6. Optional: Adjust model parameters, configure system prompts, or use additional playground features.
For information about playground features such as temperature settings, streaming responses, system prompts, model comparison, RAG integration, and code export, see [Experimenting with models in the gen AI playground](#).

Verification

- The model returns a response to your prompt.

Additional resources

- [Experimenting with models in the gen AI playground](#)
- [View subscription limits](#)

2.2.9. View your API keys

In Red Hat OpenShift AI, you can view a list of your Models-as-a-Service (MaaS) API keys in the OpenShift AI dashboard. The list shows the status, subscription, created date, last-used date, and expiration date for each key.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- Your administrator has enabled Models-as-a-Service and assigned you to a subscription.

Procedure

1. In the OpenShift AI dashboard, click **Gen AI studio** → **API keys**.
2. On the **API keys** tab, view the table with the following columns:
 - **Name:** The name assigned to the API key
 - **Status:** The current state of the key. Possible values: **Active**, **Expired**, **Revoked**.
 - **Subscription:** The subscription associated with the key
 - **Created:** The date when the key was created
 - **Last used:** The date when the key was last used to access a model
 - **Expires:** The expiration date for the key
3. Optional: Filter the list of API keys by status using the **Status** dropdown and selecting **Active**, **Expired**, or **Revoked**.
4. Optional: Sort the table by clicking any column header.

Verification

- On the **API keys** tab, verify that the table displays your API keys with columns for name, status, subscription, created date, last-used date, and expiration date.
- If you applied status filters, verify that the table shows only API keys matching the selected status.

2.2.10. Create an API key

In Red Hat OpenShift AI, you can create Models-as-a-Service (MaaS) API keys to authenticate inference requests to large language models.



IMPORTANT

Your group membership is captured at API key creation time. If your group membership changes after the key is created, the key retains the original group associations. To reflect updated group membership, revoke the existing key and create a new one.

Prerequisites

- You are logged in to the OpenShift AI dashboard.
- You authenticate to OpenShift AI using OpenShift authentication. External OIDC users create API keys through the MaaS API using **curl** or other HTTP clients, not through the dashboard.
- Your administrator has deployed Models-as-a-Service.
- You are a member of at least one OpenShift group that is included in a MaaS subscription.

Procedure

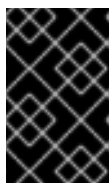
1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
2. On the **API keys** tab, click **Create API key**.
3. In the **Create API key** dialog, configure the following settings:
 - a. In the **Name** field, enter a descriptive name for the API key.
 - b. Optional: In the **Description** field, enter additional details about the key's purpose.
 - c. From the **Subscription** dropdown, select the subscription that determines which models the key can access and the applicable token limits.
The **Models** section displays the models included in the selected subscription and their configured token limits.
 - d. From the **Expiration** dropdown, select when the key expires: 30 days, 60 days, 90 days (default), 180 days, 1 year, or **Custom (days)**. If you select **Custom (days)**, enter a value between 1 and 365.



NOTE

Your administrator can set a maximum expiration limit in the **Tenant** custom resource. If not set, the default maximum is 90 days.

4. Click **Create**.
The **API key created** dialog displays the generated key with a prefix of **sk-oai-**.



IMPORTANT

The plaintext key is displayed only during creation and cannot be retrieved later. Save the key in a secrets manager before closing the **API key created** dialog. If you lose the key, you must revoke it and create a new one.

5. Click the copy icon next to the API key field to copy the key, and then save it in a secure location for use in applications.

6. Click **Close**.

Verification

1. In the OpenShift AI dashboard, navigate to **Gen AI studio → API keys**.
2. Verify that the new API key appears in the table.
3. Confirm that the **Status** column displays **Active** with a green checkmark.
4. Verify that the **Subscription** column shows a subscription that includes the models you want to access.
5. Check that the **Expires** column displays the correct expiration date based on the number of days you selected.
6. Optional: Test the API key by listing the models available through your subscription:

```
$ curl -H "Authorization: Bearer <your-api-key>" \
https://<maas-gateway-url>/maas-api/v1/models
```

The command uses the following placeholders:

<your-api-key>

Specifies the API key you created.

<maas-gateway-url>

Specifies your MaaS gateway URL.

The response lists the models accessible through your subscription in JSON format.

Next steps

- [Make API calls to models](#)

2.2.11. Revoke your API key

In Red Hat OpenShift AI, you can revoke one of your Models-as-a-Service (MaaS) API keys if you no longer need it or suspect that it has been compromised.



IMPORTANT

Revoking an API key is permanent and cannot be undone. Applications and services using the revoked key lose access and receive **401 Unauthorized** responses.

Prerequisites

- You have access to the OpenShift AI dashboard.
- You have at least one API key.
- The key you want to revoke has not already been revoked or expired.

Procedure

To revoke an individual API key:

1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
2. On the **API keys** tab, in the row for the API key you want to revoke, click the action menu (⋮) and select **Revoke**.
3. In the **Revoke API key?** dialog, review the warning that revocation is permanent.
4. Enter the API key name to confirm.
5. Click **Revoke**.

To revoke all your API keys:

1. In the OpenShift AI dashboard, click **Gen AI studio → API keys**.
2. On the **API keys** tab, click the action menu (⋮) and select **Revoke all my keys**.
3. In the dialog, review the warning that revocation is permanent.
4. Type your username to confirm the revocation.
5. Click **Revoke all keys**.

Verification

- The API key shows a **Revoked** status in the API keys table. The revoked key remains visible in the table but can no longer be used for authentication.
- Attempting to use the revoked key in an API request returns a **401 Unauthorized** response with the error code **invalid_api_key**.

Next steps

- [Create an API key](#)
- [View your API keys](#)

2.2.12. Best practices for Models-as-a-Service

As a OpenShift AI user, follow these recommendations to effectively and securely use Models-as-a-Service (MaaS).

2.2.12.1. API key security

- Never commit API keys to version control. Store API keys in environment variables, Kubernetes secrets, or secret management systems such as HashiCorp Vault.
- Use descriptive names for API keys. Create separate keys for different applications or purposes such as **production-chatbot**, **dev-testing**, or **notebook-experiments** to track usage and simplify revocation.
- Rotate API keys periodically. Generate new keys regularly and revoke old ones, especially for long-lived keys used in production applications.

- Revoke compromised keys immediately. If an API key is exposed or no longer needed, revoke it through the dashboard to prevent unauthorized access.
- Set appropriate expiration dates. Use shorter expiration periods for development and testing, and coordinate with your team for production key rotation schedules.

2.2.12.2. Quota management

- Monitor your subscription limits. Check your subscription quotas in the dashboard to understand your available token limits and avoid unexpected quota exhaustion.
- Check rate limit headers in responses. Review the **X-RateLimit-Remaining** header in API responses to track your usage against subscription limits in real time.
- Plan for quota exhaustion. Implement graceful error handling when quotas are depleted, such as queuing requests or displaying user-friendly messages.
- Choose subscriptions wisely. If you belong to multiple subscriptions with access to the same model, select the appropriate subscription for your use case when creating API keys or testing in the playground.

2.2.12.3. Token optimization

- Set reasonable **max_tokens** values. Request only as many tokens as you need for your use case. Avoid setting excessively high limits that waste quota.
- Optimize prompts for efficiency. Shorter, more focused prompts often produce better results while consuming fewer tokens than verbose or repetitive prompts.
- Use system prompts effectively. Configure consistent model behavior through system prompts rather than repeating instructions in every user message.
- Avoid redundant context. Send only the conversation history that the model needs to respond to the current request.
- Test prompts before deploying. Use the playground to refine prompts and verify token consumption before implementing them in production code.

2.2.12.4. Performance and reliability

- Implement retry logic with exponential backoff. Handle rate limit errors (HTTP 429) and temporary failures gracefully by retrying requests with increasing delays.
- Cache responses when appropriate. If you make identical requests repeatedly, cache the results to reduce token consumption and improve response time.
- Use streaming for interactive applications. Enable streaming responses to provide faster time-to-first-token and better user experience for chat-based applications.
- Handle errors gracefully. Implement proper error handling for rate limits, quota exhaustion, network errors, and model timeouts.

2.2.12.5. Testing and development

- Test in the playground first. Use the playground to validate model responses, compare models, and experiment with parameters before writing code.

- Export playground configurations. Use the playground's code export feature to generate starter code templates with your tested prompts and parameters.
- Use temporary API keys for development. Generate short-lived API keys for testing and experimentation to minimize security risk if keys are accidentally exposed.
- Validate model responses. Implement response validation in your application to ensure the model's output meets your requirements and handles edge cases.
- Test with multiple models. If multiple models are available in your subscription, test different models to find the best balance of response quality, speed, and token consumption for your use case.

2.2.12.6. Multi-subscription usage

- Understand subscription priorities. If you belong to multiple subscriptions with access to the same model, the subscription with the highest priority level is used by default for API requests.
- Select subscriptions explicitly when needed. When creating API keys or testing in the playground, explicitly choose which subscription to use rather than relying on automatic priority-based selection.
- Separate development and production usage. If possible, use different subscriptions for development, testing, and production to isolate quota consumption and costs.
- Track your usage per subscription. Monitor your token consumption separately for each subscription you belong to, especially if subscriptions have different usage policies or billing.

2.2.12.7. Endpoint selection

- Use body-based routing for new applications and SDK integrations. The standard **/v1/chat/completions** endpoint enables drop-in compatibility with OpenAI-compatible clients, requiring no custom URL paths or client-side modifications.
- Use path-based routing only for backwards compatibility with existing integrations that already use per-model URL paths.
- When using body-based routing with the Python **openai** SDK, set **base_url** to **https://<maas-gateway-url>/v1**. The SDK automatically appends the correct endpoint path.
- The model name specified in the request body must match a model name returned by the **/v1/models** endpoint. Aliases and short names are not supported.

2.2.12.8. Production deployment

- Use descriptive API key names. Create separate API keys for each production application or service to support usage tracking and selective revocation.
- Implement monitoring and logging. Log API requests and responses (excluding sensitive data) to support debugging, usage analysis, and compliance requirements.
- Set up alerting for quota limits. Monitor your subscription quota consumption and alert your team when approaching limits to avoid service disruptions.
- Coordinate with administrators. If your usage patterns change or you need higher quotas, contact your administrator to request subscription adjustments.

Additional resources

- [View subscription limits](#)
- [Create API keys](#)
- [Make API calls to Models-as-a-Service \(MaaS\) models](#)
- [Experimenting with models in the gen AI playground](#)

Additional resources

- [Models-as-a-Service user access troubleshooting](#)

2.3. MODELS-AS-A-SERVICE USER ACCESS TROUBLESHOOTING

As a OpenShift AI user, you can diagnose and resolve common issues when accessing models through Models-as-a-Service (MaaS).

2.3.1. Authentication errors: 401 Unauthorized

Symptom:

```
{
  "error": {
    "message": "Invalid or expired API key",
    "type": "authentication_error",
    "code": 401
  }
}
```

Possible causes and solutions:

- **API key expired:** If your API key has expired, create a new one from the **API keys** page.
- **API key revoked:** Check if the API key has been revoked. Create a new API key if necessary.
- **Incorrect API key format:** Ensure you're using the **Authorization: Bearer <key>** header format with the full API key including the **sk-oai-** prefix.
- **Using wrong authentication method:** Generate a MaaS API key from the dashboard. OpenShift tokens are not valid for MaaS authentication.

2.3.2. Authorization errors: 403 Forbidden

Symptom:

```
{
  "error": {
    "message": "Access denied. Your subscription does not have permission to access this model.",
    "type": "authorization_error",
    "code": 403
  }
}
```

Possible causes and solutions:

- **Model not included in your subscription** Contact your administrator to request that the model be added to your subscription.
- **No authorization policy exists** Ask your administrator to verify that an authorization policy exists for your groups to access the model through the API gateway.

2.3.3. Model not found: 404

Symptom:

```
{
  "error": {
    "message": "Model not found",
    "type": "not_found_error",
    "code": 404
  }
}
```

Possible causes and solutions:

- **Incorrect model name:** Verify the model name using the `/v1/models` endpoint.
- **Model not deployed:** Ask your administrator to check if the model is deployed and ready.
- **Typo in URL:** Ensure the URL format is correct. For body-based routing, use `\https://maas.<domain>/v1/chat/completions` with the model name in the request body. For legacy path-based routing, use `\https://maas.<domain>/llm/<model-name>/v1/chat/completions`.

2.3.4. Exceeded token limits

If you exceed the maximum number of tokens allowed per time period for a model in your subscription:

Example error response:

```
{
  "error": {
    "message": "Token limit exceeded. You have consumed the maximum number of tokens allowed for this model in your subscription.",
    "type": "token_limit_error",
    "code": 429
  }
}
```

Response headers for retry logic:

```
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1234567890
Retry-After: 42
```

Handling token limits in Python:

```

import time
import requests

def make_request_with_retry(url, headers, data, max_retries=3):
    for attempt in range(max_retries):
        response = requests.post(url, headers=headers, json=data)

        if response.status_code == 429:
            retry_after = int(response.headers.get('Retry-After', 60))
            print(f"Token limit exceeded. Retrying after {retry_after} seconds...")
            time.sleep(retry_after)
            continue

    return response

raise Exception("Max retries exceeded")

```

Mitigation strategies:

- Reduce **max_tokens** in requests to consume fewer tokens per request
- Implement exponential backoff retry logic to handle temporary limit violations
- Batch requests with longer delays between them to spread token consumption over time
- Contact your administrator to request a subscription update with higher token limits if you consistently hit limits

2.3.5. Persistent token limit errors

Symptom: You continue to receive 429 errors even after waiting.

Possible causes and solutions:

- **Subscription token limits exhausted:** Check token limits for your subscription for the model you are trying to access. Wait for the time period to reset or contact your administrator for a subscription update.
- **Incorrect retry logic:** Ensure you're respecting the **Retry-After** header value.
- **Multiple applications using the same API key** Each application should have its own API key to better track and manage token consumption.

Additional resources

- [Multi-provider API passthrough for external models](#)